VERILOG AND SYSTEM VERILOG (INTERVIEWS QUESTIONS)

Q1.0 Verilog Code: 10 Random Numbers using LFSR

LFSR (Linear Feedback Shift Register)

```
//
module random_10_numbers;
reg clk;
reg rst;
reg [7:0] lfsr;

integer count;

initial clk=0;
always #5 clk =~clk;

always @(posedge clk or posedge rst) begin
        if (rst)
                lfsr <= 8'hA5;
        else
                lfsr <= { lfsr[6:0],lfsr[7]^lfsr[5]^lfsr[4]^lfsr[3] };
end

initial begin
        rst = 1;
        count = 0;
        #10 rst = 0 ;

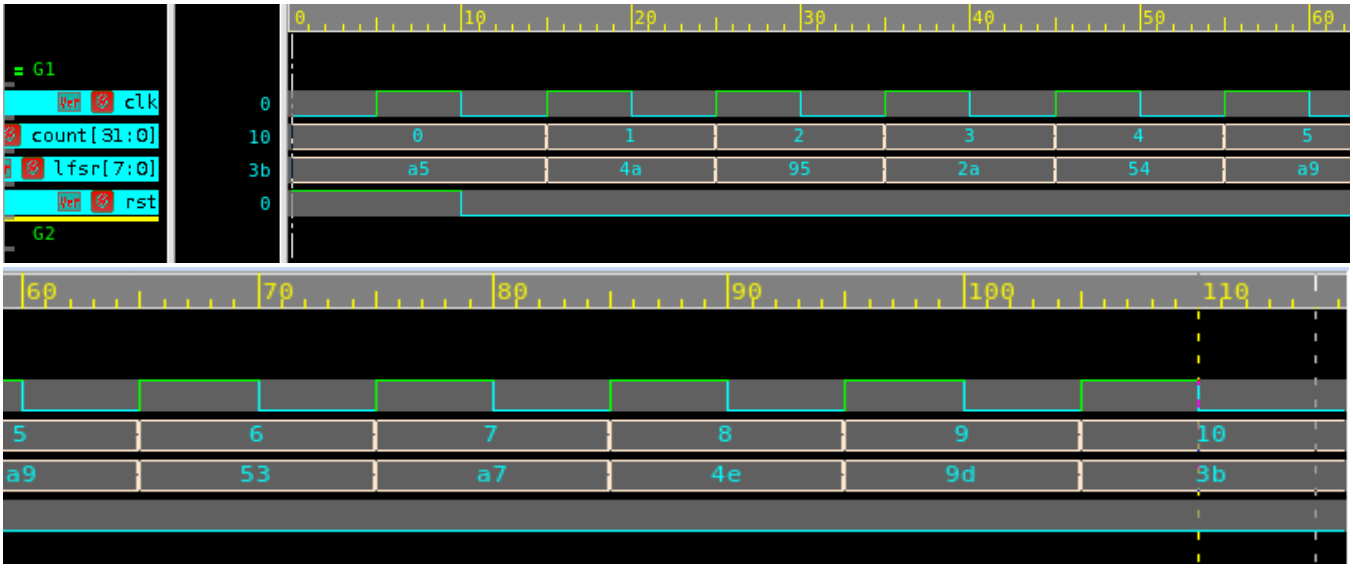
        repeat(10) begin
```

```
            @(posedge clk);
            $display("Random numbers %0d = %0d ", count+1,lfsr);
            count = count +1;
        end

        #10 $finish;
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,random_10_numbers);
end
endmodule
```

Simulation Output:

```
Random numbers 1 = 165
Random numbers 2 = 74
Random numbers 3 = 149
Random numbers 4 = 42
Random numbers 5 = 84
Random numbers 6 = 169
Random numbers 7 = 83
Random numbers 8 = 167
Random numbers 9 = 78
Random numbers 10 = 157
$finish called from file "practice.v", line 30.
$finish at simulation time                 115
         V C S  S i m u l a t i o n  R e p o r t
Time: 115
```



Q2.0 Generate 10 Random Numbers using $random

```
module random_10_numbers;

integer i;

integer rand_num;


initial begin

    for (i=0;i<10;i=i+1) begin

        rand_num=$random;

        $display("Printing the Random number %0d = %0d", i+1, rand_num);

    end

    #10 $finish();

end

initial begin

    $fsdbDumpfile("dump.fsdb");

    $fsdbDumpvars(0,random_10_numbers);

end

endmodule
```

Simulation Output:

Time=5 Printing the Random number 1 = 303379748

Time=10 Printing the Random number 2 = -1064739199

Time=15 Printing the Random number 3 = -2071669239

Time=20 Printing the Random number 4 = -1309649309

Time=25 Printing the Random number 5 = 112818957

Time=30 Printing the Random number 6 = 1189058957

Time=35 Printing the Random number 7 = -1295874971

Time=40 Printing the Random number 8 = -1992863214

Time=45 Printing the Random number 9 = 15983361

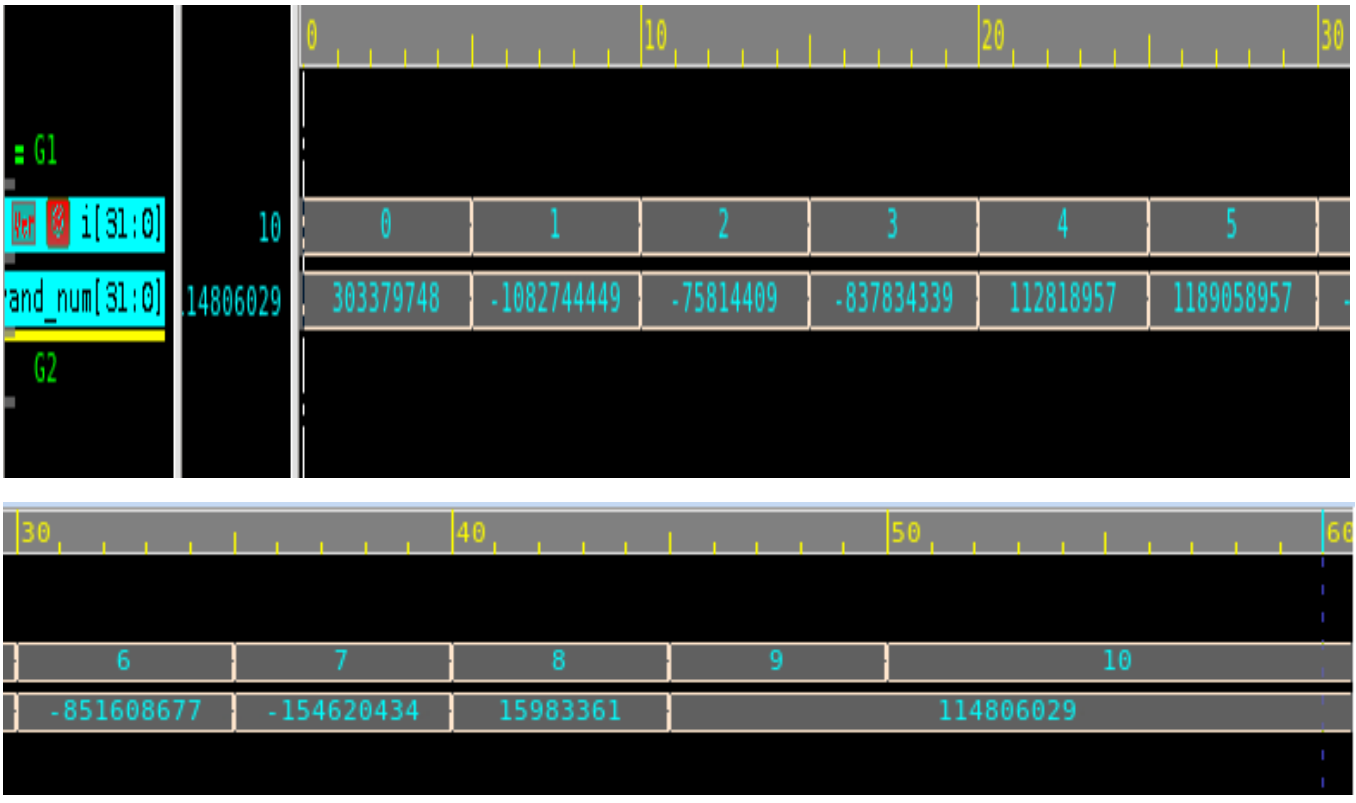
Time=50 Printing the Random number 10 = 114806029

$finish called from file "practice.v", line 13.

$finish at simulation time 60

V C S S i m u l a t i o n R e p o r t

Time: 60



Q3.0 Generate 10 Unsigned Random Numbers using $random

```
module random_10_unsign_numbers;
integer i;
reg [31:0] rand_u_num;

initial begin
    for (i=0;i<10;i=i+1) begin
        rand_u_num=$random;    //#5
        $display("Printing Unsigned Random number %0d = %0d", i+1,rand_u_num);
    end
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,random_10_unsign_numbers);
end

endmodule
```

Simulation Output:

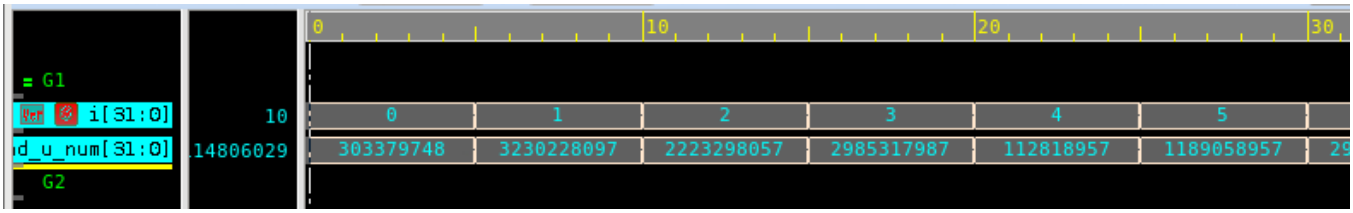
```
Printing Unsigned Random number 1 = 303379748
Printing Unsigned Random number 2 = 3230228097
Printing Unsigned Random number 3 = 2223298057
Printing Unsigned Random number 4 = 2985317987
Printing Unsigned Random number 5 = 112818957
Printing Unsigned Random number 6 = 1189058957
Printing Unsigned Random number 7 = 2999092325
```

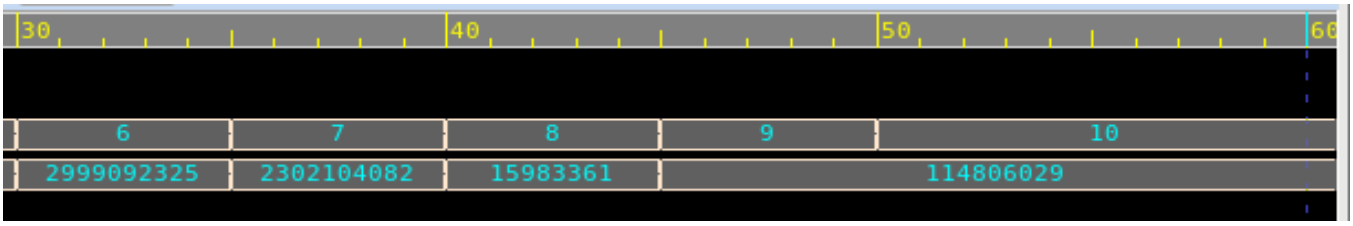
Printing Unsigned Random number 8 = 2302104082

Printing Unsigned Random number 9 = 15983361

Printing Unsigned Random number 10 = 114806029

- **Even though $random returns** signed**, assigning to a** bit-vector **makes it** unsigned**.**
- **Stored data as a unsigned.**





Q4.0 Generate 10 Even Random Numbers using $random

```
module print_10_even_random;

integer i;

integer rand_num;

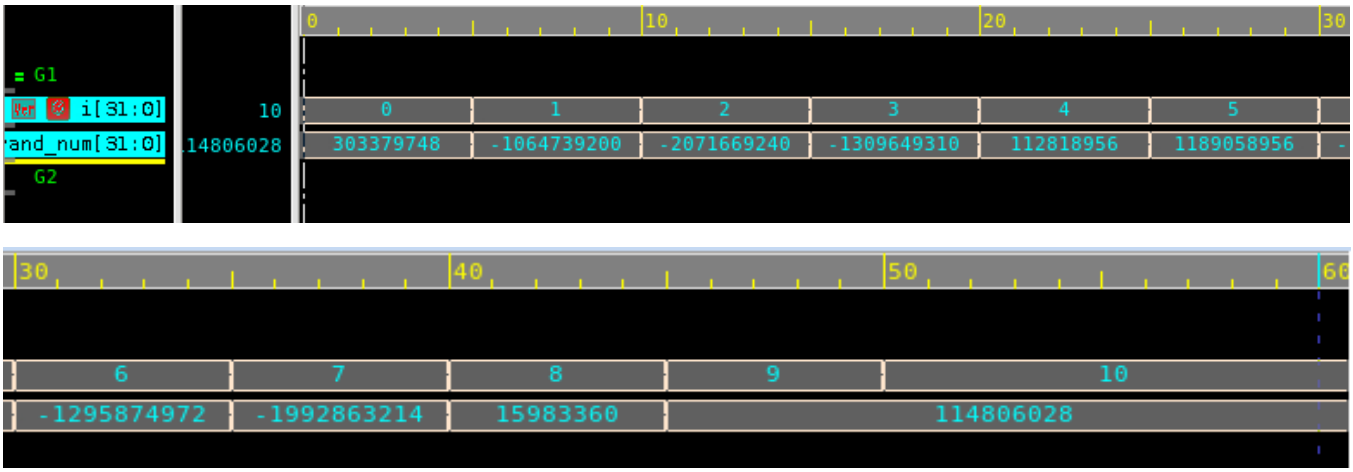

initial begin
    for (i=0;i<10;i=i+1) begin
        rand_num=$random;


        rand_num = rand_num & ~1;
        #5;
        $display("Time=%0t Printing Even number %0d = %0d",$time,i+1,rand_num);
    end
    #10 $finish();
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,print_10_even_random);
end
endmodule
```

Simulation Output:

```
Time=5 Printing Even number 1 = 303379748
Time=10 Printing Even number 2 = -1064739200
Time=15 Printing Even number 3 = -2071669240
Time=20 Printing Even number 4 = -1309649310
Time=25 Printing Even number 5 = 112818956
Time=30 Printing Even number 6 = 1189058956
Time=35 Printing Even number 7 = -1295874972
Time=40 Printing Even number 8 = -1992863214
Time=45 Printing Even number 9 = 15983360
Time=50 Printing Even number 10 = 114806028
$finish called from file "practice.v", line 14.
$finish at simulation time                   60
           V C S   S i m u l a t i o n   R e p o r t
Time: 60
```



- **“By forcing the LSB to zero using bitwise AND.” LSB=0**

Q5.0 Generate 10 Even Numbers using without $random

```
module print_10_even_numbers;
integer i;
integer even;

initial begin
    even = 0;
    for (i=0;i<10;i=i+1) begin

        #5;
        $display("Time=%0t Printing Even number %0d = %0d",$time,i+1,even);
        even = even +2;
    end
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Printing Even number 1 = 0
Time=10 Printing Even number 2 = 2
Time=15 Printing Even number 3 = 4
Time=20 Printing Even number 4 = 6
Time=25 Printing Even number 5 = 8
```

Time=30 Printing Even number 6 = 10

Time=35 Printing Even number 7 = 12

Time=40 Printing Even number 8 = 14

Time=45 Printing Even number 9 = 16

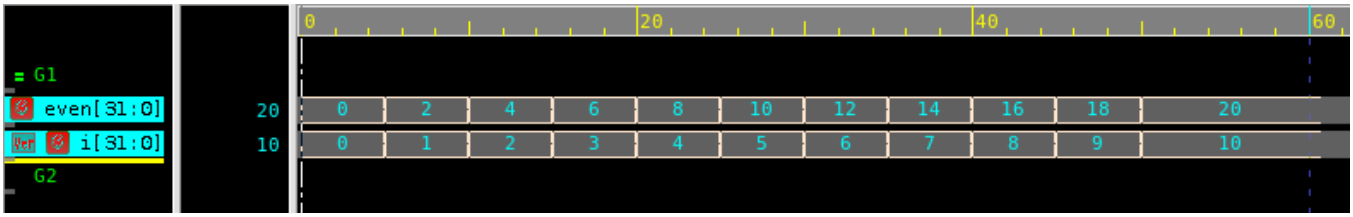
Time=50 Printing Even number 10 = 18

$finish called from file "practice.v", line 14.

$finish at simulation time 60

V C S S i m u l a t i o n R e p o r t

Time: 60



Q6.0 Generate 10 Even Numbers using $random range (0-100)

```
module print_10_even_numbers;
integer i;
reg [31:0] rand_num;

initial begin

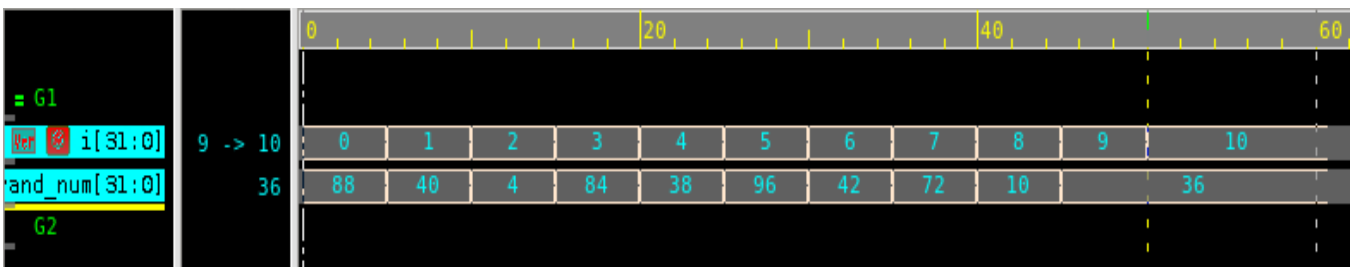
    for (i=0;i<10;i=i+1) begin
        rand_num =$random;
        rand_num = rand_num % 101;
        rand_num = rand_num & ~1;

        #5;
        $display("Time=%0t Printing Even number %0d = %0d",$time,i+1,rand_num);
```

```
        end
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,print_10_even_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Printing Even number 1 = 88
Time=10 Printing Even number 2 = 40
Time=15 Printing Even number 3 = 4
Time=20 Printing Even number 4 = 84
Time=25 Printing Even number 5 = 38
Time=30 Printing Even number 6 = 96
Time=35 Printing Even number 7 = 42
Time=40 Printing Even number 8 = 72
Time=45 Printing Even number 9 = 10
Time=50 Printing Even number 10 = 36
$finish called from file "practice.v", line 16.
$finish at simulation time                   60
          V C S   S i m u l a t i o n   R e p o r t
Time: 60
```



Q7.0 Generate 10 Even Numbers using $random range (0-100)

```
module print_10_even_numbers;
integer i;
integer rand_num;

initial begin

    for (i=0;i<10;i=i+1) begin
        rand_num =$random;
        rand_num = rand_num & 32'h7FFF_FFFF;
        rand_num = rand_num % 101;
        rand_num = rand_num & ~1;

        #5;
      $display("Time=%0t Printing Even number %0d = %0d",$time,i+1,rand_num);
    end
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Printing Even number 1 = 88
Time=10 Printing Even number 2 = 6
```

Time=15 Printing Even number 3 = 72

Time=20 Printing Even number 4 = 50

Time=25 Printing Even number 5 = 38

Time=30 Printing Even number 6 = 96

Time=35 Printing Even number 7 = 8

Time=40 Printing Even number 8 = 38

Time=45 Printing Even number 9 = 10

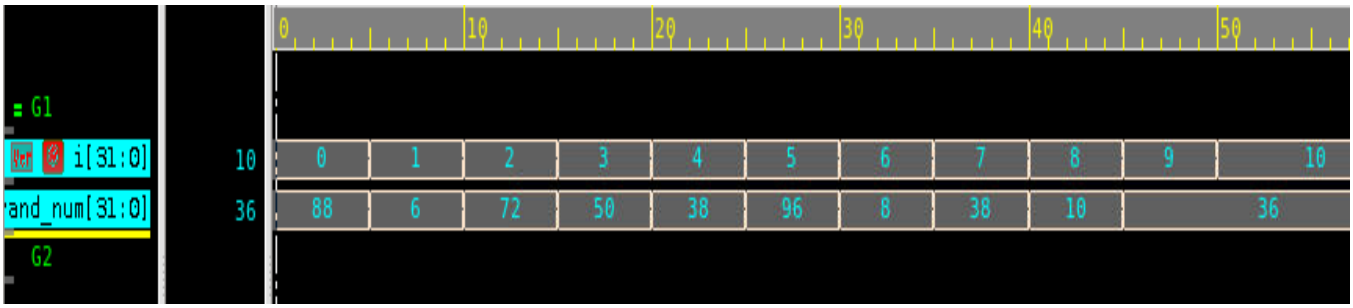
Time=50 Printing Even number 10 = 36

$finish called from file "practice.v", line 17.

$finish at simulation time 60

V C S S i m u l a t i o n R e p o r t

Time: 60



Q8.0 Generate 10 Even Numbers using for loop and if condition:

```
module print_10_even_numbers;
integer i;
integer counts;

initial begin
    counts = 0;
  for (i=0; counts <10;i=i+1) begin

      if (i % 2 ==0) begin
```

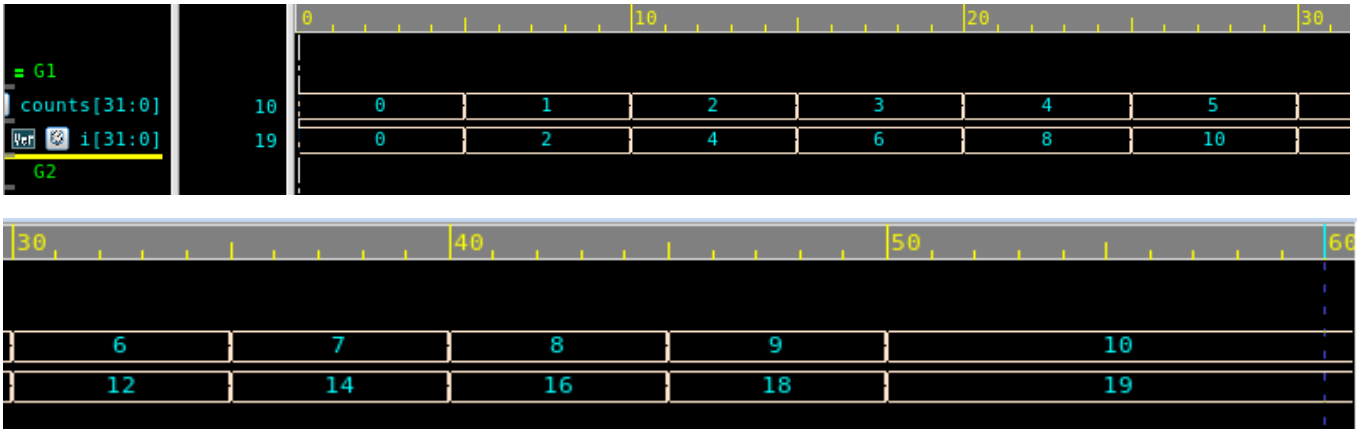
```
                #5;
                $display("Time=%0t Even number %0d = %0d",$time,counts+1,i);
                counts = counts +1;
            end


    end
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Even number 1 = 0
Time=10 Even number 2 = 2
Time=15 Even number 3 = 4
Time=20 Even number 4 = 6
Time=25 Even number 5 = 8
Time=30 Even number 6 = 10
Time=35 Even number 7 = 12
Time=40 Even number 8 = 14
Time=45 Even number 9 = 16
Time=50 Even number 10 = 18
$finish called from file "practice.v", line 17.
$finish at simulation time                60
        V C S   S i m u l a t i o n   R e p o r t
```

Time: 60



Q9.0 Generate 10 Even Numbers using for loop and check LSB bits:

```
module print_10_even_numbers;
integer i;
integer counts;

initial begin
    counts = 0;
  for (i=0; counts <10;i=i+1) begin

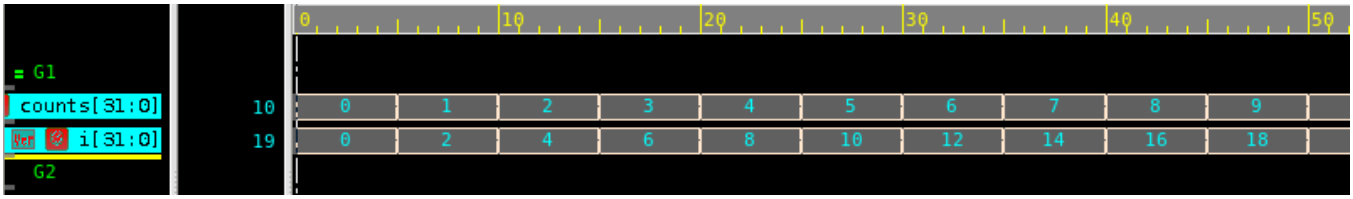
      if (i[0] == 0) begin
          #5;
          $display("Time=%0t Even number %0d = %0d",$time,counts+1,i);
          counts = counts +1;
      end

  end
  #10 $finish();
end
initial begin
```

```
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Even number 1 = 0
Time=10 Even number 2 = 2
Time=15 Even number 3 = 4
Time=20 Even number 4 = 6
Time=25 Even number 5 = 8
Time=30 Even number 6 = 10
Time=35 Even number 7 = 12
Time=40 Even number 8 = 14
Time=45 Even number 9 = 16
Time=50 Even number 10 = 18
$finish called from file "practice.v", line 17.
$finish at simulation time                   60
         V C S   S i m u l a t i o n   R e p o r t
Time: 60
```



- i[0] = **LSB (Least Significant Bit)**
- Binary rule:
- **Even number** → LSB = 0
- **Odd number** → LSB = 1
- **“Even numbers are detected by checking the LSB; if the LSB is 0, the number is even.”**

Q10.0 Generate 10 ODD Numbers using for loop and check LSB bits:

```
module print_10_odd_numbers;
integer i;
integer counts;


initial begin
        counts = 0;
      for (i=0; counts <10;i=i+1) begin


            if (i[0] == 1) begin
                  #5;
                  $display("Time=%0t Odd number %0d = %0d",$time,counts+1,i);
                  counts = counts +1;
            end


      end
      #10 $finish();
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,print_10_odd_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Odd number 1 = 1
Time=10 Odd number 2 = 3
```

Time=15 Odd number 3 = 5

Time=20 Odd number 4 = 7

Time=25 Odd number 5 = 9

Time=30 Odd number 6 = 11

Time=35 Odd number 7 = 13

Time=40 Odd number 8 = 15

Time=45 Odd number 9 = 17

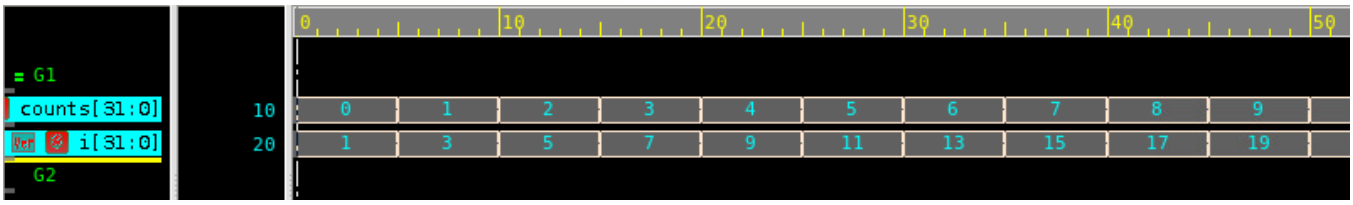
Time=50 Odd number 10 = 19

$finish called from file "practice.v", line 17.

$finish at simulation time 60

V C S S i m u l a t i o n R e p o r t

Time: 60



Q11.0 Generate 10 ODD Numbers using for loop and $random:

```
module print_10_odd_numbers;

integer i;

integer rand_num;


initial begin

    for (i=0; i <10;i=i+1) begin

        rand_num = $random;

        rand_num = rand_num | 1;


        #5;
```

```
            $display("Time=%0t Odd number %0d = %0d",$time,i+1,rand_num);

        end
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,print_10_odd_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Odd number 1 = 303379749
Time=10 Odd number 2 = -1064739199
Time=15 Odd number 3 = -2071669239
Time=20 Odd number 4 = -1309649309
Time=25 Odd number 5 = 112818957
Time=30 Odd number 6 = 1189058957
Time=35 Odd number 7 = -1295874971
Time=40 Odd number 8 = -1992863213
Time=45 Odd number 9 = 15983361
Time=50 Odd number 10 = 114806029
$finish called from file "practice.v", line 15.
$finish at simulation time                   60
          V C S   S i m u l a t i o n   R e p o r t
Time: 60
```

Q12.0 Generate 10 ODD Numbers using for loop and initialize condition odd=1:

```
module print_10_odd_numbers;
integer i;
integer odd;

initial begin
        odd = 1;
      for (i=0; i <10;i=i+1) begin

              #5;
              $display("Time=%0t Odd number %0d = %0d",$time,i+1,odd);
              odd = odd +2;

      end
      #10 $finish();
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,print_10_odd_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Odd number 1 = 1
Time=10 Odd number 2 = 3
Time=15 Odd number 3 = 5
Time=20 Odd number 4 = 7
```

Time=25 Odd number 5 = 9

Time=30 Odd number 6 = 11

Time=35 Odd number 7 = 13

Time=40 Odd number 8 = 15

Time=45 Odd number 9 = 17

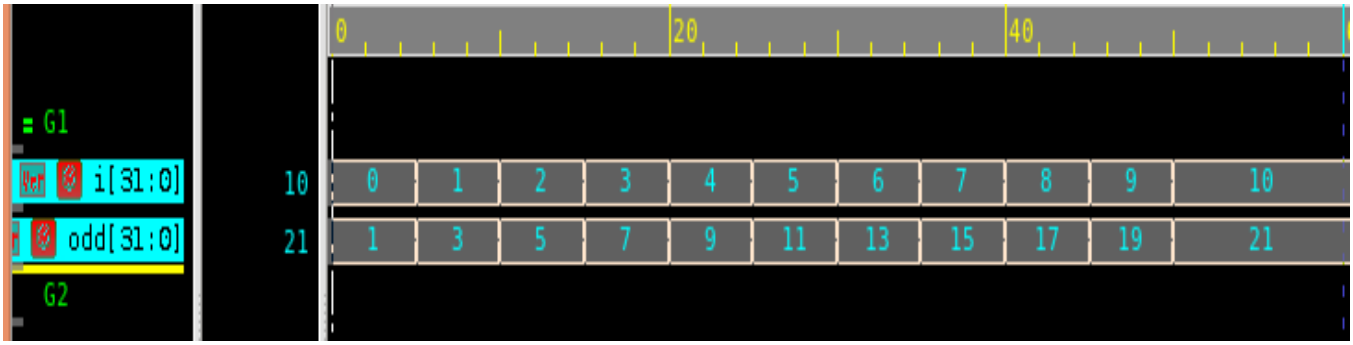
Time=50 Odd number 10 = 19

$finish called from file "practice.v", line 15.

$finish at simulation time 60

V C S S i m u l a t i o n R e p o r t

Time: 60



Q13.0 Generate 10 ODD Numbers using while loop and initialize condition:

```
//
module print_10_even_numbers;
integer num;
integer count;

initial begin
    num = 0;
    count = 0;
  while (count < 10) begin
        if (num %2 ==0) begin
```

```
            //#5;
            $display("Time=%0t Even number %0d = %0d",$time,count+1,num);
            count = count +1;
        end
        num = num +1;
        #5;
end
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_numbers);
end
endmodule
```

Simulation Output:

```
Time=0 Even number 1 = 0
Time=10 Even number 2 = 2
Time=20 Even number 3 = 4
Time=30 Even number 4 = 6
Time=40 Even number 5 = 8
Time=50 Even number 6 = 10
Time=60 Even number 7 = 12
Time=70 Even number 8 = 14
Time=80 Even number 9 = 16
Time=90 Even number 10 = 18
$finish called from file "practice.v", line 19.
```

$finish at simulation time 105

V C S S i m u l a t i o n R e p o r t

Time: 105

Q13.0 Generate 10 Even-ODD Numbers using for loop and if – else condition:

```
module print_10_even-odd_numbers;
integer i;
integer counts;

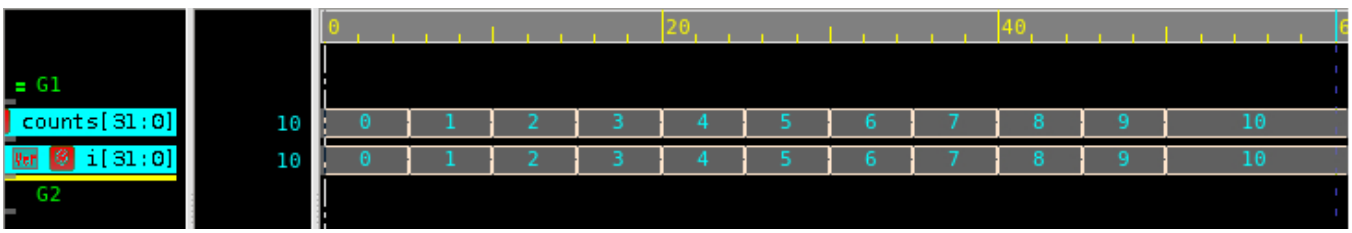
initial begin
    counts = 0;
    for (i=0; counts <10;i=i+1) begin

        if (i[0] == 0) begin
            #5;
            $display("Time=%0t Even number [%0d] = %0d",$time,counts,i);
            counts = counts +1;
        end
        else begin
            #5;
            $display("Time=%0t Odd number [%0d] = %0d",$time,counts,i);
            counts = counts +1;
        end
```

```
    end
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even-odd_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Even number [0] = 0
Time=10 Odd number [1] = 1
Time=15 Even number [2] = 2
Time=20 Odd number [3] = 3
Time=25 Even number [4] = 4
Time=30 Odd number [5] = 5
Time=35 Even number [6] = 6
Time=40 Odd number [7] = 7
Time=45 Even number [8] = 8
Time=50 Odd number [9] = 9
$finish called from file "practice.v", line 22.
$finish at simulation time                  60
          V C S   S i m u l a t i o n   R e p o r t
Time: 60
```



Q14.0 Generate 10 Even-ODD Numbers using for loop and if – else condition:

```
module print_10_even-odd_numbers;
integer i;


initial begin
	for (i=0; i <10;i=i+1) begin


		if (i[0] == 0) begin
			#5;
			$display("Time=%0t Even number [%0d] = %0d",$time,i,i);
		end
		else begin
			#5;
			$display("Time=%0t Odd number [%0d] = %0d",$time,i,i);
		end


	end
	#10 $finish();
end
initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,print_10_even-odd_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Even number [0] = 0
```

```
Time=10 Odd number [1] = 1
Time=15 Even number [2] = 2
Time=20 Odd number [3] = 3
Time=25 Even number [4] = 4
Time=30 Odd number [5] = 5
Time=35 Even number [6] = 6
Time=40 Odd number [7] = 7
Time=45 Even number [8] = 8
Time=50 Odd number [9] = 9
$finish called from file "practice.v", line 18.
$finish at simulation time                   60
         V C S   S i m u l a t i o n   R e p o r t
Time: 60
```

Q15.0 Generate 10 Even-ODD Numbers using for loop and if – else condition:

```
module print_10__numbers;
integer i;

initial begin
      for (i=0; i <10;i=i+1) begin
            #5;
            $display("Time=%0t Even number [%0d] = %0d",$time,i,2*i);
        end

        for (i=0; i <10 ; i=i+1) begin
            #5;
            $display("Time=%0t Odd number [%0d] = %0d",$time,i,(2*i+1));
```

```
        end
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,print_10__numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Even number [0] = 0
Time=10 Even number [1] = 2
Time=15 Even number [2] = 4
Time=20 Even number [3] = 6
Time=25 Even number [4] = 8
Time=30 Even number [5] = 10
Time=35 Even number [6] = 12
Time=40 Even number [7] = 14
Time=45 Even number [8] = 16
Time=50 Even number [9] = 18
Time=55 Odd number [0] = 1
Time=60 Odd number [1] = 3
Time=65 Odd number [2] = 5
Time=70 Odd number [3] = 7
Time=75 Odd number [4] = 9
Time=80 Odd number [5] = 11
Time=85 Odd number [6] = 13
```

```
Time=90 Odd number [7] = 15
Time=95 Odd number [8] = 17
Time=100 Odd number [9] = 19
$finish called from file "practice.v", line 16.
$finish at simulation time                110
           V C S   S i m u l a t i o n   R e p o r t
Time: 110
```

Q16.0 Generate 10 Even-ODD Numbers using for loop condition:

```
module print_even_odd_numbers;
integer i;
integer even;
integer odd;

initial begin
        even = 0;
        odd = 1;
        $display(" EVEN NUMBERS");
      for (i=0; i <10;i=i+1) begin
                #5;
                $display("Time=%0t Even number %0d = %0d",$time,i+1, even);
                even = even +2 ;
        end

      $display(" ODD NUMBERS");
        for (i=0; i <10 ; i=i+1) begin
                #5;
```

```
            $display("Time=%0t Odd number %0d = %0d",$time,i+1, odd);
            odd = odd +2 ;
        end

        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,print_even_odd_numbers);
end
endmodule
```

Simulation Output:

```
EVEN NUMBERS
Time=5 Even number 1 = 0
Time=10 Even number 2 = 2
Time=15 Even number 3 = 4
Time=20 Even number 4 = 6
Time=25 Even number 5 = 8
Time=30 Even number 6 = 10
Time=35 Even number 7 = 12
Time=40 Even number 8 = 14
Time=45 Even number 9 = 16
Time=50 Even number 10 = 18
 ODD NUMBERS
Time=55 Odd number 1 = 1
Time=60 Odd number 2 = 3
Time=65 Odd number 3 = 5
```

Time=70 Odd number 4 = 7

Time=75 Odd number 5 = 9

Time=80 Odd number 6 = 11

Time=85 Odd number 7 = 13

Time=90 Odd number 8 = 15

Time=95 Odd number 9 = 17

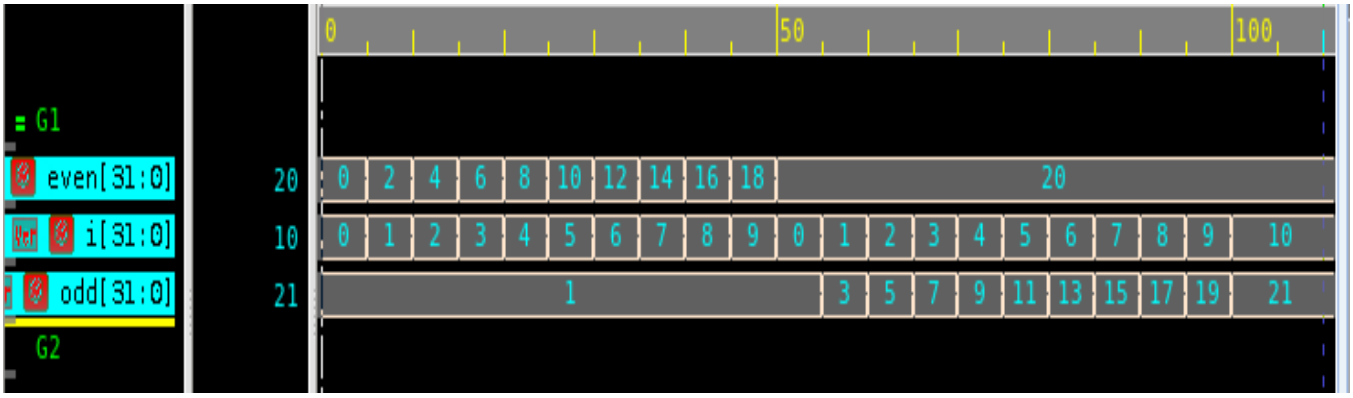
Time=100 Odd number 10 = 19

$finish called from file "practice.v", line 25.

$finish at simulation time 110

V C S S i m u l a t i o n R e p o r t

Time: 110



Q17.0 Generate 10 Even-ODD Numbers using for loop and if – else condition:

```
module print_10_even_odd_numbers;

integer i;

integer even_count;

integer odd_count;


initial begin

        even_count = 0;

        odd_count = 0;
```

```
        $display(" EVEN NUMBERS");

    for (i=0; even_count <10;i=i+1) begin
            if (i[0] ==0) begin
                #5;
                $display("Time=%0t Even number %0d =
%0d",$time,even_count+1,i);
                even_count = even_count +1 ;
            end
        end

        $display(" ODD NUMBERS");

        for (i=0; odd_count <10 ; i=i+1) begin
            if (i[0] ==1) begin
                #5;
                $display("Time=%0t Odd number %0d = %0d",$time,odd_count+1,i);
                odd_count = odd_count +1 ;
            end
        end

    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_odd_numbers);
end
endmodule
```

Simulation Output:

EVEN NUMBERS

Time=5 Even number 1 = 0

Time=10 Even number 2 = 2

Time=15 Even number 3 = 4

Time=20 Even number 4 = 6

Time=25 Even number 5 = 8

Time=30 Even number 6 = 10

Time=35 Even number 7 = 12

Time=40 Even number 8 = 14

Time=45 Even number 9 = 16

Time=50 Even number 10 = 18

ODD NUMBERS

Time=55 Odd number 1 = 1

Time=60 Odd number 2 = 3

Time=65 Odd number 3 = 5

Time=70 Odd number 4 = 7

Time=75 Odd number 5 = 9

Time=80 Odd number 6 = 11

Time=85 Odd number 7 = 13

Time=90 Odd number 8 = 15

Time=95 Odd number 9 = 17

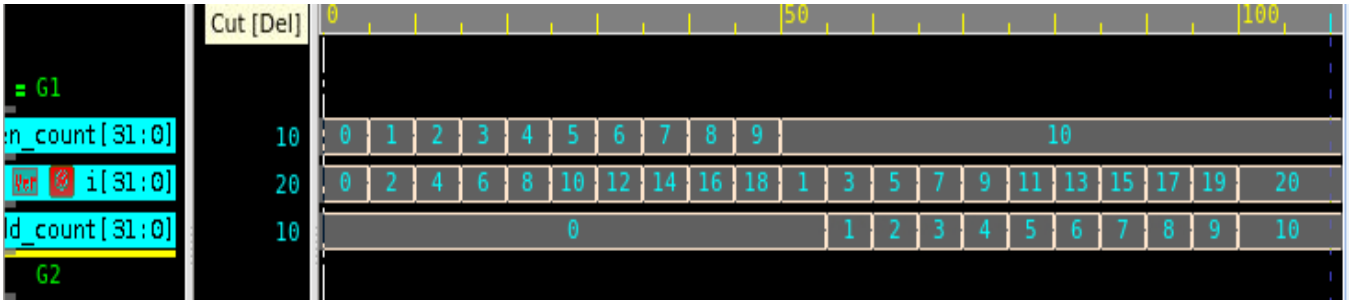
Time=100 Odd number 10 = 19

$finish called from file "practice.v", line 30.

$finish at simulation time 110

V C S S i m u l a t i o n R e p o r t

Time: 110



Q18.0 Generate 10 Even-ODD Numbers using for loop and if – else condition:

```
module print_10_even_odd_numbers;

integer i;

integer even_count;

integer odd_count;


initial begin

        even_count = 0;

        odd_count = 0;


      for (i=0; (even_count <10) || (odd_count<10);i=i+1) begin

                #5;


                if ((i[0] ==0) && (even_count<10)) begin

                        $display("Time=%0t Even number [%0d] =
%0d",$time,even_count+1,i);

                        even_count = even_count +1 ;

                end


                else if ((i[0] ==1) && (odd_count<10)) begin

                        $display("Time=%0t Odd number [%0d] =
%0d",$time,odd_count+1,i);
```

```
                odd_count = odd_count +1 ;
            end
        end

    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_10_even_odd_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 Even number [1] = 0
Time=10 Odd number [1] = 1
Time=15 Even number [2] = 2
Time=20 Odd number [2] = 3
Time=25 Even number [3] = 4
Time=30 Odd number [3] = 5
Time=35 Even number [4] = 6
Time=40 Odd number [4] = 7
Time=45 Even number [5] = 8
Time=50 Odd number [5] = 9
Time=55 Even number [6] = 10
Time=60 Odd number [6] = 11
Time=65 Even number [7] = 12
Time=70 Odd number [7] = 13
Time=75 Even number [8] = 14
```

Time=80 Odd number [8] = 15

Time=85 Even number [9] = 16

Time=90 Odd number [9] = 17

Time=95 Even number [10] = 18

Time=100 Odd number [10] = 19

$finish called from file "practice.v", line 25.

$finish at simulation time 110

V C S S i m u l a t i o n R e p o r t

Time: 110

Q19.0 Generate 10 Even-numbers at Even index by using for loop and if – else condition:

```
module print_data_even_index_numbers;
integer i;
integer count;

initial begin
        count = 0;
      for (i=0; count <10;i=i+1) begin

              if (i[0] ==0) begin
                      #5;
                      $display("Time=%0t EVEN NUMBER Index[%0d] =DATA
%0d",$time,i,i);

                      count = count +1 ;
              end
        end
```

```
    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_data_even_index_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 EVEN NUMBER Index[0] =DATA 0
Time=10 EVEN NUMBER Index[2] =DATA 2
Time=15 EVEN NUMBER Index[4] =DATA 4
Time=20 EVEN NUMBER Index[6] =DATA 6
Time=25 EVEN NUMBER Index[8] =DATA 8
Time=30 EVEN NUMBER Index[10] =DATA 10
Time=35 EVEN NUMBER Index[12] =DATA 12
Time=40 EVEN NUMBER Index[14] =DATA 14
Time=45 EVEN NUMBER Index[16] =DATA 16
Time=50 EVEN NUMBER Index[18] =DATA 18
$finish called from file "practice.v", line 19.
$finish at simulation time                   60
        V C S  S i m u l a t i o n  R e p o r
```

Q20.0 Generate 10 Odd DATA at ODD-index by using for loop and if – else condition:

```
module print_data_odd_index_numbers;
integer i;
integer count;
```

```
initial begin
        count = 0;

      for (i=0; count <10;i=i+1) begin

              if (i[0] ==1) begin
                    #5;
                    $display("Time=%0t ODD NUMBER Index[%0d] =DATA
%0d",$time,i,i);

                    count = count +1 ;
              end
        end

      #10 $finish();
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,print_data_odd_index_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 ODD NUMBER Index[1] =DATA 1
Time=10 ODD NUMBER Index[3] =DATA 3
Time=15 ODD NUMBER Index[5] =DATA 5
Time=20 ODD NUMBER Index[7] =DATA 7
Time=25 ODD NUMBER Index[9] =DATA 9
```

Time=30 ODD NUMBER Index[11] =DATA 11

Time=35 ODD NUMBER Index[13] =DATA 13

Time=40 ODD NUMBER Index[15] =DATA 15

Time=45 ODD NUMBER Index[17] =DATA 17

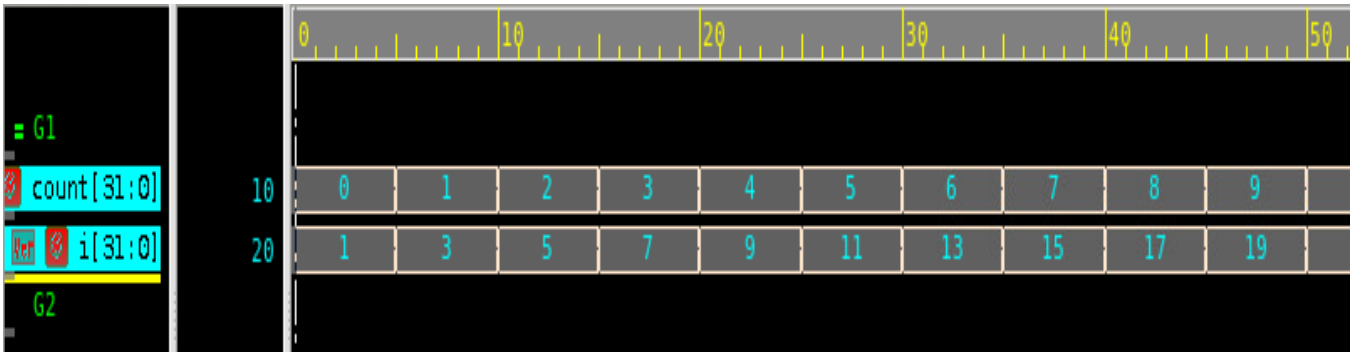
Time=50 ODD NUMBER Index[19] =DATA 19

$finish called from file "practice.v", line 19.

$finish at simulation time 60

V C S S i m u l a t i o n R e p o r t

Time: 60



Q20.0 Generate Even-DATA at even-index and vice-cersa by using for loop and if – else condition:

```
module print_data_index_numbers;

integer i;

integer count;


initial begin

        count = 0;


      for (i=0; count <20;i=i+1) begin

              if (i[0] ==0) begin

                      #5;
```

```
                $display("Time=%0t EVEN NUMBER Index[%0d]=DATA =
%0d",$time,i,i);


                count = count +1 ;
            end
            else begin
                #5;
                $display("Time=%0t ODD NUMBER Index[%0d]=DATA =
%0d",$time,i,i);
                count = count +1;
            end
        end
        #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_data_index_numbers);
end
endmodule
```

Simulation Output:

```
Time=5 EVEN NUMBER Index[0]=DATA = 0
Time=10 ODD NUMBER Index[1]=DATA = 1
Time=15 EVEN NUMBER Index[2]=DATA = 2
Time=20 ODD NUMBER Index[3]=DATA = 3
Time=25 EVEN NUMBER Index[4]=DATA = 4
Time=30 ODD NUMBER Index[5]=DATA = 5
Time=35 EVEN NUMBER Index[6]=DATA = 6
Time=40 ODD NUMBER Index[7]=DATA = 7
```

Time=45 EVEN NUMBER Index[8]=DATA = 8

Time=50 ODD NUMBER Index[9]=DATA = 9

Time=55 EVEN NUMBER Index[10]=DATA = 10

Time=60 ODD NUMBER Index[11]=DATA = 11

Time=65 EVEN NUMBER Index[12]=DATA = 12

Time=70 ODD NUMBER Index[13]=DATA = 13

Time=75 EVEN NUMBER Index[14]=DATA = 14

Time=80 ODD NUMBER Index[15]=DATA = 15

Time=85 EVEN NUMBER Index[16]=DATA = 16

Time=90 ODD NUMBER Index[17]=DATA = 17

Time=95 EVEN NUMBER Index[18]=DATA = 18

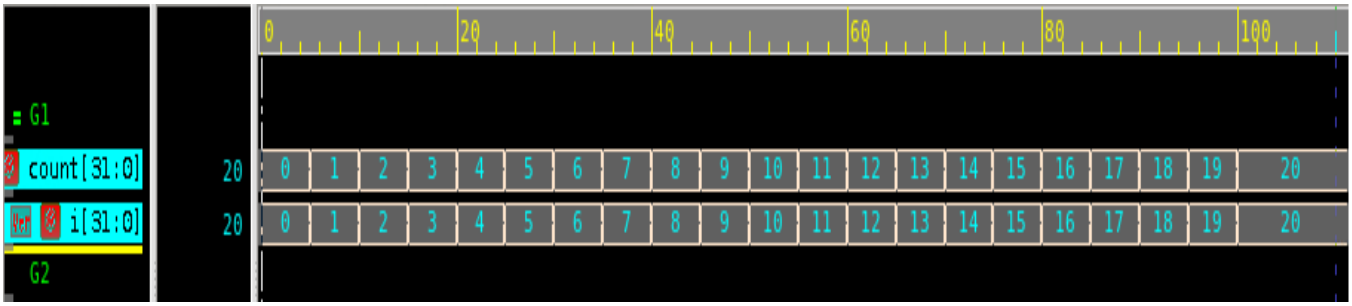
Time=100 ODD NUMBER Index[19]=DATA = 19

$finish called from file "practice.v", line 23.

$finish at simulation time 110

V C S S i m u l a t i o n R e p o r t

Time: 110



Q21.0 Generate EVEN-INDEX-ODD DATA by using for loop and if – else condition:

module print_data_index_numbers;

integer i;

integer count;

integer data;

```
initial begin
        count = 0;

      for (i=0; count <20;i=i+1) begin

            if (i[0] ==0) begin
                  data = i+1;
                  #5;
                  $display("Time=%0t EVEN Index[%0d]= ODD_DATA =
%0d",$time,i,data);

                  count = count +1 ;
            end
            else begin
                  data = i+1;
                  #5;
                  $display("Time=%0t ODD Index[%0d]= EVEN_DATA =
%0d",$time,i,data);
                  count = count +1;
            end
        end
        #10 $finish();
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,print_data_index_numbers);
end
```

endmodule

Simulation Output:

Time=5 EVEN Index[0]= ODD_DATA = 1

Time=10 ODD Index[1]= EVEN_DATA = 2

Time=15 EVEN Index[2]= ODD_DATA = 3

Time=20 ODD Index[3]= EVEN_DATA = 4

Time=25 EVEN Index[4]= ODD_DATA = 5

Time=30 ODD Index[5]= EVEN_DATA = 6

Time=35 EVEN Index[6]= ODD_DATA = 7

Time=40 ODD Index[7]= EVEN_DATA = 8

Time=45 EVEN Index[8]= ODD_DATA = 9

Time=50 ODD Index[9]= EVEN_DATA = 10

Time=55 EVEN Index[10]= ODD_DATA = 11

Time=60 ODD Index[11]= EVEN_DATA = 12

Time=65 EVEN Index[12]= ODD_DATA = 13

Time=70 ODD Index[13]= EVEN_DATA = 14

Time=75 EVEN Index[14]= ODD_DATA = 15

Time=80 ODD Index[15]= EVEN_DATA = 16

Time=85 EVEN Index[16]= ODD_DATA = 17

Time=90 ODD Index[17]= EVEN_DATA = 18

Time=95 EVEN Index[18]= ODD_DATA = 19

Time=100 ODD Index[19]= EVEN_DATA = 20

$finish called from file "practice.v", line 26.

$finish at simulation time 110

V C S S i m u l a t i o n R e p o r t

Time: 110

Q22.0 Print all prime number upto 30 by using for nested loop and if condition:

```
module print_prime_numbers;
integer i,j;
integer prime;

initial begin
        $display(" Prime number upto 20:");

        for (i=2; i <=30; i=i+1) begin
                prime = 1;

                for (j=2; j<i; j= j+1) begin
                        if (i % j ==0)
                                prime = 0;
                end
                if (prime)
                        $display("prime number: %0d", i);
        end
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,print_prime_numbers);
end
endmodule
```

Simulation Output:

Prime number upto 20:

prime number: 2

prime number: 3

prime number: 5

prime number: 7

prime number: 11

prime number: 13

prime number: 17

prime number: 19

prime number: 23

prime number: 29

Q23.0 Count number of 1s in a 10-bit number

```
module print_count_ones;
reg [9:0] data;
integer i;
integer count_ones;

initial begin
        $display(" Printing the 10-bit od data:");
        data = 10'b1011_0101_11;
        count_ones = 0;

        for (i=0; i <10; i=i+1) begin
                if (data[i] ==1'b1)
                        count_ones = count_ones + 1;
        end
        $display(" Data: %0b", data);
```

```
        $display(" Count the number of 1's : %0d", count_ones);


        #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_count_ones);
end
endmodule
```

Simulation Output:

```
Printing the 10-bit od data:
 Data: 1011010111
 Count the number of 1's : 7
```

Q24.0 Count number of 1s in a 10-bit number using SV

```
module print_count_ones;
logic [9:0] data;
integer count_ones;

initial begin
    $display(" Printing the 10-bit od data:");
    data = 10'b1011_0101_11;
    count_ones = $countones(data);

    $display(" Data: %0b", data);
    $display(" Count the number of 1's : %0d", count_ones);

    #10 $finish();
```

```
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,print_count_ones);
end
endmodule
```

Run-command:

```
[b23n_sarif_727@prac3 interview]$ vcs -sverilog practice.v -kdb -lca -debug_access+all -full64
```

Simulation Output:

```
Printing the 10-bit od data:
 Data: 1011010111
 Count the number of 1's : 7
```

Q25.0 Swap of two variable With a Temporary Register using blocking assignment

```
module swaping_with_temp;
reg [7:0] a, b;
reg [7:0] temp;

initial begin
    a = 8'd10;
    b = 8'd20;
    $display("Befor swapping: a= %0d b=%0d", a,b);

    temp = a;
    a    = b;
```

```
    b    = temp;
    $display("After swapping : a= %0d b=%0d",a,b);


    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,swaping_with_temp);
end
endmodule
```

Simulation Output:

Befor swapping: a= 10 b=20

After swapping : a= 20 b=10

- **Good for** combinational logic **and testbench code.**

Q26.0 Swap of two variable Without a Temporary Register (XOR method)

```
module swaping_with_temp;
reg [7:0] a,b;
reg [7:0] temp;


initial begin
    a = 8'd10;
    b = 8'd20;
    $display("Befor swapping: a= %0d b=%0d", a,b);


    a = a ^ b;
    b = a ^ b;
```

```
    a = a ^ b;
    $display("After swapping : a= %0d b=%0d",a,b);


    #10 $finish();
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,swaping_with_temp);
end
endmodule
```

Simulation Output:

```
Befor swapping: a= 10 b=20
After swapping : a= 20 b=10
```

Q27.0 Swapping of two variables without using the temporary variable by using the blocking assignment

```
module swap_blocking;
reg [3:0] a,b;


initial begin
    a = 4'd5;
    b = 4'd9;
    $display("Before swap: a = %0d, b = %0d", a, b);
    a = a + b;
    b = a - b;
    a = a - b;
    $display("After swap: a = %0d, b = %0d", a, b);
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,swap_blocking);
end
endmodule
```

Simulation Result

Before swap: a = 5, b = 9

After swap: a = 9, b = 5

Q28.0 Swapping of two variables without using the third variable by using the non-blocking assignment

```
module swap_variables;
reg a, b;
initial begin
        a = 0;
        b = 1;
    $display("Initial values: a = %b, b = %b", a, b);
    a <= b;
    b <= a;
    #1;
    $display("Swapped values: a = %b, b = %b", a, b);
    $finish();
    end


initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,swap_variables);
```

```
end
endmodule
```

Simulation Result

Initial values: a = 0, b = 1

Swapped values: a = 1, b = 0

Q29.0 Swapping Using a Third Variable (temp) with Blocking Assignment procedural block (always @(posedge clk))

```
module swap_block_proc;
reg [3:0] a,b,temp;
reg clk;

initial clk = 0;
always #5 clk = ~clk;
initial begin
    a = 4'd3;
    b = 4'd7;
    $display("Before swap: a = %0d, b = %0d", a, b);
end
always @(posedge clk) begin
    temp = a;
    a = b;
    b = temp;
end
initial begin
    #10;
    $display("After swap:  a = %0d, b = %0d", a, b);
```

```
    $finish();
end
endmodule
```

Simulation Result

Output

```
Before swap: a = 3, b = 7
After swap:  a = 7, b = 3
$finish at simulation time                 10
```

Q30.0 Swapping without using a Third Variable with non-Blocking Assignment procedural block (always @(posedge clk))

```
module swap_nb_proc;
reg [3:0] a,b;
reg clk;

initial clk = 0;
always #5 clk = ~clk;

initial begin
    a = 4'd3;
    b = 4'd7;
    $display("Before swap: a = %0d, b = %0d", a, b);
end
always @(posedge clk) begin
    a <= b;
    b <= a;
end
```

```
initial begin
        #10;
        $display("After swap:  a = %0d, b = %0d", a, b);
        $finish();
end
endmodule
```

Simulation Result

Output

```
Before swap: a = 3, b = 7
After swap:  a = 7, b = 3
$finish at simulation time                 10
```

Q31.0 Swapping with using a Third Variable with non-Blocking Assignment procedural block (always @(posedge clk))

```
module swap_variables;
reg clk;
reg a, b;
reg temp;

always #5 clk = ~clk;
initial begin
          clk = 0;
        a = 0;
        b = 1;
        temp =0;
```

```
        $display("Before swap: a=%b b=%b", a, b);

        #10;
        $display("After swap : a=%b b=%b", a, b);

        $finish;
end

always @(posedge clk) begin
          temp <= a;
          a    <= b;
          b    <= temp;
end

initial begin
          $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, swap_variables);
end
endmodule
```

Simulation Result

Before swap: a=0 b=1

After swap : a=1 b=0

Q31.0 Palindrome Number Check

☑ **Palindrome Number**

A palindrome number is a number that reads the same forward and backward.

- Examples: single digits varies from (0-9).
- Palindrome: `121`, `1331`, `44`

- Not Palindrome: `123`, `456`

◇ In one line:

- If the reverse of a number is the same as the original number, it is a palindrome.

A palindrome number reads the same forwards and backward meaning its digits are identical when reversed, like 121, 55, or 1331, while numbers like 123 or 121.

A number that remains unchanged when its digits are reversed.

0, 1, 11, 22, 101, 121, 12321.

- **Reverse the Number**: Create a reversed version of the number by extracting digits from the end (using modulo % 10) and building a new number
- **Compare** Check if the original number equals the reversed number.

Design Code:

```
module palindrome_number(
input [9:0] num,
output reg palindrome);

integer temp,reverse,digit;

always @(*) begin
    temp = num;
    reverse = 0;

    while (temp >0) begin
        digit = temp % 10;
        reverse = (reverse*10) + digit;
        temp = temp / 10;
    end

    palindrome = (reverse == num);
```

```
end
endmodule
```

Testbench:

```
module tb;
reg  [9:0] num;
wire palindrome;

palindrome_number dut (.num(num), .palindrome(palindrome));

integer i;

initial begin
        for (i=0 ; i< 203 ; i=i+1) begin
                num = i;
                #1;
                if (palindrome)
                        $display("Palindrome number: %0d", num);
        end

        #10 $finish;
end
initial begin
       $fsdbDumpfile("dump.fsdb");
       $fsdbDumpvars(0,tb);
end
```

endmodule

Simulation Result

Palindrome number: 0

Palindrome number: 1

Palindrome number: 2

Palindrome number: 3

Palindrome number: 4

Palindrome number: 5

Palindrome number: 6

Palindrome number: 7

Palindrome number: 8

Palindrome number: 9

Palindrome number: 11

Palindrome number: 22

Palindrome number: 33

Palindrome number: 44

Palindrome number: 55

Palindrome number: 66

Palindrome number: 77

Palindrome number: 88

Palindrome number: 99

Palindrome number: 101

Palindrome number: 111

Palindrome number: 121

Palindrome number: 131

Palindrome number: 141

Palindrome number: 151

Palindrome number: 161

Palindrome number: 171

Palindrome number: 181

Palindrome number: 191

Palindrome number: 202

$finish called from file "practice_tb.v", line 18.

$finish at simulation time 213

V C S S i m u l a t i o n R e p o r t

Time: 213

Design Code:

```
module palindrome_number(
input [9:0] num,
output reg palindrome);

integer temp,reverse,digit;

always @(*) begin
        temp = num;
        reverse = 0;

        while (temp >0) begin
                digit = temp % 10;
                reverse = (reverse*10) + digit;
                temp = temp / 10;
        end

        if (reverse == num)
```

```
            palindrome = 1'b1;
        else
            palindrome = 1'b0;

end
endmodule
```

Testbench:

```
module tb;
reg  [9:0] num;
wire palindrome;

palindrome_number dut (.num(num), .palindrome(palindrome));

integer i;

initial begin
        for (i=0 ; i< 23 ; i=i+1) begin
            num = i;
            #1;
            if (palindrome)
                $display("Palindrome number: %0d", num);
            else
                $display("Not Palendrome nuber: %0d",num);
        end

        #10 $finish;
end
```

```
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,tb);
end
endmodule
```

Simulation Result

```
Palindrome number: 0
Palindrome number: 1
Palindrome number: 2
Palindrome number: 3
Palindrome number: 4
Palindrome number: 5
Palindrome number: 6
Palindrome number: 7
Palindrome number: 8
Palindrome number: 9
Not Palendrome nuber: 10
Palindrome number: 11
Not Palendrome nuber: 12
Not Palendrome nuber: 13
Not Palendrome nuber: 14
Not Palendrome nuber: 15
Not Palendrome nuber: 16
Not Palendrome nuber: 17
Not Palendrome nuber: 18
Not Palendrome nuber: 19
Not Palendrome nuber: 20
```

Not Palendrome nuber: 21

Palindrome number: 22

$finish called from file "practice_tb.v", line 20.

$finish at simulation time 33

V C S S i m u l a t i o n R e p o r t

Time: 33

Binary Palindrome-8-bits (for fixed width numbers)

Constraint and Randomization

What is a constraint in SystemVerilog?

A constraint is a rule used in randomization to control what values a variable can take.

👉 In short:

Constraint = limits on random values

Used mainly in verification to generate valid test scenarios automatically.

Basic example (no constraint)

class packet;

```
  rand bit [3:0] addr;
endclass

packet p = new();
p.randomize();
```

- `addr` can be any value from 0 to 15.

Example with constraint

```
class packet;
  rand bit [3:0] addr;

  constraint addr_limit {
    addr < 8;
  }
endclass
```

- Now `addr` will be only 0 to 7

Constraint using range

```
constraint c1 {
  addr inside {[4:10]};
}
```

✓ addr will be between 4 and 10

Constraint with multiple conditions

```
constraint c2 {
  addr > 3;
  addr < 12;
}
```

✓ Both conditions must be true

Constraint on multiple variables

```
class sample;
  rand int a, b;

  constraint c3 {
    a < b;
    b < 20;
  }
endclass
```

✓ `a` is always less than `b`

Conditional constraint (`if`)

```
constraint c4 {
  if (a < 5)
    b == 10;
}
```

✓ If `a < 5`, then `b` must be 10

Disable a constraint

p.c1.constraint_mode(0); // disable

p.c1.constraint_mode(1); // enable

One-line interview answer

Constraint in SystemVerilog is used to control random variable values during randomization to generate valid and meaningful test cases.

What is a `soft` constraint?

A soft constraint is a default rule.

👉 It is applied only if no stronger (hard) constraint conflicts with it.

In simple words:

“Follow this rule… unless someone tells you otherwise.”

Why do we need soft constraint?

Because sometimes:

- You want a default value
- But also want the test to override it

Hard constraints cannot be overridden, soft ones can.

Basic example

```
class packet;
  rand int size;
```

```
  constraint c1 {
    soft size == 64;
  }
endclass
```

- By default → `size = 64`

Overriding a soft constraint

```
packet p = new();

// Override soft constraint
p.randomize() with { size == 128; };
```

✓ Result: `size = 128`

✓ Soft constraint is ignored

Soft vs Hard constraint

```
constraint hard_c {
  size > 0;
}

constraint soft_c {
  soft size == 64;
}
```

✓ `size` must be > 0 (hard rule)

✓ Default preferred value = 64

If conflict happens

soft size == 64;

size == 32;

✓ `size = 32`

✓ Hard constraint wins

Common UVM use-case

```
class trans;
  rand int delay;

  constraint delay_c {
    soft delay inside {[10:20]};
  }
endclass
```

✓ Normal tests → delay = 10–20

✓ Stress test → override delay to 0 or 100

Interview one-liner

Soft constraint provides a default value that can be overridden by a hard constraint or inline randomization.

Key points to remember

✓ Soft constraint is optional

✓ Hard constraint has higher priority

✓ Useful for reusable testbench code

✓ Mostly used in UVM environments

1 What is a constraint in SystemVerilog?

Answer:

A constraint is a rule used to limit random variable values during randomization so that only valid data is generated.

2 Difference between `rand` and `randc`?

Answer:

- `rand` → random values, repetition allowed
- `randc` → cyclic random, no repetition until all values are used

randc bit [2:0] id; // 0–7 without repeat

3 What is a soft constraint?

Answer:

A soft constraint provides a default value that can be overridden by hard or inline constraints.

4 What happens if constraints conflict?

Answer:

- Hard vs hard → randomize fails
- Hard vs soft → hard wins
- Inline constraint has highest priority

```
if (!obj.randomize())
  $display("Randomization failed");
```

5 What is an inline constraint?

Answer:

A constraint written inside the `randomize()` call.

```
pkt.randomize() with { size == 128; };
```

6 What is `dist` constraint?

Answer:

Used to control probability distribution of random values.

```
constraint c {
  addr dist {0 := 10, [1:7] := 90};
}
```

✓ addr = 0 → 10%

✓ addr = 1–7 → 90%

7 Can we disable a constraint?

Answer:

Yes, using `constraint_mode()`.

```
pkt.c1.constraint_mode(0); // disable
```

8 Difference between `inside` and range?

Answer:

- `inside` supports multiple ranges & values
- Range supports single continuous range

addr inside {3, 5, [8:12]};

9 When does randomization fail?

Answer:

When constraints are unsatisfiable or conflicting.

Example:

a > 10;

a < 5; // conflict

10 Can we use if-else in constraints?

Answer:

Yes.

```
constraint c {
  if (mode)
    size == 64;
  else
    size == 128;
}
```

11 Where are constraints mostly used?

Answer:

In verification/UVM testbenches to generate legal stimulus automatically.

12 Can constraints be written outside class?

Answer:

No. Constraints are class-based and used with `randomize()`.

⭐ Most important interviewer tip

If asked:

“Why do we use constraints?”

Say:

To generate valid, reusable, and coverage-driven random test scenarios with minimum manual effort.

What is `dist` constraint?

A `dist` constraint controls how often a random value appears.

👉 In short:

- `dist` = weighted randomization (probability control)

Why do we need `dist`?

Normal randomization:

- All values have equal chance

With `dist`:

- Some values occur more often
- Some values occur less often

Very useful in directed-random testing.

Basic example

```
class pkt;
  rand int addr;

  constraint c1 {
    addr dist {0 := 50, 1 := 50};
  }
endclass
```

✓ `addr = 0` → 50%

✓ `addr = 1` → 50%

Range with dist

```
constraint c2 {
  addr dist {[0:3] := 20, [4:7] := 80};
}
```

✓ Values 0–3 → 20% total

✓ Values 4–7 → 80% total

👉 Weight is for the whole range, not each value.

`:=` vs `:/` (VERY IMPORTANT)

1 `:=` → Absolute weight

Weight is shared across the whole range.

addr dist {[0:3] := 20};

✓ Each value gets 20 / 4 = 5

2 `:/` → Per-value weight

Each value gets full weight.

addr dist {[0:3] :/ 20};

✓ Each value gets 20

Clear comparison

addr dist {[0:3] := 20, [4:7] := 20};

✓ All values equally likely

addr dist {[0:3] :/ 20, [4:7] :/ 5};

✓ 0–3 appear 4× more than 4–7

Mixed values and ranges

```
constraint c3 {
  addr dist {0 := 10, [1:5] := 90};
}
```

✓ addr = 0 → 10%

✓ addr = 1–5 → 90%

Common UVM use-case

```
constraint pkt_type_c {
  pkt_type dist {
    READ  := 70,
    WRITE := 30
  };
}
```

✓ More READ packets than WRITE

✓ Realistic traffic modeling

Interview one-liner

`dist` constraint is used to control the probability distribution of randomized values.

10 tricky `dist` constraint interview questions.

1 Are `dist` weights percentages?

Answer:

❌ No. They are relative weights, not percentages.

```
a dist {0 := 1, 1 := 3};
```

👉 `1` occurs 3× more than `0`.

2 Difference between `:=` and `:/`?

Answer:

- `:=` → weight is shared across the entire range

- `:/` → weight is applied to each value

[0:3] := 20 // each gets 5

[0:3] :/ 20 // each gets 20

3 If weights are equal, what happens?

Answer:

All values have equal probability.

a dist {[0:3] := 10, [4:7] := 10};

4 Can `dist` be used without `rand`?

Answer:

❌ No.

`dist` works only on `rand` variables.

5 Does `dist` guarantee exact percentage?

Answer:

❌ No.

It is statistical, not exact for small samples.

✓ Accuracy improves with more randomizations.

6 Which has higher priority: `dist` or inline constraint?

Answer:

✓ Inline constraint wins.

obj.randomize() with { a == 5; };

7 Can we mix single values and ranges in `dist`?

Answer:

✓ Yes.

a dist {0 := 10, [1:5] := 90};

8 What happens if `dist` conflicts with another constraint?

Answer:

- Hard vs hard → ❌ randomize fails
- Hard vs soft → ✓ hard wins

9 Can `dist` be used with `randc`?

Answer:

❌ No (meaningfully).

👉 `randc` ensures no repetition, which breaks probability control.

10 Real UVM interview question

Q: How do you generate more READ than WRITE transactions?

Answer:

pkt_type dist {READ := 70, WRITE := 30};

★ Golden interview summary (say this confidently)

- `dist` constraint controls relative probability of randomized values and is mainly used for realistic traffic modeling in verification.

Top 20 interview question for VLSI design and verification engineer on topic constraint in system Verilog

◇ TOP 20 CONSTRAINT INTERVIEW QUESTIONS (VLSI Design & Verification)

1 What is a constraint in System Verilog?

Answer:

A constraint is a rule used during randomization to limit the values of `rand` variables so that valid stimulus is generated.

2 Where are constraints used?

Answer:

In verification environments (UVM) for automatic and controlled random test generation.

3 Difference between `rand` and `randc`?

Answer:

- `rand` → random, repetition allowed
- `randc` → cyclic random, no repetition until all values are used

4 What is a soft constraint?

Answer:

A soft constraint provides a default rule that can be overridden by hard or inline constraints.

5 What is an inline constraint?

Answer:

A constraint written inside the `randomize()` call using `with`.

pkt.randomize() with { size == 128; };

6 What happens if constraints conflict?

Answer:

- Hard vs hard → randomization fails
- Hard vs soft → hard constraint wins

7 How do you detect randomization failure?

Answer:

if (!obj.randomize())

  $display("Randomization failed");

8 What is `inside` constraint?

Answer:

Used to restrict values to a specific set or range.

addr inside {1, 3, [5:10]};

9 Difference between range constraint and `inside`?

Answer:

- Range → single continuous range
- `inside` → multiple ranges and discrete values

10 What is a `dist` constraint?

Answer:

Used to control probability distribution of random values.

11 Difference between `:=` and `:/` in `dist`?

Answer:

- `:=` → weight shared across range
- `:/` → weight applied per value

12 Are `dist` weights percentages?

Answer:

No. They are relative weights.

13 Can constraints be conditional?

Answer:

Yes, using `if-else`.

if (mode) size == 64;

else size == 128;

14 Can we disable a constraint?

Answer:

Yes, using `constraint_mode()`.

pkt.c1.constraint_mode(0);

15 Can constraints be enabled/disabled dynamically?

Answer:

Yes, at runtime using `constraint_mode()`.

16 What causes `randomize()` to fail?

Answer:

Unsatisfiable or conflicting constraints.

17 Can constraints be written outside a class?

Answer:

No. Constraints must be defined inside a class.

18 Can we use loops in constraints?

Answer:

Yes, using `foreach`.

foreach (arr[i]) arr[i] > 0;

19 Can `dist` be used with `randc`?

Answer:

No. `randc` breaks probability distribution.

20 Why are constraints important in verification?

Answer:

They enable automatic, reusable, coverage-driven test generation while avoiding illegal scenarios.

⭐ ONE-LINE POWER ANSWER (Interview Closer)

System Verilog constraints allow controlled random stimulus generation, making verification scalable, reusable, and efficient.

1.0 constraint to generate pattern 111222333444555

```
//
class pattern_gen;
        rand int arr[15];


        constraint pattern_c { foreach (arr[i]) { arr[i] == (i / 3) + 1; }
  }
endclass

module tb;

initial begin
        pattern_gen p = new();
        p.randomize();
        $display("%p", p.arr);
end
endmodule
```

Simulation output:

```
'{1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5}
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Method-2

```
class pattern_gen;
        rand int arr[15];


        constraint pattern_c { foreach (arr[i]) { arr[i] == (i / 3) + 1; }
  }
endclass

module tb;

initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
111222333444555
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

2.0 generate Pattern 123123123 using constraint in system verilog?

```
class pattern_gen;
```

```
        rand int arr[15];

        constraint pattern_c { foreach (arr[i]) { arr[i] == (i % 3) + 1; }
  }
endclass

module tb;

initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
123123123123123
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

3.0 generate Pattern 123412341234 using constraint in system verilog?

```
class pattern_gen;
  rand int arr[12];
```

```
  constraint pattern_c {
    foreach (arr[i]) {
      arr[i] == (i % 4) + 1;
    }
  }
endclass

module tb;
  initial begin
    pattern_gen p = new();
    p.randomize();

    foreach (p.arr[i])
      $write("%0d", p.arr[i]);
    $display("");
  end
endmodule
```

Simulation output:

```
123412341234
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

4.0 generate Pattern 121212 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];
```

```
        constraint pattern_c { foreach (arr[i])
        { arr[i] == (i % 2) + 1; }
  }
endclass

module tb;

initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
121212
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

5.0 generate Pattern 010101 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];
```

```
        constraint pattern_c { foreach (arr[i])
        { arr[i] == (i % 2) ; }
  }
endclass

module tb;

initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
010101
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

6.0 generate Pattern 101010 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];

        constraint pattern_c { foreach (arr[i])
```

```
        { arr[i] == (i+1) % 2 ; }
  }
endclass

module tb;

initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
101010
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

7.0 generate Pattern 110011 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];

        constraint pattern_c { foreach (arr[i]) {
                arr[i] == (((i / 2) % 2) ? 0 : 1); }
  }
```

```
endclass

module tb;

initial begin
    pattern_gen p = new();
    p.randomize();

    foreach (p.arr[i])
        $write("%0d",p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
110011
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

8.0 generate Pattern 111000 using constraint in system verilog?

```
class pattern_gen;
    rand int arr[6];

    constraint pattern_c { foreach (arr[i]) {
        arr[i] == ( i < 3) ; }
  }
endclass
```

```
module tb;


initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
111000
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

9.0 generate Pattern 000111 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];

        constraint pattern_c { foreach (arr[i]) {
                arr[i] == ( i >= 3) ; }
  }
endclass

module tb;
```

```
initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
              $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
000111
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

10.0 generate Pattern 123321 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];

        constraint pattern_c { foreach (arr[i]) {
              if (i < 3)
               arr[i] == i + 1 ;
              else
              arr[i] == 6 - i  ; }

  }
endclass
```

```
module tb;


initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
123321
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

11.0 generate Pattern 101100 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];

        constraint pattern_c { foreach (arr[i]) {
                if (i ==0 || i ==2 || i==3)
                 arr[i] == 1 ;
                else
                arr[i] == 0 ; }
  }
```

```
endclass

module tb;

initial begin
    pattern_gen p = new();
    p.randomize();

    foreach (p.arr[i])
        $write("%0d",p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
101100
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

12.0 generate Pattern 1001001 using constraint in system verilog?

```
class pattern_gen;
    rand int arr[6];

    constraint pattern_c { foreach (arr[i]) {
        if (i % 3 ==0)
         arr[i] == 1 ;
        else
```

```
            arr[i] == 0 ; }
  }
endclass

module tb;
initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
100100
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

13.0 generate Pattern 100010001 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[9];

        constraint pattern_c { foreach (arr[i]) {

                arr[i] == (i % 4 ==0) ;
       }
```

```
  }
endclass

module tb;
initial begin
        pattern_gen p = new();
        p.randomize();

        foreach (p.arr[i])
                $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
100010001
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

14.0 generate Pattern 001122 using constraint in system verilog?

```
class pattern_gen;
        rand int arr[6];

        constraint pattern_c { foreach (arr[i]) {

                arr[i] == (i / 2) ;
       }
  }
```

```
endclass

module tb;
initial begin
    pattern_gen p = new();
    p.randomize();

    foreach (p.arr[i])
        $write("%0d",p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
001122
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

15.0 Generate an array of size 10 where even indices contain even numbers and odd indices contain odd numbers using constraints.

```
class pattern_gen;
    rand int arr[10];

    constraint pattern_c { foreach (arr[i]) {
        if (i % 2 == 0)
            arr[i] % 2 == 0;
        else
```

```
                arr[i] % 2 == 1;
    }
  }
endclass

module tb;
initial begin
    pattern_gen p = new();
    p.randomize();

    foreach (p.arr[i])
        $display("arr[%0d]=%0d",i,p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
arr[0]=1540940906
arr[1]=795244529
arr[2]=-590128402
arr[3]=1333629921
arr[4]=680794568
arr[5]=535335395
arr[6]=-752355124
arr[7]=24099795
arr[8]=950453918
arr[9]=246673239
```

Time: 0

16.0 Generate an array where each value is repeated twice (example:1122334455) using constraints.

```
class pattern_gen;
	rand int arr[10];

	constraint pattern_c { foreach (arr[i]) {
		arr[i] == (i / 2) + 1 ;
	}
  }
endclass

module tb;
initial begin
	pattern_gen p = new();
	p.randomize();

	foreach (p.arr[i])
		$write("%0d",p.arr[i]);
	$display("");
end
endmodule
```

Simulation output:

```
1122334455
	V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

17.0 Generate a palindrome pattern (example: 123454321) using System Verilog constraints.

```
class pattern_gen;
	rand int arr[9];

	constraint pattern_c { foreach (arr[i]) {
		if ( i < 5)
			arr[i] == i + 1 ;
		else
			arr[i] == 9 - i;
	}
  }
endclass

module tb;
initial begin
	pattern_gen p = new();
	p.randomize();

	foreach (p.arr[i])
		$write("%0d",p.arr[i]);
	$display("");
end
endmodule
```

Simulation output:

```
123454321
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

18.0 Generate a strictly increasing pattern where the difference between adjacent elements is 2 using constraints.

```
class pattern_gen;
	rand int arr[5];

	constraint pattern_c { arr[0]==2; foreach (arr[i]) {
		if ( i > 0)
			arr[i] == arr[i-1] + 2 ;
	}
  }
endclass

module tb;
initial begin
	pattern_gen p = new();
	p.randomize();

	foreach (p.arr[i])
		$write("%0d",p.arr[i]);
	$display("");
end
endmodule
```

Simulation output:

```
246810
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

19.0 Generate a pattern where every third element is 0 and others are random non-zero values using constraints.

```
class pattern_gen;
	rand bit arr[9];


	constraint pattern_c { foreach (arr[i]) {
		if ((i+1) % 3 ==  0)
			arr[i] == 0 ;
		else
			arr[i] == 1;
	}
  }
endclass


module tb;
initial begin
	pattern_gen p = new();
	p.randomize();


	foreach (p.arr[i])
		$write("%0d",p.arr[i]);
	$display("");
```

```
end

endmodule
```

Simulation output:

```
110110110

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

20.0 Generate a pattern 555444333222111 using constraints (no procedural code).

```
class pattern_gen;

      rand int arr[15];


      constraint pattern_c { foreach (arr[i]) {

                  arr[i] == 5 - (i/3) ;

     }

 }

endclass


module tb;

initial begin

      pattern_gen p = new();

      p.randomize();


      foreach (p.arr[i])
```

```
            $write("%0d",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
555444333222111
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

21.0 Generate a pattern where array values depend on index (arr[i] = i*i) using constraints.

```
class pattern_gen;
        rand int arr[8];

        constraint pattern_c { foreach (arr[i]) {
                        arr[i] == i*i ;
       }
  }
endclass

module tb;
initial begin
        pattern_gen p = new();
        p.randomize();
```

```
        foreach (p.arr[i])
                $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

0 1 4 9 16 25 36 49

V C S S i m u l a t i o n R e p o r t

Time: 0

22.0 System Verilog constraint to generate the following pattern 5,-10,15,-20,25,-30,35,-40,45,-50

```
class pattern_gen;
        rand int arr[10];

        constraint pattern_c { foreach (arr[i]) {
                if (i % 2 ==0)
                        arr[i] == 5*i + 5 ;
                else
                        arr[i] == -(5*i+5);
       }
  }
endclass
```

```
module tb;
initial begin
        pattern_gen p = new();
        p.randomize();


        foreach (p.arr[i])
                $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
5 -10 15 -20 25 -30 35 -40 45 -50
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

23.0 Write a constraint for a 4-bit dynamic array in such a way that the size of the array should be in between 15 to 20 with even numbers in odd locations and odd numbers in even location.

```
class pattern_gen;
        rand bit [3:0]arr[];


        constraint pattern_c { arr.size() inside {[15:20]};
```

```
        foreach (arr[i]) {
                if (i % 2 ==0)
                        arr[i] % 2 == 1 ;
                else
                        arr[i] % 2 == 0;
        }
  }
endclass


module tb;
initial begin
        pattern_gen p = new();
        p.randomize();
        foreach (p.arr[i])
                $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
15 6 9 10 13 4 15 12 1 2 1 8 3 8 3
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

24.0 Constraint for unique elements in an array without using unique keyword

```
class pattern_gen;
	rand bit [3:0]arr[];


	constraint pattern_c { arr.size==10 ;
	foreach (arr[i]) {
		foreach (arr[j]) {


		if (i !=j)
			arr[i] != arr[j] ;
	}
	}
  }
endclass
module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ",p.arr[i]);
	$display("");
end
```

```
endmodule
```

Simulation output:

```
7 12 5 13 9 8 6 15 0 14

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

25.0 Constraint for unique elements in an array with using unique keyword

```
class pattern_gen;
      rand bit [3:0] arr[10];


      constraint pattern_c { unique {arr} ;
  }
endclass


module tb;
initial begin
      pattern_gen p = new();
      p.randomize();
      foreach (p.arr[i])
            $write("%0d ",p.arr[i]);
      $display("");
end
endmodule
```

Simulation output:

9 1 3 14 4 5 2 10 8 0

V C S S i m u l a t i o n R e p o r t

Time: 0

26.0 Constraint for ascending/ descending ordr of elements in an array

```
class pattern_gen;
	rand bit [3:0] arr[];


	constraint pattern_c { arr.size==10;
	foreach (arr[i])
		if (i > 0)
			arr[i] > arr[i-1];
  }
endclass


module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ",p.arr[i]);
	$display("");
```

```
end

endmodule
```

Simulation output:

```
1 5 7 8 9 10 12 13 14 15

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

27.0 Constraint to generate the values which are divisible by 5

```
class pattern_gen;

      rand bit [3:0] arr[];


      constraint pattern_c { arr.size() inside {[5:10]};

      foreach (arr[i]) {

            arr[i] inside {[0:50]};

      arr[i] % 5 == 0;

      }

  }

endclass


module tb;

initial begin

      pattern_gen p = new();

      p.randomize();
```

```
        foreach (p.arr[i])
                $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
15 0 5 15 0
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Method-2

```
class pattern_gen;
        rand int arr[10];

        constraint pattern_c { foreach (arr[i]) {
                arr[i] % 5 == 0;
        arr[i] inside {[0:100]};
        }
  }
endclass

module tb;
initial begin
        pattern_gen p = new();
```

```
        p.randomize();
        foreach (p.arr[i])
                $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
35 25 70 95 45 95 95 50 25 35
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

28.0 Constraint to generate a 10 real number between 1.356 to 2.658

```
class pattern_gen;
        rand int arr[10];
        real arr2[10];

        constraint pattern_c { foreach (arr[i]) {
                arr[i] inside {[1356:2658]};
        }
  }

function void post_randomize();
        foreach (arr[i])
```

```
        arr2[i] = arr[i] / 1000.0;
endfunction
endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
    foreach (p.arr2[i])
        $display("%0f ",p.arr2[i]);
    $display("");
end
endmodule
```

Simulation output:

```
1.391000
1.494000
1.980000
1.900000
2.006000
2.161000
2.385000
2.344000
2.342000
2.367000
```

V C S S i m u l a t i o n R e p o r t

Time: 0

29.0 Constraint to generate 0 1 3 6 10 15 21 28 36 45

```
class pattern_gen;
	rand bit [5:0]arr[];


	constraint pattern_c { arr.size==10; foreach (arr[i]) {
		if (i > 0)
			arr[i] == arr[i-1] + i;
		else
			arr[i] == 0;
	}
}
endclass
module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ",p.arr[i]);
	$display("");
end
```

```
endmodule
```

Simulation output:

```
0 1 3 6 10 15 21 28 36 45

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

30.0 Constraint to generate 0 1 0 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9 0 1 0 2 0 3 0 4 0 5

```
class pattern_gen;
    rand int arr[30];


    constraint pattern_c { foreach (arr[i]) {
        if (i % 2 == 0)
            arr[i] == 0 ;
        else
            arr[i] == ((i / 2) % 9) + 1;
    }
}
endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
    foreach (p.arr[i])
```

```
            $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
0 1 0 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9 0 1 0 2 0 3 0 4 0 5 0 6
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

31.0 Constraint to generate 1 22 333 4444 55555

```
class pattern_gen;
    rand int arr[15];

    constraint pattern_c { foreach (arr[i]) {
        if (i < 1)
            arr[i] == 1 ;
        else if (i < 3)
            arr[i] == 2 ;
        else if (i < 6)
            arr[i] == 3 ;
        else if (i < 10)
            arr[i] == 4 ;
        else
            arr[i] == 5 ;
```

```
        }
}
endclass
module tb;
initial begin
        pattern_gen p = new();
        p.randomize();
        foreach (p.arr[i])
                $write("%0d ",p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
1 2 2 3 3 3 4 4 4 4 5 5 5 5 5
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

32.0 Constraint to generate 5 7 10 14 15 21 20 28

```
class pattern_gen;
        rand int arr[10];

        constraint pattern_c { foreach (arr[i]) {
                if (i % 2 == 0)
                        arr[i] == 5*(i/2 +1) ;
```

```
        else
            arr[i] == 7*(i/2 +1) ;
    }
}
endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
    foreach (p.arr[i])
        $write("%0d ",p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
5 7 10 14 15 21 20 28 25 35
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

33.0 Constraint to generate first half multiple of 8 and second half multiple of 7 in between range 0 to 100

```
class pattern_gen;
    rand int arr[10];
```

```
    constraint pattern_c { foreach (arr[i]) {
        arr[i] inside {[0:100]};

        if (i < arr.size()/2)
            arr[i] % 8 == 0 ;
        else
            arr[i] % 7 == 0 ;
    }
}
endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
    foreach (p.arr[i])
        $write("%0d ",p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
72 72 16 24 56 91 91 49 21 35
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

34.0 Constraint to generate 1 2 3 4 0 4 3 2 1

```
class pattern_gen;
	rand int arr[9];


	constraint pattern_c { foreach (arr[i]) {
		if (i < arr.size()/2)
			arr[i] == i + 1 ;
		else if (i == arr.size()/2)
			arr[i] == 0 ;
		else
			arr[i] == arr[arr.size() -1-i] ;
	}
}
endclass
module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ",p.arr[i]);
	$display("");
end
```

```
endmodule
```

Simulation output:

```
1 2 3 4 0 4 3 2 1

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

35.0 Write the constraint of **10-bits** but next data will be created only **2-bit changes only** means differ only 2-bit from previous data of 10 bit?

```
class pattern_gen;
    rand bit [9:0] arr[5];


    constraint pattern_c { foreach (arr[i]) {
        if (i > 0)
            $countones (arr[i] ^ arr[i-1]) == 2 ;
    }
}
endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
    foreach (p.arr[i])
        $display("%0d : %010b",i, p.arr[i]);
```

```
        $display("");
end
endmodule
```

Simulation output:

```
0 : 1000011010
1 : 1110011010
2 : 0111011010
3 : 1111010010
4 : 1101010011
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

36.0 Constraint to generate 0 1 0 2 0 3 0 4 0 5

```
class pattern_gen;
        rand int arr[10];


        constraint pattern_c { foreach (arr[i]) {
                if (i % 2 == 0)
                        arr[i] == 0 ;
                else
                        arr[i] == (i+1) / 2 ;
        }
}
endclass
```

```
module tb;
initial begin
        pattern_gen p = new();
        p.randomize();
        foreach (p.arr[i])
                $write("%0d ", p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
0 1 0 2 0 3 0 4 0 5
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

37.0 write the constraint of two different variable of 10-bit a and b are not equal but number of bits of toggle are only 5? like a!=b

```
class pattern_gen;
  rand bit [9:0] a, b;


  constraint toggle_c {
    a != b;
    $countones(a ^ b) == 5;
  }
```

```
endclass
```

38.0 Write a constraint to generate the pattern 9 19 29 39 49 59 69 79 89 99

```
class pattern_gen;
	rand int arr[10];


	constraint pattern_c { foreach (arr[i]) {
		arr[i] == (10*i + 9) ;
	}
}
endclass
module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ", p.arr[i]);
	$display("");
end
endmodule
```

Simulation output:

```
9 19 29 39 49 59 69 79 89 99
	V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

39.0 Write a constraint to generate a random even number between 50 to 100

```
class pattern_gen;
	rand int arr[10];


	constraint pattern_c { foreach (arr[i]) {
		arr[i] inside {[50:100]} ;
		arr[i] % 2 == 0 ;
	}
}
endclass
module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ", p.arr[i]);
	$display("");
end
endmodule
```

Simulation output:

```
90 90 82 58 70 60 50 88 84 98
```

V C S S i m u l a t i o n R e p o r t

Time: 0

40.0 Write the constraint such that even locations contain odd numbers and odd locations contain even number.

```
class pattern_gen;
	rand bit[4:0] arr[10];


	constraint pattern_c { foreach (arr[i]) {
		if (i % 2 == 0)
			arr[i] % 2 == 1;
		else
			arr[i] % 2 == 0 ;
	}
}
endclass
module tb;
initial begin
	pattern_gen p = new();
	p.randomize();
	foreach (p.arr[i])
		$write("%0d ", p.arr[i]);
	$display("");
end
```

```
endmodule
```

Simulation output:

```
11 16 15 0 9 2 13 18 31 22

        V C S  S i m u l a t i o n  R e p o r t

Time: 0
```

41.0 Write a constraint on a two dimensional array to generate even number in the first 4 locations and odd numbers in next 4 locations.

```
class pattern_gen;
    rand bit[3:0] arr[2][4];


    constraint pattern_c { foreach (arr[i,j]) {
        if (i*4 + j < 4)
            arr[i][j] % 2 == 0;
        else
            arr[i][j] % 2 == 1 ;
    }
}
endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
```

```
        foreach (p.arr[i,j])
            $write("arr[%0d][%0d] = %0d ",i,j, p.arr[i][j]);
        $display("");
end
endmodule
```

Simulation output:

```
arr[0][0] = 10 arr[0][1] = 14 arr[0][2] = 8 arr[0][3] = 12 arr[1][0] = 15 arr[1][1] = 1
arr[1][2] = 1 arr[1][3] = 3

          V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

42.0 Write a constraint to generate the Fibonacci sequence

```
class pattern_gen;
        rand int arr[10];


        constraint pattern_c { arr[0]==0; arr[1]==1;
        foreach (arr[i]) {
            if (i >= 2)
                arr[i] == arr[i-1] + arr[i-2];
        }
}
endclass
module tb;
initial begin
```

```
    pattern_gen p = new();
    p.randomize();
    foreach (p.arr[i])
        $write("%0d ", p.arr[i]);
    $display("");
end
endmodule
```

Simulation output:

```
0 1 1 2 3 5 8 13 21 34
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

43.0 Write a constraint such that when rand bit [3:0]a; is randomized. the value of a should not be same as 5 previous occurrences of a?

```
class pattern_gen;
    rand bit [3:0] a;
    bit [3:0] b[$];

    constraint no_repeat_c {
        !(a inside {b});
    }

    function void post_randomize();
```

```
            b.push_back(a);
            if (b.size() > 5)
                b.pop_front();
        endfunction
endclass

module tb;
initial begin
        pattern_gen p = new();
        repeat(10) begin
            p.randomize();
            $display("a= %0d b=%p", p.a ,p.b);
        end
end
endmodule
```

Simulation output:

```
a= 5 b='{'h5}
a= 1 b='{'h5, 'h1}
a= 3 b='{'h5, 'h1, 'h3}
a= 13 b='{'h5, 'h1, 'h3, 'hd}
a= 8 b='{'h5, 'h1, 'h3, 'hd, 'h8}
a= 2 b='{'h1, 'h3, 'hd, 'h8, 'h2}
a= 9 b='{'h3, 'hd, 'h8, 'h2, 'h9}
```

a= 7 b='{'hd, 'h8, 'h2, 'h9, 'h7}

a= 5 b='{'h8, 'h2, 'h9, 'h7, 'h5}

a= 4 b='{'h2, 'h9, 'h7, 'h5, 'h4}

V C S S i m u l a t i o n R e p o r t

Time: 0

44.0 Write a constraint to generate this pattern 0 9 1 99 2 999 3 9999 ....

```
class pattern_gen;
        rand int arr[10];


        constraint pattern_c {
                foreach (arr[i]) {
                        if (i % 2 == 0)
                                arr[i] == i/2;
                        else {
                                if (i == 1)
                                        arr[i] == 9 ;
                                else
                                        arr[i] == 10*arr[i-2] + 9;
                        }


                }
 }
```

```
endclass

module tb;

initial begin

        pattern_gen p = new();

        p.randomize();


        foreach (p.arr[i])

                $write("%0d ", p.arr[i]);

        $display("");

  end

endmodule
```

Simulation output:

```
0 9 1 99 2 999 3 9999 4 99999

          V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

45.0 Write a constraint to randomize a memory address before storing data, ensuring that if the address is odd, the last 2 bits of data are 0, and if the address is even, the last 3 bits of the data are set to 1?

```
class pattern_gen;

        rand bit [3:0] addr;

        rand bit [7:0] data;
```

```
        constraint pattern_c {
                solve addr before data;

        if (addr % 2 != 0)
                data[1:0] == 2'b00 ;
        else
                data[2:0] == 3'b111;
}

endclass
module tb;
initial begin
        pattern_gen p = new();
        repeat(5) begin
                p.randomize();
                $display("addr= %d data =%b", p.addr, p.data);
        end
end
endmodule
```

Simulation output:

```
addr=  1 data =10101100
addr= 15 data =10100100
addr= 11 data =00110000
```

addr= 3 data =01010100

addr= 8 data =00001111

V C S S i m u l a t i o n R e p o r t

Time: 0

46.0 How can we write a constraint to generate an array of size 10 where only two elements are non-zero?

```
class pattern_gen;
        rand bit[3:0] arr[10];
        rand bit [9:0] a;


        constraint two_non_zero_c {
                $countones(a) == 2;


        foreach (arr[i]) {
                if (a[i])
                        arr[i] != 0;
                else
                        arr[i] == 0;
        }
}
endclass
module tb;
initial begin
```

```
        pattern_gen p = new();
        p.randomize();
        foreach (p.arr[i])
                $write("%0d ", p.arr[i]);
        $display("");
end
endmodule
```

Simulation output:

```
0 0 15 0 0 0 0 2 0 0
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

47.0 How can we write a constraint to generate a number where the sum of its digits equal 15?

```
class digit_sum_gen;
  rand int num;
  rand int d1, d2, d3;

  constraint digit_c {
    d1 inside {[0:9]};
    d2 inside {[0:9]};
    d3 inside {[0:9]};

    d1 + d2 + d3 == 15;
```

```
    num == d1*100 + d2*10 + d3;
  }
endclass



module tb;
  initial begin
    digit_sum_gen p = new();

    repeat (5) begin
      p.randomize();
      $display("Number = %0d Digits:%0d + %0d + %0d = 15",p.num, p.d1, p.d2, p.d3);
    end
  end
endmodule
```

Simulation output:

```
Number = 690 Digits:6 + 9 + 0 = 15
Number = 438 Digits:4 + 3 + 8 = 15
Number = 393 Digits:3 + 9 + 3 = 15
Number = 861 Digits:8 + 6 + 1 = 15
Number = 519 Digits:5 + 1 + 9 = 15
          V C S   S i m u l a t i o n   R e p o r t
```

Time: 0

48.0 How can we write a constraint to generate an array where the first element repeats every 4th position?

```
class pattern_gen;
  rand bit[3:0] arr[16];


  constraint repeat_c { foreach (arr[i]) {
        if (i % 4 == 0)
              arr[i] == arr[0];
  }
}
endclass


module tb;
initial begin
      pattern_gen p = new();
      p.randomize();
      foreach (p.arr[i])
            $write("%0d ", p.arr[i]);
      $display("");
end
endmodule
```

Simulation output:

5 3 15 1 5 10 4 3 5 2 9 14 5 0 10 3

V C S S i m u l a t i o n R e p o r t

Time: 0

49.0 How can we write a constraint to generate an array where the first element repeats every 4th position?

```
class pattern_gen;
    rand bit [7:0] data;
    rand int pos;


    constraint three_ones_c { pos inside {[0:5]};
    data == (8'b0000_0111 << pos);
}
endclass
module tb;
initial begin
    pattern_gen p = new();
    repeat (5) begin
        p.randomize();
        $display("data = %b", p.data);
    end
end
endmodule
```

Simulation output:

data = 00111000

data = 00111000

data = 00111000

data = 00011100

data = 00000111

V C S S i m u l a t i o n R e p o r t

Time: 0

50.0 How can we write a constraint to generate an array where exactly 1/2 of the elements are even and the other are odd?

```
class pattern_gen;
	rand bit[4:0] arr[10];


	constraint even_odd_half_c {
		arr.sum() with (int'(item % 2 == 0)) == arr.size()/2;
	arr.sum() with (int'(item % 2 == 1)) == arr.size()/2;
}
endclass


module tb;
  initial begin
    pattern_gen p = new();
    p.randomize();
```

```
    foreach (p.arr[i])
      $write("%0d ", p.arr[i]);
    $display("");
  end
endmodule
```

Simulation output:

19 6 12 30 12 27 19 11 26 9

V C S S i m u l a t i o n R e p o r t

Time: 0

51.0 How can we create a #Task to find the maximum and minimum elements of an array?

```
module tb;

int arr[] = '{12, 45, 7, 89, 23};

task find_min_max(input int a[], output int min, output int max);

        int i;
begin
        min = a[0];
       max = a[0];
```

```
        for (i = 1; i < a.size(); i++) begin
                if (a[i] < min)
                        min = a[i];
                if (a[i] > max)
                        max = a[i];
        end
end
endtask

initial begin
        int min_val, max_val;

        find_min_max(arr, min_val, max_val);
        $display("Array_element = %p", arr);
        $display("Min = %0d, Max = %0d", min_val, max_val);
end
endmodule
```

Simulation output:

```
Array_element = '{12, 45, 7, 89, 23}
Min = 7, Max = 89
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

52.0 How can we write a constraint to generate a sorted array where the difference between consecutive elements is atleast 2?

```
class sorted_array;
	rand bit[4:0] arr[10];


	constraint sorted_c {
		foreach (arr[i]) {
			if (i > 0) {
				arr[i] > arr[i-1];
				arr[i] - arr[i-1] >= 2;
			}
		}
}
endclass
module tb;
initial begin
	sorted_array p = new();
	repeat (3) begin
		p.randomize();
		$display("Sorted Array = %p", p.arr);
	end
end
```

```
endmodule
```

Simulation output:

```
Sorted Array = '{'h0, 'h5, 'h8, 'hf, 'h14, 'h16, 'h18, 'h1a, 'h1d, 'h1f}

Sorted Array = '{'h1, 'h4, 'h6, 'h8, 'ha, 'he, 'h12, 'h16, 'h1a, 'h1c}

Sorted Array = '{'h3, 'h6, 'h9, 'hb, 'hd, 'hf, 'h11, 'h13, 'h1c, 'h1f}

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

53.0 How can we write a constraint to generate a power of 2 without using the multiplication operator? 2 4 8 16 32 64 128 256....?

```
class power2_gen;
        rand bit [9:0] num;
        constraint power2_c {
                num > 1;
        $countones(num) == 1;
}
endclass


module tb;
initial begin
        power2_gen p = new();
        repeat (10) begin
                p.randomize();
```

```
        $display("Power of 2 = %0d in (Hex=%0h)", p.num, p.num);
    end
end
endmodule
```

Simulation output:

```
Power of 2 = 128 in (Hex=80)
Power of 2 = 32 in (Hex=20)
Power of 2 = 2 in (Hex=2)
Power of 2 = 64 in (Hex=40)
Power of 2 = 2 in (Hex=2)
Power of 2 = 128 in (Hex=80)
Power of 2 = 512 in (Hex=200)
Power of 2 = 64 in (Hex=40)
Power of 2 = 16 in (Hex=10)
Power of 2 = 16 in (Hex=10)
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

54.0 How can we write a constraint to generate an array where elements at even positions are positive and elements are at odd positions are negative?

```
class arr_gen;
    rand byte signed arr[10];
```

```
    constraint even_pos_odd_neg_c { foreach (arr[i]) {
        if (i % 2 == 0)
            arr[i] > 0;
        else
            arr[i] < 0;
    }
}
endclass
module tb;
arr_gen a = new();

initial begin
    a.randomize();
    $display("Array = %p", a.arr);
end
endmodule
```

Simulation output:

```
Array = '{48, -13, 33, -12, 88, -65, 119, -97, 56, -85}
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

55.0 Write a constraint on system verilog to generate 10-digit number of 5 times but start of first three digit should be 498?

```
class pattern_gen;
```

```
    rand bit[63:0] num;


    constraint pattern_c {
        num inside {[64'd4980000000 : 64'd4989999999]};
    }
endclass


module tb;
initial begin
    pattern_gen p = new();
    repeat (5) begin
        if (p.randomize())
            $display("Generated Number = %0d", p.num);
        else
            $display("Randomization Failed");
    end
end
endmodule
```

Simulation output:

```
Generated Number = 4984579949

Generated Number = 4981415623

Generated Number = 4988513671

Generated Number = 4988856662
```

Generated Number = 4983700787

V C S S i m u l a t i o n R e p o r t

Time: 0

56.0 Write a constraint on system Verilog to generate 10-digit number of 5 times but end of last three digit should be 498?

```
class pattern_gen;
   rand bit [63:0] num;


   constraint pattern_c {
      num inside {[64'd1000000000 : 64'd9999999999]};
      num % 1000 == 498;
   }
endclass
module tb;
initial begin
   pattern_gen p = new();
   repeat (5) begin
      if (p.randomize())
         $display("Generated 10_Digit_Mobile_Number = %0d", p.num);
      else
         $display("Randomization Failed");
   end
end
```

```
endmodule
```

Simulation output:

```
Generated 10_Digit_Mobile_Number = 3303243498

Generated 10_Digit_Mobile_Number = 5854339498

Generated 10_Digit_Mobile_Number = 4504728498

Generated 10_Digit_Mobile_Number = 7889484498

Generated 10_Digit_Mobile_Number = 2281032498

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

57.0 How can we write a constraint to generate Real Numbers Between 0 and 1 ?

```
class pattern_gen;
   rand int tmp;
   real num;

   constraint pattern_c {
      tmp > 0 ;
      tmp < 100 ;
   }

   function void post_randomize();
         num = tmp/100.0;
   endfunction
```

```
endclass

module tb;

initial begin

    pattern_gen p = new();

    repeat (5) begin

        p.randomize();

        $display("Print the real number b/n 0_1 = %0f", p.num);

    end

end

endmodule
```

Simulation output:

```
Print the real number b/n 0_1 = 0.030000

Print the real number b/n 0_1 = 0.170000

Print the real number b/n 0_1 = 0.150000

Print the real number b/n 0_1 = 0.730000

Print the real number b/n 0_1 = 0.500000

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

58.0 How can we write a constraint to generate this pattern 000111222333444555…..

```
class pattern_gen;

    rand int arr[12];
```

```
    constraint pattern_c {foreach (arr[i]) {
            if (i < 3)
                arr[i] ==0 ;
            else
                arr[i] == arr[i-3] + 1;
    }
 }

endclass
module tb;
initial begin
    pattern_gen p = new();
    p.randomize();
    $display("Print the number = %p", p.arr);

end
endmodule
```

Simulation output:

```
Print the number = '{0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3}
            V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

59.0 How can we write a constraint to generate an array with a size between 5 to 10, ensuring the values are in ascending order?

```
class pattern_gen;
   rand bit[3:0] arr[];


   constraint pattern_c { arr.size() inside {[5:10]};
   foreach (arr[i]) {
         if (i > 0)
               arr[i] > arr[i-1];
   }
}


endclass
module tb;
initial begin
   pattern_gen p = new();
   repeat(5) begin
         p.randomize();
         $display("Print Array= %p", p.arr);
   end


end
```

endmodule

Simulation output:

Print Array= '{'h6, 'h8, 'h9, 'he, 'hf}

Print Array= '{'h1, 'h2, 'h5, 'h6, 'h7, 'hb}

Print Array= '{'h0, 'h2, 'h6, 'h8, 'ha}

Print Array= '{'h0, 'h2, 'h7, 'h8, 'ha, 'hb, 'hc, 'hd, 'he}

Print Array= '{'h0, 'h1, 'h2, 'h5, 'ha, 'hd, 'he, 'hf}

V C S S i m u l a t i o n R e p o r t

Time: 0

60.0 Explain only task and function in Verilog with examples in simple way?

☑ 1. 0 Function in Verilog

A function is used to perform a calculation and return one value.

Think like:

- Function = Small calculator that returns a value.

Rules of Function

- Must return only one value
- Cannot contain delay (#)
- Cannot contain @ or wait
- Executes in zero simulation time

Example 1: Addition Function

```
module tb;

function integer add;

    input integer a, b;

    begin

        add = a + b;   // return value

    end

endfunction


initial begin

    $display("Sum = %0d", add(10, 5));

end

endmodule
```

Output:

```
Sum = 15
```

◇ Where Function is Used?

- Combinational calculations
- Arithmetic operations
- Priority encoders
- Small logic blocks

☑ 2 Task in Verilog

A task is used to perform a group of operations.

Think like:

Task = Mini program inside a module.

◇ Rules of Task

- Can return multiple values
- Can contain delay (#)
- Can contain @, wait
- Can consume simulation time

Example 2: Task with Delay

```
module tb;

task display_msg;
   input [7:0] data;
   begin
      #5;
      $display("Data = %0d at time %0t", data, $time);
   end
endtask


initial begin
   display_msg(25);
end
endmodule
```

Output:

```
Data = 25 at time 5
```

See 👆 it consumed time → so must be task.

☑ Difference Between Task and Function

Feature	Function	Task
Return value	Only one	Zero or multiple
Delay allowed	❌ No	☑ Yes
@ / wait allowed	❌ No	☑ Yes
Simulation time	0 time	Can consume time

🎯 Simple Real-Life Analogy

- Function = Calculator → gives answer instantly
- Task = Cooking recipe → takes time and multiple steps

🎯 Interview Trick Question

❓ Can function call a task?

👉 ❌ No (because task consumes time)

❓ Can task call function?

👉 ☑ Yes

61.0 Explain only difference between task/function in Verilog vs SystemVerilog with example in simple way?

☑ 1 Return Type Difference

🔹 In Verilog

- Function must return only one value

- Return type is written before function name

```
function integer add;
   input integer a, b;
   begin
      add = a + b;
   end
endfunction
```

◇ **In SystemVerilog**

- You can write return type after `function`
- You can use `void`
- More C-like style

```
function int add(int a, int b);
   return a + b;
endfunction
```

✓ Cleaner

✓ Modern syntax

☑ 2 Argument Passing

◇ Verilog

- Only input/output/inout
- Pass by value only

```
task swap;
    input a;
    output b;
endtask
```

◇ **SystemVerilog**

Supports:

- `input`
- `output`
- `inout`
- `ref` (pass by reference)
- `const ref`

```
task swap(ref int a, ref int b);
    int temp;
    temp = a;
    a = b;
    b = temp;
endtask
```

🔥 `ref` is NOT available in Verilog.

☑ 3 Default Arguments (Only in SystemVerilog)

```
function int multiply(int a, int b = 2);
    return a * b;
```

```
endfunction
```

Call:

```
multiply(5);   // b automatically = 2
```

❌ Not possible in Verilog.

☑ 4 Automatic Variables

◇ **Verilog**

Functions/tasks are static by default.

◇ **SystemVerilog**

You can use `automatic` (re-entrant)

```
function automatic int factorial(int n);
   if (n <= 1)
      return 1;
   else
      return n * factorial(n-1);
endfunction
```

- Recursive functions ❌ not safe in Verilog
- Recursive functions ☑ safe in SystemVerilog

☑ 5 Data Types

SystemVerilog supports:

- Int
- Bit
- Logic
- String
- class handle
- dynamic array
- struct
- enum

Verilog supports only:

- reg
- wire
- integer
- real

☑ Summary Table

| Feature | Verilog | SystemVerilog |

| ---------- | ---------- | ---------- |

| Syntax style | Old style | C-like modern |

| `void` function | ❌ No | ☑ Yes |

| `ref` arguments | ❌ No | ☑ Yes |

| Default arguments | ❌ No | ☑ Yes |

| Recursion safe | ❌ No | ☑ Yes (automatic) |

| Advanced data types | ❌ No | ☑ Yes |

🎯 Interview-Level Answer (Short Version)

SystemVerilog enhances Verilog tasks and functions by adding modern C-like syntax, pass-by-reference (`ref`), default arguments, automatic storage, recursion support, and advanced data types. Verilog supports only basic static tasks/functions with limited argument and type support.

62.0 Explain the blocking and non-blocking assignments in Verilog with a example in simple way?

☑ 1 Blocking Assignment (`=`)

Symbol:

=

Meaning:

- Execution happens line by line (sequentially).

👉 Next statement waits until previous statement finishes.

◇ Example: Blocking

```
module tb;

reg a, b, c;


initial begin

   a = 1;

   b = a;

   c = b;

   $display("a=%0d b=%0d c=%0d", a, b, c);

end

endmodule
```

◇ Execution Flow:

1. `a = 1`

2. `b = a` → b = 1

3. `c = b` → c = 1

☑ Output:

a=1 b=1 c=1

✓ Works like normal programming (C language style).

☑ 2 Non-Blocking Assignment (`<=`)

Symbol:

<=

Meaning:

- All assignments happen at the same time (parallel) at end of time step.

👉 RHS calculated first

👉 Update happens together

Example: Non-Blocking

```
module tb;

reg a, b, c;


initial begin
    a <= 1;
    b <= a;
    c <= b;
    #1;
```

```
    $display("a=%0d b=%0d c=%0d", a, b, c);
end
endmodule
```

🔷 Execution Flow:

At time 0:

- a gets 1
- b gets old value of a
- c gets old value of b

So initially a, b, c were 0.

After update:

a = 1

b = 0

c = 0

☑ Output:

```
a=1 b=0 c=0
```

- Most Important Example (Flip-Flop Case)

❌ Wrong Way (Blocking in sequential logic)

```
always @(posedge clk) begin
    q1 = d;
    q2 = q1;
end
```

Problem:

- q2 immediately gets updated q1
- Acts like combinational logic
- No pipeline behavior

☑ Correct Way (Non-Blocking in sequential logic)

always @(posedge clk) begin

q1 <= d;

q2 <= q1;

end

Now:

- q1 updates first clock
- q2 updates next clock
- Correct pipeline behavior

☑ Simple Rule (Very Important)

| Use Case | Assignment Type |

| ---------- | ---------- |

| Combinational logic | `=` (blocking) |

| Sequential logic (flip-flop) | `<=` (non-blocking) |

🎯 Real-Life Analogy

- Blocking (`=`)

Like washing dishes one by one:

- Finish plate 1
- Then plate 2
- Then plate 3

- Non-blocking (`<=`)

Like students submitting exam paper:

- Everyone writes first
- Teacher collects all together

🎯 Interview One-Line Answer

Blocking assignments execute sequentially and update immediately, while non-blocking assignments execute in parallel and update at the end of the time step. Blocking is used in combinational logic, and non-blocking is used in sequential logic.

63.0 Explain only timing region of operation in Verilog with example in simple way?

Timing regions are very important for understanding race conditions and blocking vs non-blocking behavior in simulation.

☑ What is Timing Region in Verilog?

In one simulation time step, Verilog does not execute everything at once.

It divides execution into different regions (phases).

These regions control:

- When RHS is evaluated
- When LHS is updated
- When `$display` prints

☑ Main Timing Regions (Simplified View)

We'll focus on the 3 important ones:

1 Active Region

2 Inactive Region

3 NBA (Non-Blocking Assignment) Region

1. 0 Active Region

👉 All blocking assignments (`=`) execute here.

👉 RHS of non-blocking (`<=`) is also evaluated here.

Example:

always @(posedge clk) begin

    a = b;   // blocking → Active region

    c <= d;   // RHS evaluated here

end

2 Inactive Region

👉 Executes statements with `#0` delay.

Example:

#0 a = 5;

This runs after active region in same time step.

3 NBA Region (Non-Blocking Assignment Region)

👉 LHS of `<=` updates here.

👉 All non-blocking assignments update together.

This prevents race conditions.

🔥 Simple Example

```
module tb;

reg a, b;


initial begin

    a = 0;

    b = 1;


    a <= b;

    b <= a;

    #1;

    $display("a=%0d b=%0d", a, b);

end

endmodule
```

🎾 **Step-by-Step Execution**

At time 0:

Active Region:

- RHS evaluated
- `a <= b` → schedule a = 1
- `b <= a` → schedule b = 0

(No updates yet!)

NBA Region:

Now updates happen together:

a = 1

b = 0

Output:

a=1 b=0

If blocking was used, result would be different (race issue).

🎯 Why Timing Regions Are Important?

Because they:

- Prevent race conditions
- Control update order
- Define how simulation behaves

☑ Simple Timeline View

Time = 0

Active Region

Inactive Region

NBA Region

Time moves forward

🎯 Interview One-Line Answer

In Verilog, each simulation time step is divided into timing regions like Active, Inactive, and NBA. Blocking assignments execute in the Active region, while non-blocking assignments update in the NBA region, ensuring proper sequential behavior and avoiding race conditions.

Since you're working on RTL and verification, understanding NBA region is extremely important for debugging pipelines and clocked logic 🔥

64.0 Explain the complete 5-region model including monitor and postponed region (advanced level).

advanced simulation concept — very important for debugging race conditions in RTL + testbench. Complete 5-region event scheduling model in Verilog in simple way.

☑ Complete 5 Timing Regions (Advanced Model)

In one simulation time step, execution happens in this order:

1️⃣ Active

2️⃣ Inactive

3️⃣ NBA (Non-Blocking Assignment)

4️⃣ Monitor

5️⃣ Postponed

1.0 Active Region

👉 Executes:

- Blocking assignments (`=`)
- RHS of non-blocking (`<=`)
- Continuous assignments
- $display

Example:

```
a = b;      // runs in Active

c <= d;     // RHS evaluated in Active

$display("Hello");
```

2.0 Inactive Region

👉 Executes `#0` delay statements.

Example:

#0 a = 5;

- It executes after Active region, but still same simulation time.
- Used rarely in good RTL coding.

3.0 NBA Region (Non-Blocking Assignment Region)

👉 LHS updates of `<=` happen here.

👉 All updates happen together (parallel).

Example:

always @(posedge clk) begin

q1 <= d;

q2 <= q1;

end

- Both updates happen in NBA region.
- This avoids race conditions.

4.0 Monitor Region

👉 Used for:

- `$monitor`
- `$strobe`

These execute after NBA updates.

Meaning: they print final stable values.

Example:

initial begin

$monitor("a=%0d b=%0d", a, b);

end

- It prints values after all updates are done.

5.0 Postponed Region

👉 Used for:

- Final value sampling
- Assertion checking (SystemVerilog)

Executes after everything is stable.

Think of it as:

- Final check stage of simulation time step.

🔥 Full Example to Understand Order

module tb;

reg a, b;

initial begin

a = 0;

b = 1;

a <= b;

b <= a;

$display("Display: a=%0d b=%0d", a, b);

$strobe("Strobe: a=%0d b=%0d", a, b);

end

endmodule

🔎 Execution Order

◇ **Active Region**

- RHS evaluated
- display executes → prints old values

Output:

Display: a=0 b=1

◇ **NBA Region**

a = 1

b = 0

- Monitor/Postponed Region
- `$strobe` prints updated values

Output:

Strobe: a=1 b=0

🎯 Final Execution Flow

Time = 0

Active → Blocking + RHS

Inactive → #0 delays

NBA → <= updates

Monitor → $monitor / $strobe

Postponed → Assertions / final checks

Time advances

🔥 Why This Is Important for You

Since you're working with:

- RTL pipelines
- Testbench debugging
- Verification flows

Understanding this helps you:

✓ Debug race conditions

✓ Understand why $display shows old values

✓ Know why <= behaves correctly

✓ Debug clocked logic issues

🎯 Interview-Level Answer (Advanced)

In Verilog simulation, each time step is divided into Active, Inactive, NBA, Monitor, and Postponed regions. Blocking assignments execute in the Active region, non-blocking updates occur in the NBA region, and monitoring tasks like $strobe execute after updates. This scheduling ensures deterministic simulation and prevents race conditions.

Top 15 important interview question for design and verification role on Complete 5 timing regions (advance level)

5 timing regions in Verilog (Active, Inactive, NBA, Monitor, Postponed):

☑ Top 15 Interview Questions on 5 Timing Regions (Advanced)

1.0 What are the 5 timing regions in Verilog simulation?

Expected Answer:

Active → Inactive → NBA → Monitor → Postponed

2.0 In which region do blocking assignments execute?

👉 Active region

3.0 In which region does the LHS of non-blocking assignment update?

👉 NBA region

Very common question.

4.0 Where is RHS of non-blocking assignment evaluated?

👉 Active region

This is a tricky one 💧

5.0 Why are non-blocking assignments used in sequential logic?

Expected answer:

Because updates happen in NBA region simultaneously, avoiding race conditions.

6.0 What happens if we use blocking assignments in clocked always block?

Expected:

Race condition + incorrect pipeline behavior

Executes in Active region immediately.

7.0 What is the purpose of Inactive region?

Expected:

Executes `#0` delayed statements within same time step.

Follow-up:

“Is #0 recommended in RTL?”

Correct answer: ❌ No (bad practice)

8.0 Difference between $display and $strobe?

Expected:

- `$display` → Active region
- `$strobe` → Monitor region (after NBA updates)

This checks deep understanding 🔥

9.0 When does $monitor execute?

👉 Monitor region

It prints values after all updates are completed.

10 Why does $display sometimes show old values?

Because it executes in Active region before NBA updates.

Very common debugging question.

11 What is the purpose of Postponed region?

Expected:

- Final value sampling
- Assertion checking (mainly SystemVerilog)

12 Explain race condition using timing regions.

Expected explanation:

If two always blocks use blocking assignments in Active region, execution order becomes simulator-dependent → race.

13 Can two non-blocking assignments cause race condition?

Expected:

Generally No, because updates happen in NBA region simultaneously.

14 What is the event scheduling order within one time step?

Expected:

Active

Inactive

NBA

Monitor

Postponed

In correct order.

15.0 Debug Scenario Question (Very Common)

Interviewer gives:

```
always @(posedge clk)
    q = d;


always @(posedge clk)
    r = q;
```

Question:

Why is r not delayed by one clock?

Expected answer:

Because blocking assignments execute in Active region, so q updates immediately before r assignment → no pipeline effect.

Correct fix:

Use non-blocking assignments.

🔥 Most Important Concepts They Test

They want to see if you understand:

✓ RHS vs LHS evaluation timing

✓ Why <= prevents race

✓ Why $display shows old value

✓ Why blocking breaks pipelines

✓ Why #0 is dangerous

🎯 One Strong Interview Closing Line

If they ask summary:

Verilog divides each simulation time step into multiple scheduling regions to ensure deterministic execution. Blocking assignments execute in the Active region, while non-blocking assignments update in the NBA region. Monitoring tasks execute after updates, preventing race conditions and ensuring stable value sampling.

Top 15 important interview question for design and verification role on Complete 5 timing regions(advance level) only coding questions and answer?

Top 15 Advanced Coding-Based Interview Questions on the 5 timing regions.

1.0 Blocking vs NBA Basic Swap

```
module tb;

reg a=0,b=1;

initial begin

  a <= b;

  b <= a;

  #1 $display("%0d %0d",a,b);
```

```
end

endmodule
```

☑ Output

```
1 0
```

✓ RHS evaluated in Active

✓ Update in NBA region

2.0 Same Code with Blocking

```
module tb;

reg a=0,b=1;

initial begin

  a = b;

  b = a;

  $display("%0d %0d",a,b);

end

endmodule
```

☑ Output:

```
1 1
```

✓ Blocking executes immediately in Active region

☑ **Simple Explanation (Short):**

- a = b; → a becomes **1**
- b = a; → now b gets updated value of **a (which is 1)**

Because **blocking (=) executes line by line in Active region**,
so updates happen immediately.

3.0 $display vs $strobe

```
module tb;

reg a=0;

initial begin

  a <= 1;

  $display("Display a=%0d",a);

  $strobe("Strobe a=%0d",a);

end

endmodule
```

Output:

Display a=0

Strobe a=1

✓ $display → Active

✓ $strobe → Monitor region

4.0 #0 Delay Effect

```
module tb;

reg a=0;

initial begin
```

```
  a = 5;
  #0 a = 10;
  $display("%0d", a);
end
endmodule
```

Output:

10

✓ #0 executes in Inactive region

5.0 Pipeline Wrong Coding

```
always @(posedge clk) begin
  q1 = d;
  q2 = q1;
end
```

❌ Problem:

q2 gets updated immediately

No 1-cycle delay

✓ Because blocking executes in Active region

6.0 Correct Pipeline

```
always @(posedge clk) begin
```

```
  q1 <= d;
  q2 <= q1;
end
```

✓ Updates in NBA

✓ Correct 1-cycle delay

7.0 Two Always Blocks (Race)

```
always @(posedge clk)
  a = b;
always @(posedge clk)
  b = a;
```

❌ Result:

- Race condition
- Execution order simulator dependent

✓ Both in Active region

8.0 Same with Non-Blocking

```
always @(posedge clk)
  a <= b;
always @(posedge clk)
  b <= a;
```

✓ Safe

✓ No race

✓ Update in NBA region

9.0 RHS Evaluation Timing

```
module tb;
reg a=1,b=2;
initial begin
  a <= b;
  b <= 5;
  #1 $display("%0d",a);
end
endmodule
```

Output:

2

✓ RHS evaluated first in Active

10.0 Mixed Blocking & NBA

```
module tb;
reg a=0,b=1;
initial begin
  a = b;
  b <= a;
  #1 $display("%0d %0d",a,b);
end
```

```
endmodule
```

Output:

1 1

✓ Blocking executes first

✓ NBA uses updated value

11.0 Multiple NBA Same Variable

```
module tb;
reg a=0;
initial begin
  a <= 1;
  a <= 2;
  #1 $display("%0d",a);
end
endmodule
```

Output:

2

✓ Last NBA wins

12.0 Active vs NBA Ordering

```
module tb;
reg a=0;
```

```
initial begin
  a <= 1;
  a = 2;
  #1 $display("%0d",a);
end
endmodule
```

Output:

1

✓ Blocking updates immediately

✓ NBA overwrites in NBA region

13.0 $monitor Example

```
module tb;
reg a=0;
initial begin
  $monitor("a=%0d",a);
  a <= 5;
end
endmodule
```

Output:

a=5

✓ $monitor prints after NBA update

14.0 Zero Time Multiple Updates

```
module tb;

reg a=0;

initial begin

  a = 1;

  a <= 2;

  $display("a=%0d",a);

end

endmodule
```

Output:

a=1

✓ $display in Active

✓ NBA update not happened yet

15.0 Classic Debug Question

```
module tb;

reg a=0;

initial begin

  a <= 1;

  #0 $display("a=%0d",a);

end

endmodule
```

Output:

a=0

✓ #0 executes in Inactive

✓ NBA update happens after that

🔥 What Interviewer Checks

- Active vs NBA understanding
- Race condition detection
- $display vs $strobe difference
- #0 knowledge
- Pipeline correctness
- Mixed assignment debugging

10 Real race-condition debugging interview Traps: -

1.0 Two Always Blocks – Blocking Race

```
always @(posedge clk)
  a = b;
always @(posedge clk)
  b = a;
```

❌ **Problem:**

Both execute in Active region.

Execution order is simulator-dependent → race condition.

☑ **Fix:**

```
always @(posedge clk)
  a <= b;
```

```
always @(posedge clk)
  b <= a;
```

✓ **Updates happen in NBA region → no race.**

2 Pipeline Broken by Blocking

```
always @(posedge clk) begin
  q1 = d;
  q2 = q1;
end
```

❌ Problem:

q2 updates immediately → no 1-cycle delay.

☑ Fix:

```
always @(posedge clk) begin
  q1 <= d;
  q2 <= q1;
end
```

✓ Correct pipeline behavior.

3 Mixed Blocking and Non-Blocking

```
always @(posedge clk) begin
  a = b;
  c <= a;
end
```

❌ Problem:

c gets updated value of a (not previous cycle value).

☑ Fix:

Use consistent NBA:

```
always @(posedge clk) begin
  a <= b;
  c <= a;
end
```

4.0 $display Shows Wrong Value

```
initial begin
  a <= 5;
  $display("a=%0d",a);
end
```

❌ Output:

a=0

Because `$display` executes in Active region.

☑ Fix:

Use `$strobe`:

```
$strobe("a=%0d",a);
```

5.0 Multiple Drivers Same Variable

```
always @(posedge clk)
  a <= 1;
```

```
always @(posedge clk)
  a <= 2;
```

❌ Problem:

Last assignment wins (NBA overwrite).

Creates unpredictable behavior.

☑ Fix:

Combine logic in one always block.

6.0 Testbench vs DUT Race

```
// DUT
always @(posedge clk)
  q <= d;

// TB
always @(posedge clk)
  d = 1;
```

❌ Problem:

TB uses blocking → runs in Active region

May update before DUT samples → race.

☑ Fix:

Use NBA in TB or drive on negedge.

```
always @(posedge clk)
  d <= 1;
```

7.0 #0 Delay Misuse

```
initial begin
  a = 1;
  #0 a = 2;
end
```

❌ Problem:

Creates Inactive region update

Hard to debug

Bad RTL practice

☑ Fix:

Avoid #0 in RTL.

8.0 Read-After-Write Race

```
always @(posedge clk)
  a = b;
always @(posedge clk)
  c = a;
```

❌ **Problem:**

c may get new or old value depending on execution order.

☑ Fix:

Use NBA:

```
always @(posedge clk)
  a <= b;
always @(posedge clk)
  c <= a;
```

9.0 Incorrect Reset Handling

```
always @(posedge clk or posedge rst)
  if (rst)
    q = 0;
  else
    q <= d;
```

❌ Problem:

Mixing blocking and non-blocking in same block.

☑ Fix:

Use consistent NBA:

```
always @(posedge clk or posedge rst)
  if (rst)
    q <= 0;
  else
    q <= d;
```

10.0 Non-Blocking Overwrite Trap

```
initial begin
  a <= 1;
  a <= 2;
  a <= 3;
  #1 $display("%0d",a);
end
```

❌ Output:

3

Only last NBA survives.

☑ Fix:

Avoid multiple NBA assignments to same signal in same time step.

🎯 What Interviewer Is Testing

They want to check if you understand:

- Active vs NBA region
- Execution ordering
- Pipeline correctness
- Testbench/DUT race
- Why <= is required for sequential logic
- Why mixing = and <= is dangerous

Strong Interview Closing Line

If they ask:

“What is the root cause of most RTL race conditions?”

Answer:

Most race conditions occur because blocking assignments execute in the Active region and depend on execution order. Using non-blocking assignments in sequential logic ensures updates happen in the NBA region simultaneously, preventing race conditions.

Real interview rapid-fire timing questions

1.0 What is the Output?

```
module tb;

reg a=0;

initial begin

  a <= 5;
```

```
  a = 3;
  #1 $display("%0d",a);
end
endmodule
```

Answer:

5

✓ Blocking sets a=3 in Active

✓ NBA later updates a=5

2.0 Output?

```
module tb;
reg a=1,b=2;
initial begin
  a <= b;
  b = 5;
  #1 $display("%0d",a);
end
endmodule
```

Answer:

2

✓ RHS evaluated first (Active region)

3.0 $display vs $strobe

```
module tb;
reg a=0;
initial begin
  a <= 10;
  $display("D=%0d",a);
  $strobe("S=%0d",a);
end
endmodule
```

Output:

D=0

S=10

✓ display → Active

✓ strobe → Monitor

4.0 Race Condition?

```
always @(posedge clk)
  a = b;
always @(posedge clk)
  b <= a;
```

Answer:

YES ❌

Because blocking executes in Active

NBA scheduled later → race possible.

5.0 Output?

```
module tb;
```

```
reg a=0;
initial begin
  a <= 1;
  #0 $display("%0d",a);
end
endmodule
```

Output:

0

✓ #0 → Inactive

✓ NBA not updated yet

6.0 Output?

```
module tb;
reg a=0;
initial begin
  a = 1;
  a <= 2;
  $display("%0d",a);
end
endmodule
```

Output:

1

✓ display in Active

✓ NBA not executed yet

7.0 Final Value?

```
module tb;
reg a=0;
initial begin
  a <= 1;
  a <= 2;
  a <= 3;
  #1 $display("%0d",a);
end
endmodule
```

Output:

3

✓ Last NBA wins

8.0 Pipeline Behavior?

```
always @(posedge clk) begin
  q1 = d;
  q2 <= q1;
end
```

☑ Answer:

Incorrect pipeline ✗

q2 may get updated value immediately due to blocking.

9.0 Safe or Unsafe?

```
always @(posedge clk)
  a <= b;
always @(posedge clk)
  b <= a;
```

Answer:

SAFE ☑

Both update in NBA region.

10.0 Why does this print old value?

```
initial begin
  a <= 5;
  $display("%0d",a);
end
```

Answer:

Because $display executes in Active region before NBA update.

Most Important Rapid-Fire Concepts

Interviewers test:

✓ RHS vs LHS timing

✓ Active vs NBA

✓ #0 understanding

✓ display vs strobe

✓ Blocking inside clocked block

✓ Mixed assignment traps

10 waveform-based puzzles (very tricky)

1.0 Basic NBA Pipeline

```
module tb;
reg clk=0, d=1;
reg q1=0, q2=0;

always #5 clk = ~clk;

always @(posedge clk) begin
  q1 <= d;
  q2 <= q1;
end
endmodule
```

❓ What happens after 1st clock edge?

Answer:

At first posedge:

- q1 = 1
- q2 = 0

At second posedge:

- q1 = 1
- q2 = 1

✓ Correct 1-cycle delay pipeline.

2.0 Same Code with Blocking

```
always @(posedge clk) begin
  q1 = d;
```

```
  q2 = q1;
end
```

❓ What happens?

☑ Answer:

At first posedge:

- q1 = 1
- q2 = 1 ❌

✓ No pipeline delay

✓ Because blocking executes sequentially in Active region

3.0 Mixed Assignment Trap

```
always @(posedge clk) begin
  q1 = d;
  q2 <= q1;
end
```

Result:

q2 may get updated immediately

Behavior depends on ordering → unsafe design

4.0 Cross Always Block Race

```
always @(posedge clk)
  a = b;
always @(posedge clk)
  b = a;
```

Initial:

a=0, b=1

❓ After first clock?

☑ Answer:

Undefined (race condition)

✓ Both execute in Active

✓ Order simulator dependent

5.0 Safe Cross-Assignment

```
always @(posedge clk)
  a <= b;
always @(posedge clk)
  b <= a;
```

Initial:

a=0, b=1

☑ After first clock:

a=1

b=0

✓ Both evaluated first

✓ Updated together in NBA

6.0 $display vs $strobe Waveform

```
initial begin
  a <= 5;
  $display("D=%0d",a);
  $strobe("S=%0d",a);
end
```

☑ Waveform:

At time 0:

- display prints old value
- NBA updates
- strobe prints new value

Output:

D=0

S=5

7.0 #0 Delay Trap

```
initial begin
  a <= 10;
  #0 $display("%0d",a);
end
```

Output:

0

✓ #0 runs in Inactive

✓ NBA not executed yet

8.0 Multiple NBA Same Cycle

always @(posedge clk) begin

  a <= 1;

  a <= 2;

end

❓ After clock?

☑ Answer:

a = 2

✓ Last NBA wins

9.0 Read After Write Race

always @(posedge clk)

  a = b;

always @(posedge clk)

  c = a;

❓ Does c get new or old a?

Answer:

Depends on execution order ❌

Race condition.

10.0 Advanced Testbench vs DUT Race

// DUT

always @(posedge clk)

```
  q <= d;

// TB
always @(posedge clk)
  d = 1;
```

❓ Is this safe?

❌ No.

TB updates in Active

DUT samples maybe before or after → race

☑ Fix:

Drive d using non-blocking or on negedge.

🎯 Master Rule for You (Very Important)

- In sequential logic:

Use <= everywhere

- In combinational logic:

Use =

- Never mix them in same always block.

15 toughest real interview puzzles (Expert level)

1.0 NBA Overwrite Trap

```
initial begin
  a <= 1;
```

```
  a <= a + 1;

  #1 $display("%0d", a);

end
```

Output:

1

🔥 Why:

Both RHS evaluated in Active (a=0 initially).

So:

- a <= 1
- a <= 0+1 = 1
- Last NBA wins → 1

2 Self-Referencing NBA

```
initial begin

  a = 5;

  a <= a + 1;

  a <= a + 2;

  #1 $display("%0d", a);

end
```

Output:

7

🔥 Why:

Both RHS see a=5

So:

- 5+1=6
- 5+2=7

Last NBA wins → 7

3.0 Blocking Inside Clock

always @(posedge clk) begin

a = b;

b <= a;

end

☑ Behavior:

b gets updated value of a (new value)

Pipeline broken ❌

🔥 Why:

Blocking updates immediately in Active.

4.0 Display Misleading Debug

initial begin

a <= 10;

```
  $display("a=%0d", a);
  #1;
end
```

Output:

a=0

🔥 Why:

Display executes in Active before NBA.

5.0 Two Always Same Signal (NBA)

```
always @(posedge clk)
  a <= 1;
always @(posedge clk)
  a <= 2;
```

☑ Behavior:

a becomes 2 (last scheduled wins)

🔥 Why:

Both scheduled in NBA region.

Order not guaranteed → bad design.

6.0 Race Between TB & DUT

```
// DUT
```

```
always @(posedge clk)
  q <= d;
// TB
always @(posedge clk)
  d = 1;
```

❌ Unsafe

🔥 Why:

TB blocking runs in Active → race with DUT sampling.

7.0 #0 Delay Confusion

```
initial begin
  a <= 5;
  #0;
  a <= 10;
  #1 $display("%0d",a);
end
```

Output:

10

🔥 Why:

Second NBA scheduled later → overwrites first.

8.0 Combinational Loop Trap

always @(*) a = b;

always @(*) b = a;

❌ Infinite loop / unstable

🔥 Why:

Continuous re-triggering.

9.0 Partial Assignment Trap

always @(posedge clk)

  if (en)

    a <= 1;

❓ What if en=0?

☑ a retains old value

🔥 Why:

No assignment → holds previous value (flip-flop behavior)

10.0 Blocking in Combinational with Missing Else

always @(*) begin

  if (sel)

    y = a;

end

❌ Latch inferred

🔥 Why:

y not assigned in all paths.

11.0 Multiple NBA Same Block

always @(posedge clk) begin

  a <= b;

  a <= c;

end

☑ a gets c

🔥 Why:

Last NBA wins.

12.0 RHS Evaluation Trap

initial begin

  a = 5;

  a <= a + 1;

  a = 10;

  #1 $display("%0d",a);

end

Output:

6

🔥 Why:

RHS evaluated when a=5

Later blocking sets a=10

NBA overwrites with 6

13.0 Reset Mixed Style Trap

always @(posedge clk or posedge rst)

if (rst)

q = 0;

else

q <= d;

❌ Bad practice

🔥 Why:

Mixing blocking & NBA inside same block.

14.0 Event Order Dependency

always @(posedge clk)

```
  a = b;

always @(posedge clk)

  c <= a;
```

❌ Race

c may get new or old value.

15.0 Simulation vs Synthesis Mismatch

```
always @(posedge clk)

  if (en)

    a = b;
```

❌ Simulation race possible

🔥 Why:

Blocking in sequential logic may behave differently in synthesis.

🚀 Strong Expert-Level Line

If interviewer asks:

“What is the most dangerous timing mistake in RTL coding?”

Answer:

Mixing blocking and non-blocking assignments in clocked logic creates race conditions because blocking executes in the Active region while non-blocking updates occur in the NBA region.

5 real tricky coding problems based on timing regions

1.0 Active vs NBA Overwrite Trap

```
module tb;

reg a=0;


initial begin

  a <= 5;

  a = 3;

  #1 $display("%0d", a);

end

endmodule
```

Output:

5

🔥 Why:

- `a = 3` → executes immediately in Active region
- `a <= 5` → scheduled in NBA region
- NBA update happens later → overwrites 3

👉 Final value = 5

2.0 Multiple NBA Self-Reference Trap

```
module tb;

reg a=10;
```

```
initial begin
  a <= a + 1;
  a <= a + 2;
  #1 $display("%0d", a);
end
endmodule
```

Output:

12

🔥 Why:

Both RHS evaluated in Active region

Both see `a = 10`

So scheduled:

- 11
- 12
- Last NBA wins → 12

3.0 $display vs $strobe Timing Puzzle

```
module tb;
reg a=0;

initial begin
  a <= 7;
```

```
  $display("D=%0d", a);
  $strobe("S=%0d", a);
end
endmodule
```

Output:

```
D=0
S=7
```

🔥 Why:

- `$display` → Active region (before NBA update)
- NBA update happens
- `$strobe` → Monitor region (after NBA)

4.0 Cross Always Block Race

```
module tb;
reg clk=0, a=0, b=1;
always #5 clk = ~clk;

always @(posedge clk)
  a = b;
always @(posedge clk)
  b <= a;
```

endmodule

☑ Behavior:

Race condition ❌

Value of b depends on simulator execution order.

🔥 Why:

- `a = b` executes in Active
- `b <= a` RHS evaluated in Active
- Order of always blocks is undefined

Mixing `=` and `<=` across blocks → dangerous.

5 #0 Delay vs NBA Timing

```
module tb;

reg a=0;


initial begin

  a <= 9;

  #0 $display("After #0 a=%0d", a);

  #1;

end

endmodule
```

Output:

After #0 a=0

🔥 Why:

Timing order:

1. Active → schedule NBA

2. Inactive → `#0` executes → display prints old value

3. NBA → update happens

So value not updated yet during #0

🚀 Strong Interview Summary Line

If interviewer asks:

"What is the root cause of tricky timing bugs?"

Answer:

Most tricky timing bugs occur due to misunderstanding that RHS of non-blocking assignments is evaluated in the Active region, but updates occur later in the NBA region, causing unexpected overwrites or race conditions.

10 most confusing real interview questions

1.0 Blocking vs Non-Blocking Mix

```
module tb;

reg a=0,b=1;


initial begin

  a = b;

  b <= a;
```

```
  $display("a=%0d b=%0d",a,b);
  #0 $display("After #0 : a=%0d b=%0d",a,b);
end
endmodule
```

Output:

```
a=1 b=1
After #0 : a=1 b=1
```

2.0 NBA Scheduling Confusion

```
module tb;
reg a=0;

initial begin
  a <= 1;
  a <= 2;
  $display("a=%0d",a);
  #0 $display("After #0 a=%0d",a);
end
endmodule
```

Output:

a=0

After #0 a=2

(Last NBA wins)

3.0 #0 Delay Trick

```
module tb;

reg a=0;


initial begin
  a = 5;
  #0 a = 10;
  $display("a=%0d",a);
end
endmodule
```

Output:

a=5

(`$display` executes before #0 inactive region update)

4.0 Fork-Join Race

```
module tb;

reg a=0;
```

```
initial fork
  a = 5;
  a = 10;
join

initial #1 $display("a=%0d",a);
endmodule
```

Output:

a=5 OR a=10

(Race condition – Active region)

5.0 Continuous vs Procedural

```
module tb;
reg a=0;
wire b;

assign b = a;

initial begin
  a <= 1;
  $display("b=%0d",b);
```

```
  #0 $display("After #0 b=%0d",b);
end
endmodule
```

Output:

```
b=0
After #0 b=1
```

6.0 Multiple Always Blocks Race

```
module tb;
reg clk=0;
reg a=0;

always #5 clk = ~clk;

always @(posedge clk)
  a = 1;
always @(posedge clk)
  a = 2;

initial #20 $display("a=%0d",a);
endmodule
```

Output:

a=1 OR a=2

(Race in Active region)

7.0 Display vs Strobe

```
module tb;

reg a=0;


initial begin
  a <= 1;
  $display("display a=%0d",a);
  $strobe("strobe a=%0d",a);
end
endmodule
```

Output:

display a=0

strobe a=1

($strobe executes in Monitor region)

8.0 Function Timing Confusion

```
function int add;
```

```
  input int x,y;
  add = x+y;
endfunction

module tb;
initial
  $display("%0d",add(2,3));
endmodule
```

Output:

5

(Function executes in Active region immediately)

9.0 Event Scheduling Order

```
module tb;
reg a=0;

initial begin
  a <= 5;
  #0 $display("a=%0d",a);
end
endmodule
```

Output:

a=5

(NBA applied before next time slot)

10.0 Monitor vs Postponed

```
module tb;

reg a=0;


initial begin

  a <= 1;

  $monitor("monitor a=%0d",a);

end

endmodule
```

Output:

monitor a=1

($monitor runs in Monitor/Postponed region after NBA)

These 10 questions test:

- Active region behavior
- Inactive (#0) region
- NBA region
- Monitor/Postponed region
- Race conditions

- Fork-join ordering
- Blocking vs nonblocking
- Continuous assignment timing

SWAPPING OF TWO VARIABLES IN VERILOG/ SYSTEM VERILOG BY USING THREE METHOD: -

Main aim

a = 5;

b = 10;

We want:

a = 10

b = 5

Method 1: Using Temporary Variable (Safest – Works Everywhere)

```
module swap_with_temp;
reg [3:0] a, b, temp;

initial begin
    a = 4'd7;
    b = 4'd12;
    $display("Before swapping: a = %0d, b = %0d", a, b);

    temp = a;
    a = b;
    b = temp;
    $display("After swapping:  a = %0d, b = %0d", a, b);
```

```
end
endmodule
```

Output:

```
Before swapping: a = 7, b = 12
After swapping:  a = 12, b = 7
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Method 2: Using Non-Blocking Assignment (Best for Sequential Logic)

```
module swap_variables;
reg [3:0] a, b;
initial begin
      a = 5;
      b = 10;
   $display("Initial values: a = %b, b = %b", a, b);
   a <= b;
   b <= a;
   #1;
   $display("Swapped values: a = %b, b = %b", a, b);
   $finish();
   end
endmodule
```

Simulation Result

Output

Initial values: a = 5, b = 10

Swapped values: a = 10, b = 5

$finish at simulation time 1

- RHS evaluated first (Active region)
- Updates happen together in NBA region
- So values swap correctly

Method 3: Using XOR Swap (Hardware Style)

```
module tb;
reg [7:0] a = 5, b = 10;

initial begin
  a = a ^ b;
  b = a ^ b;
  a = a ^ b;

  $display("a=%0d b=%0d", a, b);
end
endmodule
```

Output:

a=10 b=5

XOR property:

x ^ y ^ y = x

Interview Best Practice Answer

If interviewer asks:

"How do you swap two registers in sequential logic?"

Best answer:

Use non-blocking assignments (`<=`) so RHS values are evaluated first and updated simultaneously in the NBA region.

Summary

Method	Use Case	Safe?
Temp variable	General	☑ Always safe
Non-blocking	Sequential logic	☑ Best choice
XOR method	Theoretical / combinational	Rarely used

Q1.0 What will be the output?

```
module tb;


byte a;
initial begin
    a = 8'b1110100;
    $display("value of a :%d %b", a,a);
end
endmodule
```

Simulation Output:

value of a : 116 01110100

V C S S i m u l a t i o n R e p o r t

Time: 0

Q2.0 what is the output of below code?

```
module tb;

int a=0;
int b=0;

initial begin
    a =1;
    a <= b;
    b <= a;
    $display("value of a:%d and value of b:%d",a,b);
    #1 $display("value of a:%d and value of b:%d",a,b);
end
endmodule
```

Simulation Output:

value of a: 1 and value of b: 0

value of a: 0 and value of b: 1

V C S S i m u l a t i o n R e p o r t

Time: 1

Q3.0 what is the output of below code?

```
module tb;

int a=0;
int b=0;
```

```
initial begin
    a =1;
    #1;
    a <= b;
    b <= a;
    $display("value of a:%0d and value of b:%0d",a,b);
    #1 $display("value of a:%0d and value of b:%0d",a,b);
end
endmodule
```

Simulation Output:

value of a:1 and value of b:0

value of a:0 and value of b:1

V C S S i m u l a t i o n R e p o r t

Time: 2

Q4.0 what is the output of below code?

```
module tb;

int a=0;
int b=0;

initial begin
    a =1;
    #1;
    a <= b;
    #1;
    b <= a;
    $display("value of a:%0d and value of b:%0d",a,b);
```

```
    #1 $display("value of a:%0d and value of b:%0d",a,b);
end
endmodule
```

Simulation Output:

value of a:0 and value of b:0

value of a:0 and value of b:0

V C S S i m u l a t i o n R e p o r t

Time: 3

Q5.0 what is the output of below code?

```
module tb;

int a=0;
int b=0;

initial begin
    a =1;
    a <= b;
    b <= a;
    a = 2;
    $display("value of a:%0d and value of b:%0d",a,b);
    #1 $display("value of a:%0d and value of b:%0d",a,b);
end
endmodule
```

Simulation Output:

value of a:2 and value of b:0

value of a:0 and value of b:1

V C S S i m u l a t i o n R e p o r t

Time: 1

Q6.0 what is the output of below code?

```
module tb;

int a=0;
int b=0;

initial begin
    a =1;
    a =2;
    a <= b;
    b = a;
    $display("value of a:%0d and value of b:%0d",a,b);
    #1 $display("value of a:%0d and value of b:%0d",a,b);
end
endmodule
```

Simulation Output:

value of a:2 and value of b:2

value of a:0 and value of b:2

V C S S i m u l a t i o n R e p o r t

Time: 1

Q7.0 what is the output of below code?

```
module tb;
```

```
int a=0;
int b=0;

initial begin
    a =1;
    a =2;
    a = b;
    b <= a;
    $display("value of a:%0d and value of b:%0d",a,b);
    #1 $display("value of a:%0d and value of b:%0d",a,b);
end
endmodule
```

Simulation Output:

value of a:0 and value of b:0

value of a:0 and value of b:0

V C S S i m u l a t i o n R e p o r t

Time: 1

Q8.0 Write the system Verilog assertions for a simple D-FF and verified the behaviour using simulation+ waveform analysis.

```
module d_ff (
    input  logic clk,
    input  logic rst,
    input  logic d,
    output logic q
```

```
);

always_ff @(posedge clk or posedge rst) begin
      if (rst)
         q <= 0;
      else
         q <= d;
end
endmodule

//testbench
module tb;
logic clk;
logic rst;
logic d;
logic q;


d_ff dut (.clk(clk),.rst(rst),.d(d),.q(q));

initial clk = 0;
always #5 clk = ~clk;

initial begin
      $monitor("Time=%0t for rst=%b,d=%d,q=%d",$time,rst,d,q);

      rst = 1; d = 0;
```

```
        #12;

        rst = 0; d = 1;
        #10;

        d = 0;
        #10;

        d = 1;
        #10;

        rst = 1;
        #10;

        rst = 0; d = 0;
        #10;

        $finish();
end

initial begin
          $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb);
end

// Reset Behavior Check
property p_reset;
```

```
        @(posedge clk)
        rst |-> (q == 0);
endproperty

assert property (p_reset)
else $error("Reset assertion failed");
// assert property (@(posedge clk) rst |-> (q == 0)) else $error("Reset assertion failed");
property p_dff_behavior;
        @(posedge clk)
      disable iff (rst || $past(rst))
      q == $past(d);
endproperty

assert property (p_dff_behavior)
else $error("DFF behavior assertion failed");

assert_no_x_z: assert property (
      @(posedge clk)
      !$isunknown(q)
      );

endmodule
```

Simulation results

Time=0 for rst=1,d=0,q=0

Time=12 for rst=0,d=1,q=0

Time=15 for rst=0,d=1,q=1

Time=22 for rst=0,d=0,q=1

Time=25 for rst=0,d=0,q=0

Time=32 for rst=0,d=1,q=0

Time=35 for rst=0,d=1,q=1

Time=42 for rst=1,d=1,q=0

Time=52 for rst=0,d=0,q=0

$finish called from file "tb.sv", line 35.

$finish at simulation time 62

V C S S i m u l a t i o n R e p o r t

Time: 62

[b23n_sarif_727@prac3 practice_sv]$ vcs -sverilog practice.sv tb.sv -kdb -lca -debug_access+all -full64

Q9.0 What is the output of below code?

```
module tb;
bit[7:0] a;

initial begin
    a = 'bxx1010101;
    $display("value of a :%0b", a);
    $display("value of a :%b", a);

    a = 'bxx111111111;
    $display("value of a :%0d", a);
    $display("value of a :%d", a);
end
endmodule
```

Simulation results

value of a :1010101

value of a :01010101

value of a :255

value of a :255

V C S S i m u l a t i o n R e p o r t

Time: 0

Because bit converts x and z to **0 automatically**.

Q10.0 What is the output of below code?

```
module tb;
bit[7:0] a;

initial begin
    a = 'bxz1010101;
    $display("value of a :%0b", a);
    $display("value of a :%b", a);

    a = 'bxz111111111;
    $display("value of a :%0d", a);
    $display("value of a :%d", a);
end
endmodule
```

Simulation results

value of a :1010101

value of a :01010101

value of a :255

value of a :255

V C S S i m u l a t i o n R e p o r t

Time: 0

Q11.0 What is the output of below code?

```
module tb;
int a;

initial begin
    a = 'hxz1010101;
    $display("value of a :%0b", a);
    $display("value of a :%b", a);

    a = 'hxz111111111;
    $display("value of a :%0d", a);
    $display("value of a :%d", a);
end
endmodule
```

Simulation results

value of a :10000000100000000100000001

value of a :00000001000000010000000100000001

value of a :286331153

value of a : 286331153

V C S S i m u l a t i o n R e p o r t

Time: 0

- int is **32-bit**
- int is **2-state** (only 0 and 1)
- Any x or z becomes **0**

Q12.0 What is the output of below code?

```
module tb;
int a;

initial begin
    a = 'hxz1010101;
    $display("value of a :%0h", a);
    $display("value of a :%h", a);

    a = 'hxz111111111;
    $display("value of a :%0b", a);
    $display("value of a :%b", a);
end
endmodule
```

Simulation results

value of a :1010101

value of a :01010101

value of a :10001000100010001000100010001

value of a :00010001000100010001000100010001

V C S Simulation Report

Time: 0

Q13.0 What is the output of below code?

```
module tb;
int a;
```

```
initial begin
    a = 'hxz1010101;
    $display("value of a :%0h", a);
    $display("value of a :%h", a);

    a = 'hxzxxx11;
    $display("value of a :%0b", a);
    $display("value of a :%b", a);
end
endmodule
```

Simulation results

value of a :1010101

value of a :01010101

value of a :10001

value of a :00000000000000000000000000010001

V C S S i m u l a t i o n R e p o r t

Time: 0

Q14.0 What is the output of below code?

```
module tb;
bit [2:0] arr[3:0];
initial begin
    arr[0] = 3'b001;
    arr[1] = 3'b010;
    arr[2] = 3'b011;
    arr[3] = 3'b100;
    $display("value of array : %b", {arr[3], arr[2], arr[1], arr[0]});
```

```
end
endmodule
```

Simulation results

value of array : 100011010001

V C S S i m u l a t i o n R e p o r t

Time: 0

Q15.0 What is the output of below code?

```
module tb;
bit [2:0] [3:0] arr;
initial begin
    arr[0] = 3'b001;
    arr[1] = 3'b010;
    arr[2] = 3'b011;
    arr[3] = 3'b100;
    $display("value of array : %b", {arr[3], arr[2], arr[1], arr[0]});
end
endmodule
```

Simulation results

value of array : 0000001100100001

V C S S i m u l a t i o n R e p o r t

Time: 0

Q16.0 What is the output of below code?

```
module tb;
bit [7:0] a;
byte b;
```

```
initial begin
    a = 8'b11011101;
    b = 8'b11011101;
    $display("value of a:%d  %b", a,a);
    $display("value of b:%d  %b", b,b);

end
endmodule
```

Simulation results

value of a:221 11011101

value of b: -35 11011101

V C S S i m u l a t i o n R e p o r t

Time: 0

Q17.0 What is the output of below code?

```
module tb;
bit [7:0] a;
byte b;
initial begin
    a = -7;
    b = -7;
    $display("value of a :%d  %b", a,a);
    $display("value of b :%d  %b", b,b);
    a = 127;
    b = 127;
    $display("value of a :%d  %b", a,a);
```

```
        $display("value of b :%d  %b", b,b);
        a = 130;
        b = 130;
        $display("value of a :%d  %b", a,a);
        $display("value of b :%d  %b", b,b);

end
endmodule
```

Simulation results

```
value of a :249  11111001
value of b :        -7  11111001
value of a :127  01111111
value of b :       127  01111111
value of a :130  10000010
value of b :      -126  10000010
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Q18.0 What is the output of below code?

```
module tb;
int associative[string];

initial begin
        string a[] = '{"sarif","ahmad"};
        int b[] = '{ 1 , 2};

        foreach (a[i])
                associative[a[i]] = b[i];
```

```
    foreach (associative[name])
        $display(" %s => %0d", name,associative[name]);

end
endmodule
```

Simulation results

ahmad => 2

sarif => 1

V C S S i m u l a t i o n R e p o r t

Time: 0

Q19.0 What is the output of below code?

```
module tb;
bit [7:0] a = 8'b01000111;
bit [7:0] b = 8'b11100110;

initial begin
    {a[7] , b[7]} = { b[7] , a[7]};
    {a[0] , b[0]} = { b[0] , a[0]};

    $display("after swapping of new data of a & b");
    $display("value of a = %b", a);
    $display("value of b = %b", b);
end
endmodule
```

Simulation results

after swapping of new data of a & b

value of a = 11000110

value of b = 01100111

V C S S i m u l a t i o n R e p o r t

Time: 0

Q20.0 What is the output of below code?

```
module tb;
int a[$];
int b[$];
int c[];
int d;
bit[7:0] e;

initial begin
    a = {1,2,3,4,5};
    $display("value of a : %p", a);
    a.push_front(5);
    $display("value of a : %p", a);
    a.pop_back();
    $display("value of a : %p", a);

    b = {1,3,a};
    $display("value of b : %p", b);
    b = {a,1};
    $display("value of b : %p", b);
end
```

```
endmodule
```

Simulation results

value of a : '{1, 2, 3, 4, 5}

value of a : '{5, 1, 2, 3, 4, 5}

value of a : '{5, 1, 2, 3, 4}

value of b : '{1, 3, 5, 1, 2, 3, 4}

value of b : '{5, 1, 2, 3, 4, 1}

V C S S i m u l a t i o n R e p o r t

Time: 0

Q21.0 What is the output of below code?

```
//
module tb;
int q[$]='{10,50,30,5,70,25};
int max_val, min_val;

initial begin
        max_val = q[0];
        min_val = q[0];

        q.sort();
        $display("value of q:%p",q);
        q.rsort();
        $display("value of q:%p",q);

        $display("value of q:%p",q[1]);
      $display("value of q:%p",q[2]);
```

```
        foreach (q[i]) begin
                if (q[i] > max_val )
                        max_val = q[i];
                if (q[i] < min_val )
                        min_val = q[i];
        end
        $display("Queue : %p", q);
        $display("maximum = %0d", max_val);
        $display("minimum =%0d", min_val);

end
endmodule
```

Simulation results

```
value of q:'{5, 10, 25, 30, 50, 70}
value of q:'{70, 50, 30, 25, 10, 5}
value of q:        50
value of q:        30
Queue : '{70, 50, 30, 25, 10, 5}
maximum = 70
minimum =5
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Q22.0 What is the output of below code?

How to find the number is even or odd without any mathematical operator

```
module tb;
bit[3:0] a[] = {0,1,2,3,4,5,6,7,8};

initial begin

    foreach(a[i]) begin
        if (a[i][0] == 1'b0)
            $display("value of number: %0d\t   | %d\t  | %0b\t | even", i , a[i], a[i]);
        else
            $display("value of number: %0d\t   | %d\t  | %0b\t | odd", i , a[i], a[i]);
    end
    $display("Index\tvalue");
    $display("0\t10");
    $display("1\t20");
end
endmodule
```

Simulation results

```
value of number: 0   |  0    | 0     | even
value of number: 1   |  1    | 1     | odd
value of number: 2   |  2    | 10    | even
value of number: 3   |  3    | 11    | odd
value of number: 4   |  4    | 100  | even
value of number: 5   |  5    | 101  | odd
value of number: 6   |  6    | 110  | even
value of number: 7   |  7    | 111  | odd
value of number: 8   |  8    | 1000         | even
Index value
```

0 10

1 20

V C S S i m u l a t i o n R e p o r t

Time: 0

Q23.0 What is the output of below code?

```
module tb;
typedef union {
    int a;
    byte b;
    bit [3:0]c ;} student;

    student s1;
    initial begin
        s1.a = 50;
        $display("value of a: %b", s1.a);
        s1.b = 20;
        $display(" a: %b & b :%b", s1.a, s1.b);
        s1.c = 10;
        $display(" a: %b & b :%b & c: %b", s1.a, s1.b, s1.c);
end
endmodule
```

Simulation results

value of a: 00000000000000000000000000110010

a: 00000000000000000000000000010100 & b :00010100

a: 00000000000000000000000000001010 & b :00001010 & c: 1010

V C S S i m u l a t i o n R e p o r t

Time: 0

Q24.0 Implement a 4-bit counter in system verilog and verified it using functional coverage. the goal was to ensure all 16 counters states (0-15) were covered during simulation. (Design a 4-bit counter DUT, Built a system verilog testbench, created a coverage class with manual sampling cg.sample() and finally we can achive 100% functional coverage.)

```
//
module counter_4bit (
    input  logic clk,
    input  logic rst,
    output logic [3:0] count
);
always_ff @(posedge clk or posedge rst) begin
    if (rst)
        count <= 4'd0;
    else
        count <= count + 1;
end
endmodule
```

//Testbench with functional verification:

```
//testbench
module tb;
logic clk;
logic rst;
logic [3:0] count;


counter_4bit dut (.clk(clk),.rst(rst),.count(count));
```

```
always #5 clk = ~clk;

class counter_cov;
        covergroup cg;
                coverpoint count {
                        bins all_states[] = {[0:15]};
        }
        endgroup

        function new();
                cg = new();
        endfunction
endclass

counter_cov cov;

initial begin
        clk = 0;
       rst = 1;

        cov = new();
        #10 rst = 0;

        repeat (20) begin
                @(posedge clk);
                cov.cg.sample();
```

```
        end


        $display("Coverage = %0.2f %%", cov.cg.get_coverage());
        $finish;
end
endmodule
```

Simulation results

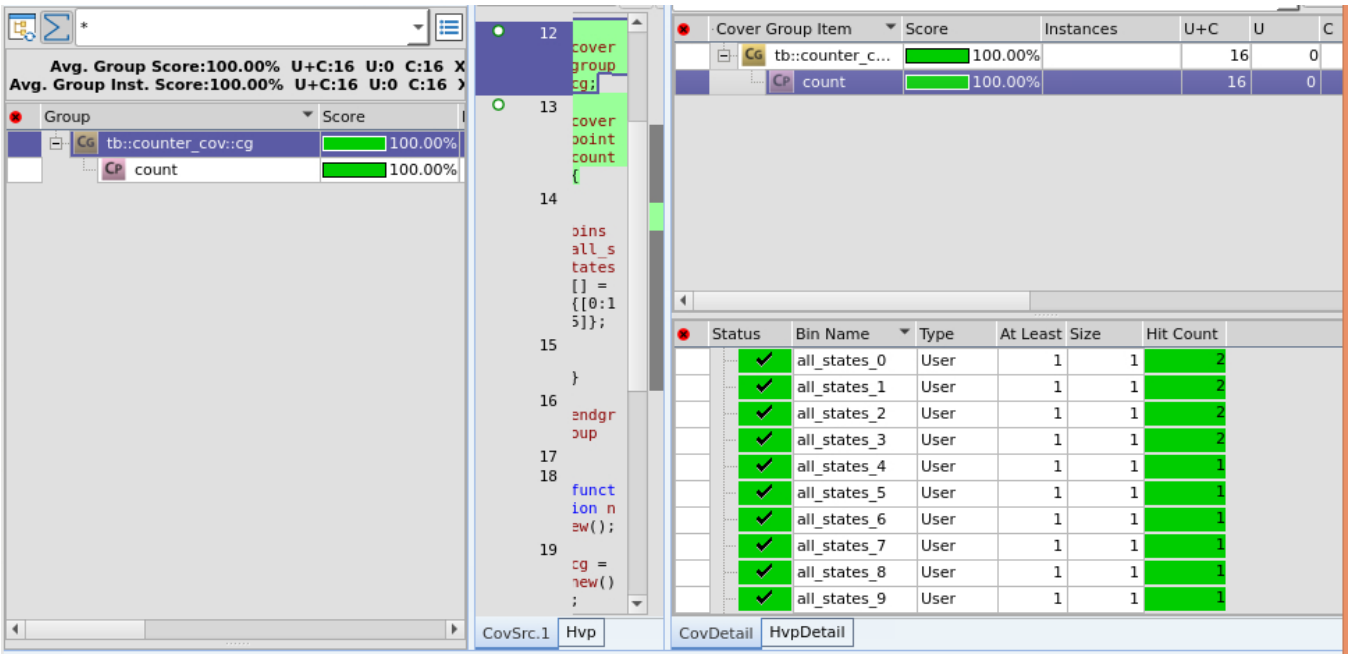
Coverage = 100.00 %

$finish called from file "tb.sv", line 38.

$finish at simulation time 205

V C S S i m u l a t i o n R e p o r t

Time: 205



Q25.0 What is the output of below code?

```
class generator;
rand byte a[14];
rand byte b[$];
```

```
constraint c1 { b.size() == a.size();}
constraint c2 {
    foreach (a[i])
        b[i] == (a[i]%2 == 0 ? 0:1);
}
endclass

module tb;
generator pkt;
initial begin
    repeat(5) begin
        pkt = new();
        pkt.randomize();
        $display("random value of a:%p",pkt.a);
        $display("random value of b:%p",pkt.b);
    end
end
endmodule
```

Simulation results

```
random value of a:'{-40, -123, -126, -107, -81, -119, -124, -125, -62, -18, 30, -43, -107, 64}
random value of b:'{0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0}
random value of a:'{67, -2, 102, -21, -31, 79, -6, -83, -77, -92, -99, 53, -2, -49}
random value of b:'{1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1}
random value of a:'{27, -61, -32, 84, -88, -86, 28, -38, -71, -42, 75, 15, 65, 112}
random value of b:'{1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0}
random value of a:'{17, 17, -74, 65, 73, -23, 5, 6, -8, 69, 10, 119, 112, -33}
```

```
random value of b:'{1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1}
random value of a:'{9, -6, -65, 28, 6, 90, -15, -49, 89, 118, 61, -25, -91, 22}
random value of b:'{1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0}
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Q26.0 Sum of all elements must be less than 10 & If element divisible by 4 → it must be 0

```
class generator;
rand byte a[14];

constraint c1 { a.sum() < 10; }
constraint c2 {
    foreach (a[i])
        (a[i]%4 == 0) -> (a[i]==0);
}
endclass

module tb;
generator pkt;
initial begin
    repeat(5) begin
        pkt = new();
        pkt.randomize();
        $display("random value of a:%p",pkt.a);
    end
end
endmodule
```

Simulation results

random value of a:'{70, -3, -117, 106, 10, -1, -55, 98, 126, 50, 14, 49, -105, -75}

random value of a:'{127, -87, 98, -45, 1, -42, -69, -39, 53, -55, -51, 69, -109, -103}

random value of a:'{23, -125, 3, -17, -105, 71, 54, -75, 73, -65, 13, -102, 79, -91}

random value of a:'{118, 73, -7, -58, 37, 18, -85, -9, 26, 125, 73, 45, -118, -21}

random value of a:'{94, 90, 117, -67, -83, -49, -33, -106, -118, 75, -35, -85, -5, -91}

V C S S i m u l a t i o n R e p o r t

Time: 0

Q27.0 What is the output of below code?

```
module tb;
int a= 10;
reg b;
int c;

initial begin
    a =1;
    b <=a;
    #1 $display("value of a:%d  b:%d",a,b);

    c <= b;
    #1 $display("value of a:%d  b:%d  c:%d", a, b,c);
    $display("value of a:%d  b:%d  c:%d", a, b,c);
end
endmodule
```

Simulation results

value of a: 1 b:1

value of a: 1 b:1 c: 1

value of a: 1 b:1 c: 1

V C S S i m u l a t i o n R e p o r t

Time: 2

Q28.0 What is the output of below code?

```
module tb;
int a = 10;
reg b = 5;
int c;
wire d;

initial begin
    a =1;
    b <=a;
    a = 2;
    $display("value of a:%d and b:%d",a,b);

end
endmodule
```

Simulation results

value of a: 2 and b:1

V C S S i m u l a t i o n R e p o r t

Time: 0

Q29.0 What is the output of below code?

```
module tb;
int a = 10;
reg b = 5;
int c;
wire d;

initial begin
    a =1;
    b <=a;
    a = 2;
    #1 $display("value of a:%d and b:%d",a,b);

end
endmodule
```

Simulation results

value of a: 2 and b:1

V C S S i m u l a t i o n R e p o r t

Time: 1

Q30.0 What is the output of below code?

```
module tb;
int a = 10;
int b = 5;
int c;
wire d;
```

```
initial begin
    a =1;
    b <=a;
    a = 2;
    $display("value of a:%d and b:%d",a,b);
end
endmodule
```

Simulation results

value of a: 2 and b: 5

V C S S i m u l a t i o n R e p o r t

Time: 0

Q31.0 What is the output of below code?

```
module tb;
int a = 10;
int b = 5;
int c;
wire d;

initial begin
    a =1;
    a =2;
    b <=a;
    #2 $display("value of a:%d and b:%d",a,b);

end
endmodule
```

Simulation results

value of a: 2 and b: 2

V C S S i m u l a t i o n R e p o r t

Time: 2

Q32.0 What is the output of below code?

```
module child (input a,b,output sum);
assign sum = a^b;
endmodule

module tb;
reg a,b;
wire sum;

child dut(.a(a),.b(b),.sum(sum));

initial begin
    a = 1;
    b = 0;
    $display("value of sum: %b",sum);
end
endmodule
```

Simulation results

value of sum: 1

V C S S i m u l a t i o n R e p o r t

Time: 0

Q33.0 What is the output of below code?

```
module child (input a,b,en,output sum);
assign sum = en ? (a^b) : 0;
endmodule

module tb;
reg a,b,en;
wire sum;

child dut(.a(a),.b(b),.en(en),.sum(sum));

initial begin
    a = 1;
    b = 0;
    en = 1;
    $display("value of sum: %b",sum);
end
endmodule
```

Simulation results

value of sum: x

V C S S i m u l a t i o n R e p o r t

Time: 0

⏱ Timeline at time 0

- **a=1, b=0, en=1 assigned**
- **$display runs immediately**
- **BUT sum has not updated yet**
- **So sum is still unknown → x**

Q34.0 What is the output of below code?

```
module child (my_if intf);
assign intf.sum = intf.a ^ intf.b;
endmodule

interface my_if;
        logic a;
        logic b;
        logic sum;
endinterface

module tb;
my_if intf();
child dut(intf);

initial begin
        intf.a = 1;
        intf.b = 0;
        $display("value of sum: %b",intf.sum);
end
endmodule
```

Simulation results

value of sum: 1

V C S S i m u l a t i o n R e p o r t

Time: 0

Q35.0 What is the output of below code?

```
module child (my_if intf);
assign intf.sum = intf.en ? intf.a ^ intf.b : 0;
endmodule

interface my_if;
    logic a;
    logic b;
    logic en;
    logic sum;
endinterface

module tb;
my_if intf();
child dut(intf);

initial begin
    intf.a = 1;
    intf.b = 0;
    intf.en = 1;
    $display("value of sum: %b",intf.sum);
    #1 $display("value of sum: %b",intf.sum);

end
endmodule
```

Simulation results

value of sum: x

value of sum: 1

V C S S i m u l a t i o n R e p o r t

Time: 1

OOPS: -

Adder Design Code Using Interface

```
// adder circuit
module full_adder (
        input  logic a,
      input  logic b,
      input  logic cin,
      output logic sum,
      output logic cout
);

assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (a & cin);

endmodule

interface fa_if;
        logic a;
      logic b;
      logic cin;
      logic sum;
```

```
    logic cout;
endinterface
```

Testbench

```
//
module tb;
fa_if intf();
full_adder dut (.a(intf.a),.b(intf.b),.cin(intf.cin),.sum (intf.sum),.cout(intf.cout));

initial begin
        intf.a   = 0;
        intf.b   = 0;
        intf.cin = 0;
        #1;
        $display("a=%0b b=%0b cin=%0b sum=%0b cout=%0b",intf.a, intf.b, intf.cin,intf.sum, intf.cout);

        intf.a   = 0;
        intf.b   = 1;
        intf.cin = 1;
        #1;
        $display("a=%0b b=%0b cin=%0b sum=%0b cout=%0b",intf.a, intf.b, intf.cin,intf.sum, intf.cout);

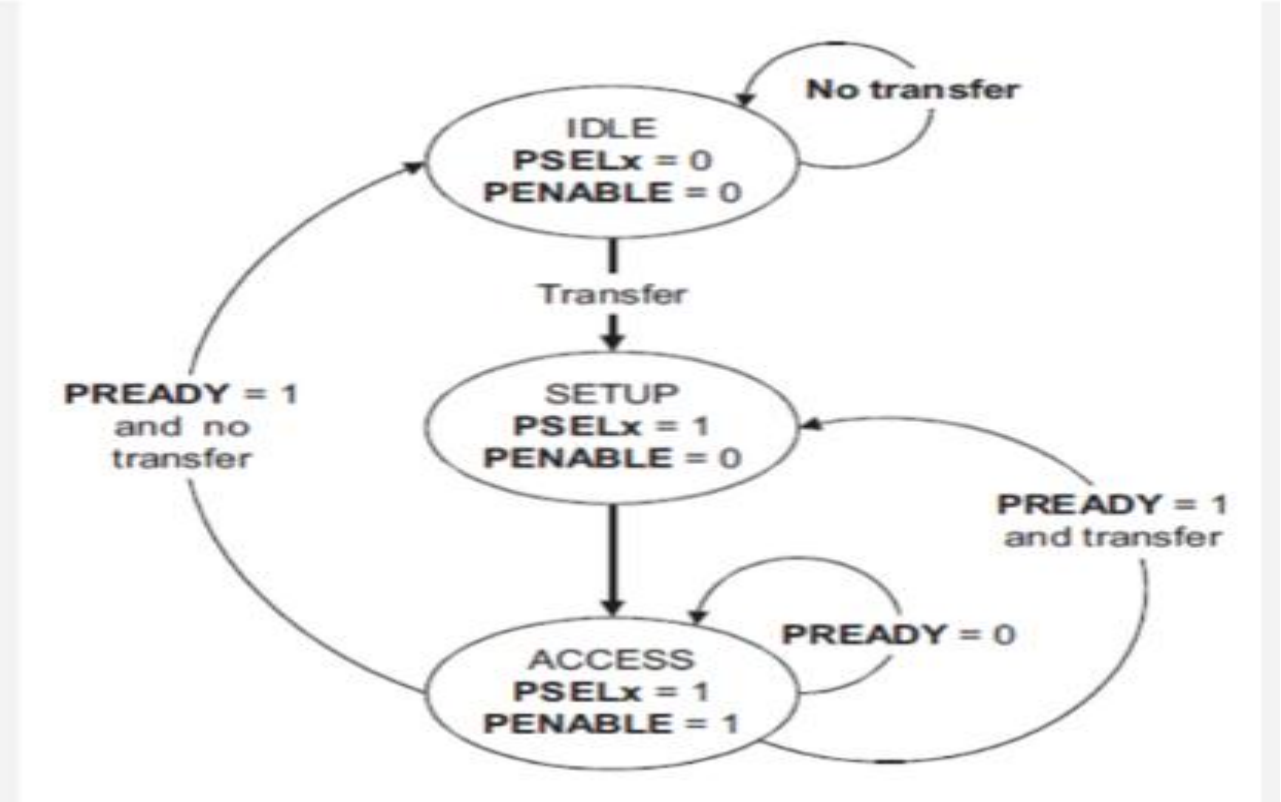
        intf.a   = 1;
        intf.b   = 0;
        intf.cin = 1;
        #1;
```

```
        $display("a=%0b b=%0b cin=%0b sum=%0b cout=%0b",intf.a, intf.b, intf.cin,intf.sum, intf.cout);


        intf.a   = 1;
        intf.b   = 1;
        intf.cin = 1;
        #1;
        $display("a=%0b b=%0b cin=%0b sum=%0b cout=%0b",intf.a, intf.b, intf.cin,intf.sum, intf.cout);
        $finish;
end
endmodule
```

Simulation results

```
Compiler version W-2024.09-1_Full64; Runtime version W-2024.09-1_Full64;  Jul 22 20:47 2026
a=0 b=0 cin=0 sum=0 cout=0
a=0 b=1 cin=1 sum=0 cout=1
a=1 b=0 cin=1 sum=0 cout=1
a=1 b=1 cin=1 sum=1 cout=1
$finish called from file "adder_tb.sv", line 31.
$finish at simulation time                   4
         V C S   S i m u l a t i o n   R e p o r t
Time: 4
CPU Time:      0.080 seconds;       Data structure size:   0.0Mb
Wed Jul 22 20:47:50 2026
```



APB Protocol

```
//
module apb_slave (
    input  logic        pclk,

    input  logic        rst_n,
    input  logic [31:0]  paddr,
    input  logic        psel,

    input  logic        penable,
    input  logic        pwrite,
    input  logic [31:0]  pwdata,

    output logic [31:0]  prdata,
    output logic         pready
```

```
);

  // Internal memory and states
  logic [31:0] mem [0:255];
  logic [1:0]  apb_st;

  // State encoding
  localparam logic [1:0]
    SETUP    = 2'b00,
    W_ENABLE = 2'b01,
    R_ENABLE = 2'b10;

   // Sequential logic
  always_ff @(posedge pclk or negedge rst_n) begin
    if (!rst_n) begin
      apb_st <= SETUP;
      prdata <= 0;
      pready <= 1;
      for (int i = 0; i < 256; i++)
        mem[i] <= i;  // initialize memory for demo
    end
    else begin
      case (apb_st)

        //================================
        // SETUP phase: prepare to read or write
        //================================
```

```
 SETUP: begin
   prdata <= 0;
   if (psel && !penable) begin
     if (pwrite)
       apb_st <= W_ENABLE;
     else begin
       apb_st <= R_ENABLE;
       prdata <= mem[paddr]; // return stored data
     end
   end
 end
//================================
// WRITE ENABLE phase
//================================

 W_ENABLE: begin
   if (psel && penable && pwrite) begin
     mem[paddr] <= pwdata; // perform write
   end
   apb_st <= SETUP; // return to idle
 end

 //================================
 // READ ENABLE phase
 //================================
 R_ENABLE: begin
   apb_st <= SETUP; // read already done in previous state
```

```
        end


      endcase
    end
  end

endmodule
module tb_apb;

  logic        pclk;
  logic        rst_n;
  logic [31:0]  paddr;
  logic        psel;
  logic        penable;
  logic        pwrite;
  logic [31:0]  pwdata;
  logic [31:0]  prdata;
  logic        pready;


  // Instantiate DUT
  apb_slave dut
(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pwrite),.pwd
ata(pwdata),.prdata(prdata),.pready(pready) );
initial begin
    pclk = 0;
    forever #5 pclk = ~pclk;
  end
initial begin
```

```
  rst_n = 0;
  #20 rst_n = 1;
 end

// APB write task
 task automatic apb_write(input [31:0] addr, input [31:0] data);
  @(posedge pclk);
  paddr   = addr;
  pwrite  = 1;
  pwdata  = data;
  psel    = 1;
  penable = 0;

  @(posedge pclk);
  penable = 1;
  @(posedge pclk);
  psel    = 0;
  penable = 0;
  pwrite  = 0;

  $display("[%0t] WRITE: addr=%0d data=0x%0h", $time, addr, data);
 endtask

task automatic apb_read(input [31:0] addr, output [31:0] data);
  @(posedge pclk);
  paddr   = addr;
  pwrite  = 0;
```

```
    psel    = 1;
    penable = 0;

    @(posedge pclk);
    penable = 1;

    @(posedge pclk);
    data = prdata;
    psel    = 0;
    penable = 0;
    $display("[%0t] READ : addr=%0d data=0x%0h", $time, addr, data);
  endtask

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars("tb_apb",0);
end

 // Stimulus
  logic [31:0] rdata;
  initial begin
    psel = 0; penable = 0; pwrite = 0; paddr = 0; pwdata = 0;
    wait(rst_n == 1);
    #10;

    apb_write(32'd10, 32'hAABB_CCDD);
    #10;
```

```
    apb_read(32'd10, rdata);

    if (rdata == 32'hAABB_CCDD)
      $display("[%0t]  READBACK SUCCESS", $time);
    else
      $display("[%0t]  READBACK FAIL: got %h", $time, rdata);

    #20;
    apb_read(32'd5, rdata);
    $display("[%0t] READ addr=5 got=%0d", $time, rdata);

    #50 $finish;
  end
endmodule
```

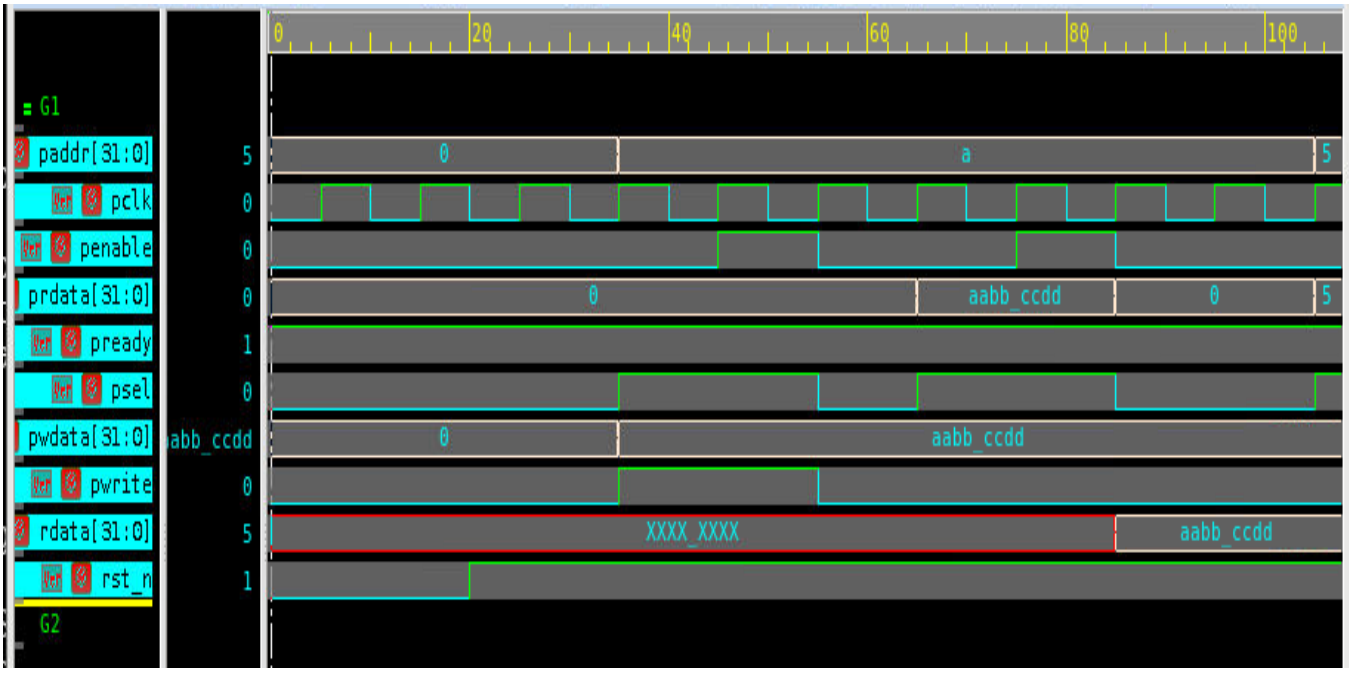
Simulation Command

```
[b23n_sarif_727@prac3 apb]$ vcs -sverilog apb.v -kdb -lca -debug_access+all -full64
```

Simulation results

```
[55] WRITE: addr=10 data=0xaabbccdd
[85] READ : addr=10 data=0xaabbccdd
[85]  READBACK SUCCESS
[125] READ : addr=5 data=0x5
[125] READ addr=5 got=5
$finish called from file "apb.v", line 159.
$finish at simulation time                 175
           V C S   S i m u l a t i o n   R e p o r t
Time: 175
CPU Time:      0.090 seconds;       Data structure size:   0.0Mb
```

Wed Jul 22 20:55:55 2026



APB Slave with PLVERR and wait-state N=2

With Wait-state

```
//
module apb_slave #(parameter WAIT_CYCLES = 2)(
        input  logic         pclk,
      input  logic         rst_n,
      input  logic [31:0]  paddr,
      input  logic         psel,
      input  logic         penable,
      input  logic         pwrite,
      input  logic [31:0]  pwdata,
        output logic [31:0]  prdata,
        output logic         pready,
        output logic         pslverr
);
```

```
logic [31:0] mem [0:255];
logic [1:0] apb_st;
localparam SETUP    = 2'b00, W_ENABLE = 2'b01,R_ENABLE = 2'b10;
integer wait_count;

always_ff @(posedge pclk or negedge rst_n) begin
        if (!rst_n) begin
                apb_st     <= SETUP;
                prdata     <= 32'd0;
            pready     <= 1'b1;
            pslverr    <= 1'b0;
            wait_count <= 0;

                for (int i = 0; i < 256; i++)
                        mem[i] <= i;
        end
        else begin
                case (apb_st)

                        SETUP: begin
                                prdata  <= 32'd0;
                         pslverr <= 1'b0;
                         pready  <= 1'b0;

                                if (psel && !penable) begin
                                        if (paddr > 255) begin
```

```
                pslverr <= 1'b1;
                pready  <= 1'b1;
                if (pwrite)
                        apb_st <= W_ENABLE;
                else
                        apb_st <= R_ENABLE;
        end
        else begin
                wait_count <= WAIT_CYCLES;
                if (pwrite)
                        apb_st <= W_ENABLE;
                else
                        apb_st <= R_ENABLE;
        end
    end
end

W_ENABLE: begin
    if (psel && penable) begin
        if (wait_count > 0) begin
                pready     <= 1'b0;
                wait_count <= wait_count - 1;
        end
        else begin
                pready <= 1'b1;
                if (paddr <= 255)
                        mem[paddr] <= pwdata;
```

```
                        apb_st <= SETUP;
                    end
                end
            end
            R_ENABLE: begin
                if (psel && penable) begin
                    if (wait_count > 0) begin
                        pready     <= 1'b0;
                        wait_count <= wait_count - 1;
                    end
                    else begin
                        pready <= 1'b1;
                        if (paddr <= 255)
                            prdata <= mem[paddr];
                        apb_st <= SETUP;
                    end
                end
            end
            default: apb_st <= SETUP;
        endcase
    end
end
endmodule
```

Testbench

```
//
module tb_apb;
logic       pclk;
```

```
logic       rst_n;
logic [31:0] paddr;
logic       psel;
logic       penable;
logic       pwrite;
logic [31:0] pwdata;
logic [31:0] prdata;
logic       pready;
logic       pslverr;
logic [31:0] rdata;


apb_slave #(.WAIT_CYCLES(2))
dut(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pwrite),.p
wdata(pwdata),.prdata(prdata),.pready(pready),.pslverr(pslverr));


initial begin
        pclk = 0;
        forever #5 pclk = ~pclk;
end


initial begin
        rst_n = 0;
        #20;
        rst_n = 1;
end


task automatic apb_write(input [31:0] addr,input [31:0] data);
begin
```

```
        @(posedge pclk);
        paddr   <= addr;
      pwdata  <= data;
      pwrite  <= 1'b1;
      psel    <= 1'b1;
      penable <= 1'b0;
        $display("[%0t] WRITE SETUP phase : addr=%0d data=%h",$time, addr, data);

        @(posedge pclk);
        penable <= 1'b1;
        $display("[%0t] WRITE ACCESS phase: PREADY=%b",$time, pready);

while (!pready)
begin
        @(posedge pclk);
        $display("[%0t] WRITE WAIT-state : PREADY=%b",$time, pready);
end

@(posedge pclk);
$display("[%0t] WRITE DONE-state : addr=%0d data=%h PSLVERR=%b",$time, addr,
data, pslverr);
psel    <= 1'b0;
penable <= 1'b0;
pwrite  <= 1'b0;
end
endtask

task automatic apb_read(input  [31:0] addr,output [31:0] data);
```

```
begin
        @(posedge pclk);
        paddr   <= addr;
      pwrite  <= 1'b0;
      psel    <= 1'b1;
      penable <= 1'b0;
        $display("[%0t] READ SETUP  : addr=%0d",$time, addr);

        @(posedge pclk);
        penable <= 1'b1;
        $display("[%0t] READ ACCESS : PREADY=%b", $time, pready);

while (!pready)
begin
        @(posedge pclk);
        $display("[%0t] READ WAIT   : PREADY=%b",$time, pready);
end

@(posedge pclk);
data = prdata;
$display("[%0t] READ DONE   : addr=%0d data=%h PSLVERR=%b",$time, addr, data,
pslverr);
psel    <= 1'b0;
penable <= 1'b0;
end
endtask

initial begin
```

```
    psel    = 0;
  penable = 0;
  pwrite  = 0;
  paddr   = 0;
  pwdata  = 0;

    wait(rst_n == 1);
    #10;
    apb_write(32'd10, 32'hAABB_CCDD);
    apb_read(32'd10, rdata);

    if (rdata == 32'hAABB_CCDD) begin
      $display("[%0t] READBACK SUCCESS", $time);
  end
  else begin
      $display("[%0t] READBACK FAIL: got=%h", $time, rdata);
  end

    apb_write(32'd300, 32'h1234_5678);
    if (pslverr)
          $display("[%0t] INVALID ADDRESS: PSLVERR=1", $time);
    else
          $display("[%0t] ERROR: PSLVERR=0", $time);
    #20;
    $finish;
end
initial begin
```

```
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb_apb);
end
endmodule
```

Simulation Command

[b23n_sarif_727@prac3 apb_protocol]$ vcs -sverilog apb_design.sv apb_tb.sv -kdb -lca -debug_access+all -full64 && ./simv

[b23n_sarif_727@prac3 apb_protocol]$ ./simv -verdi

Simulation results

[35] WRITE SETUP phase : addr=10 data=aabbccdd

[45] WRITE ACCESS phase: PREADY=0

[55] WRITE WAIT-state : PREADY=0

[65] WRITE WAIT-state : PREADY=0

[75] WRITE WAIT-state : PREADY=0

[85] WRITE WAIT-state : PREADY=1

[95] WRITE DONE-state : addr=10 data=aabbccdd PSLVERR=0

[105] READ SETUP : addr=10

[115] READ ACCESS : PREADY=0

[125] READ WAIT   : PREADY=0

[135] READ WAIT   : PREADY=0

[145] READ WAIT   : PREADY=0

[155] READ WAIT   : PREADY=1

[165] READ DONE   : addr=10 data=00000000 PSLVERR=0

[165] READBACK FAIL: got=00000000

[175] WRITE SETUP phase : addr=300 data=12345678

[185] WRITE ACCESS phase: PREADY=0

[195] WRITE WAIT-state : PREADY=1

[205] WRITE DONE-state : addr=300 data=12345678 PSLVERR=1

[205] INVALID ADDRESS: PSLVERR=1

$finish called from file "apb_tb.sv", line 108.

$finish at simulation time 225

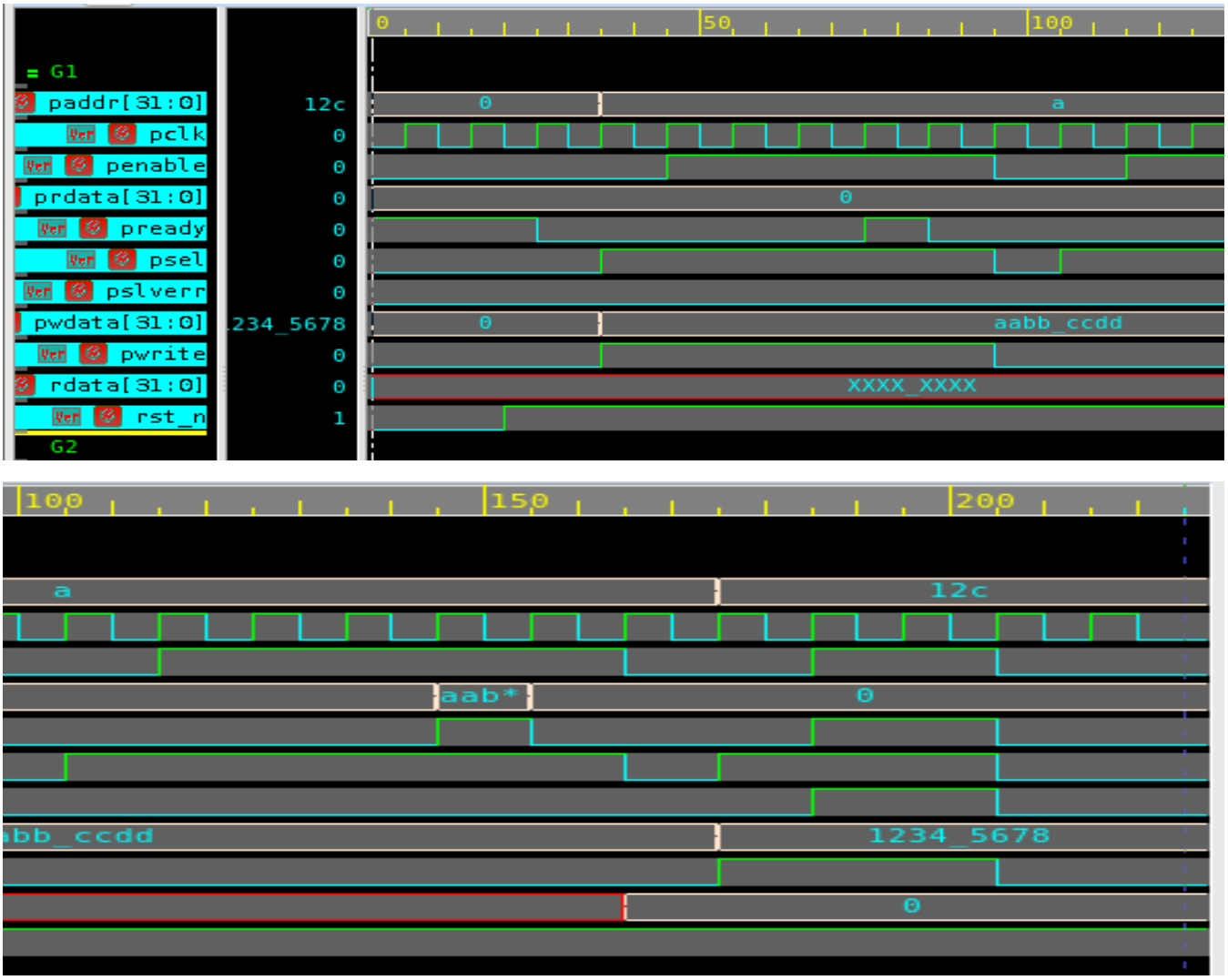
V C S S i m u l a t i o n R e p o r t

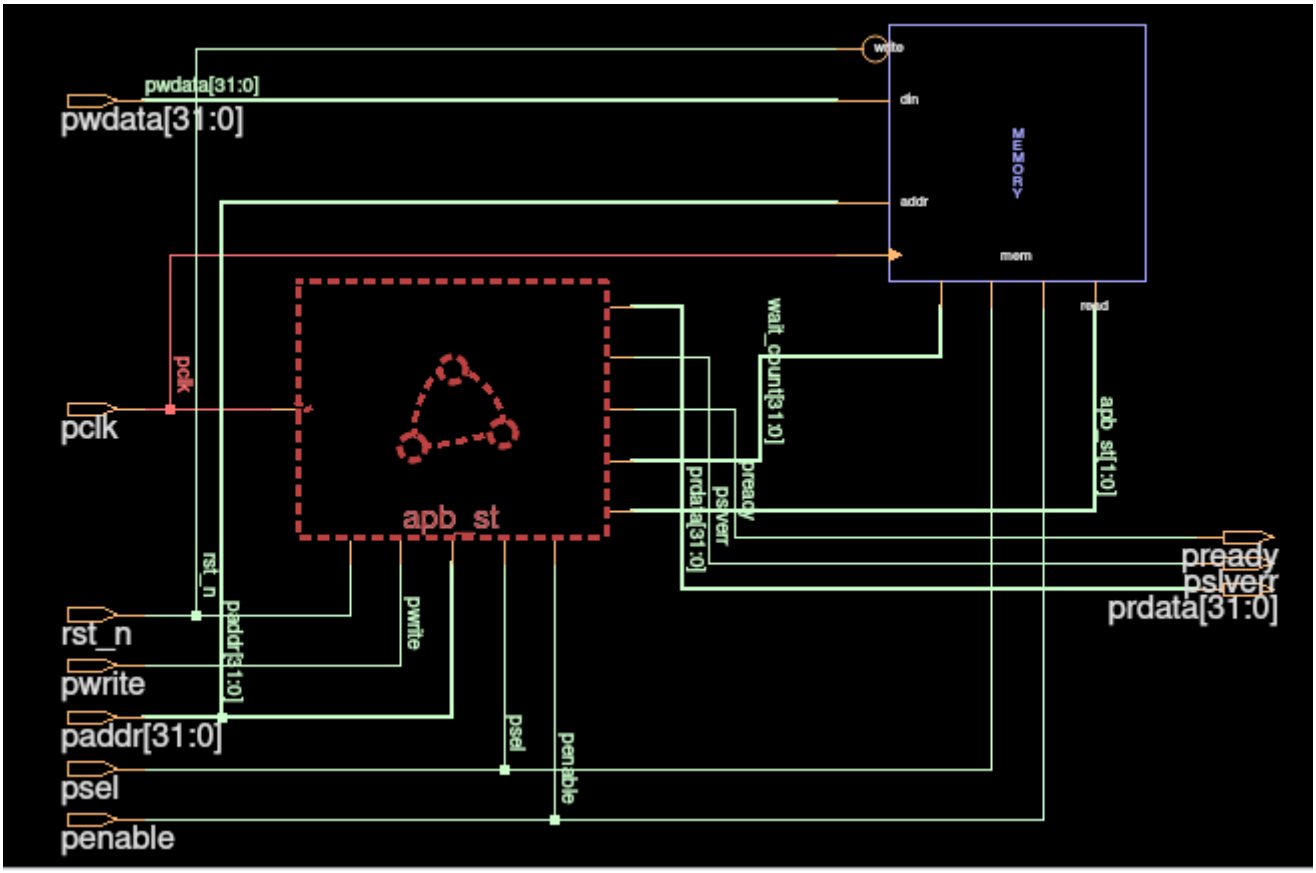
Time: 225

CPU Time: 0.100 seconds; Data structure size: 0.0Mb

Mon Aug 17 23:46:48 2026

Simulation waveform





Without Wait-state

```
//
module apb_slave (
        input  logic        pclk,
      input  logic        rst_n,
      input  logic [31:0] paddr,
      input  logic        psel,
      input  logic        penable,
      input  logic        pwrite,
      input  logic [31:0] pwdata,
      output logic [31:0] prdata,
      output logic        pready,
      output logic        pslverr
```

```
);

logic [31:0] mem [0:255];
logic [1:0] apb_st;
localparam logic [1:0] SETUP = 2'b00, W_ENABLE = 2'b01, R_ENABLE = 2'b10;

always_ff @(posedge pclk or negedge rst_n) begin
        if (!rst_n) begin
                apb_st  <= SETUP;
            prdata  <= 32'd0;
            pready  <= 1'b1;
            pslverr <= 1'b0;
                for (int i = 0; i < 256; i++)
                        mem[i] <= i;
        end
        else begin
                case (apb_st)
                        SETUP: begin
                                prdata  <= 32'd0;
                                pslverr <= 1'b0;
                                pready  <= 1'b1;

                                if (psel && !penable) begin
                                        if (paddr > 255) begin
                                                pslverr <= 1'b1;
                                                apb_st <= SETUP;
                                        end
```

```
                else if (pwrite) begin
                        apb_st <= W_ENABLE;
                end
                else begin
                        apb_st <= R_ENABLE;
                end
        end
    end
    W_ENABLE: begin
        if (psel && penable && pwrite) begin
                mem[paddr] <= pwdata;
        end
        apb_st <= SETUP;
    end
    R_ENABLE: begin
        if (psel && penable && !pwrite) begin
                prdata <= mem[paddr];
        end
        apb_st <= SETUP;
    end
    default: begin
        apb_st  <= SETUP;
        prdata  <= 32'd0;
        pready  <= 1'b1;
        pslverr <= 1'b0;
    end
endcase
```

```
        end
end
endmodule
```

Testbench

```
//
module tb_apb;
logic       pclk;
logic       rst_n;
logic [31:0] paddr;
logic       psel;
logic       penable;
logic       pwrite;
logic [31:0] pwdata;
logic [31:0] prdata;
logic       pready;
logic       pslverr;


apb_slave dut
(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pwrite),.pwd
ata(pwdata),.prdata(prdata),.pready(pready),.pslverr(pslverr));


initial pclk = 0;
always #5 pclk = ~pclk;


initial begin
        rst_n = 0;
        #20 rst_n = 1;
end
```

```
task apb_write (input [31:0] addr, input [31:0] data);
begin
	@(posedge pclk);
	paddr   = addr;
	pwdata  = data;
	pwrite  = 1'b1;
	psel    = 1'b1;
	penable = 1'b0;

	@(posedge pclk);
	if (pslverr) begin
		$display("[%0t] WRITE ERROR: Addr=%0d Data=%h PSLVERR=%b", $time, addr, data, pslverr);
	end
	else begin
		penable = 1'b1;
		@(posedge pclk);
		$display("[%0t] WRITE: Addr=%0d Data=%h PSLVERR=%b", $time, addr, data, pslverr);
	end
	psel    = 1'b0;
	penable = 1'b0;
	pwrite  = 1'b0;
end
endtask

task apb_read (input [31:0] addr);
```

```
begin
        @(posedge pclk);
        paddr   = addr;
       pwrite  = 1'b0;
       psel    = 1'b1;
       penable = 1'b0;
        @(posedge pclk);
        if (pslverr) begin
                $display("[%0t] READ ERROR: Addr=%0d Data=%h PSLVERR=%b",
$time, addr, prdata,pslverr);
        end
        else begin
                penable = 1'b1;
                @(posedge pclk);
                $display("[%0t] READ: Addr=%0d Data=%h PSLVERR=%b", $time, addr,
prdata, pslverr);
        end
        psel    = 1'b0;
        penable = 1'b0;
end
endtask


initial begin
        psel    = 1'b0;
        penable = 1'b0;
        pwrite  = 1'b0;
        paddr   = 32'd0;
        pwdata  = 32'd0;
```

```
    wait(rst_n == 1);
    #10;
  apb_write(32'd10, 32'hAABBCCDD);
  apb_read(32'd10);
  apb_write(32'd300, 32'h11112222);
  apb_read(32'd300);
    #30 $finish;
end
initial begin
  $fsdbDumpfile("dump.fsdb");
  $fsdbDumpvars(0,tb_apb);
end
endmodule
```

Run-command

```
[b23n_sarif_727@prac3 apb_slave]$ vcs -sverilog apb_slave.sv apb_tb.sv -kdb -lca -debug_access+all -full64
[b23n_sarif_727@prac3 apb_slave]$ ./simv
[b23n_sarif_727@prac3 apb_slave]$ ./simv -verdi
```

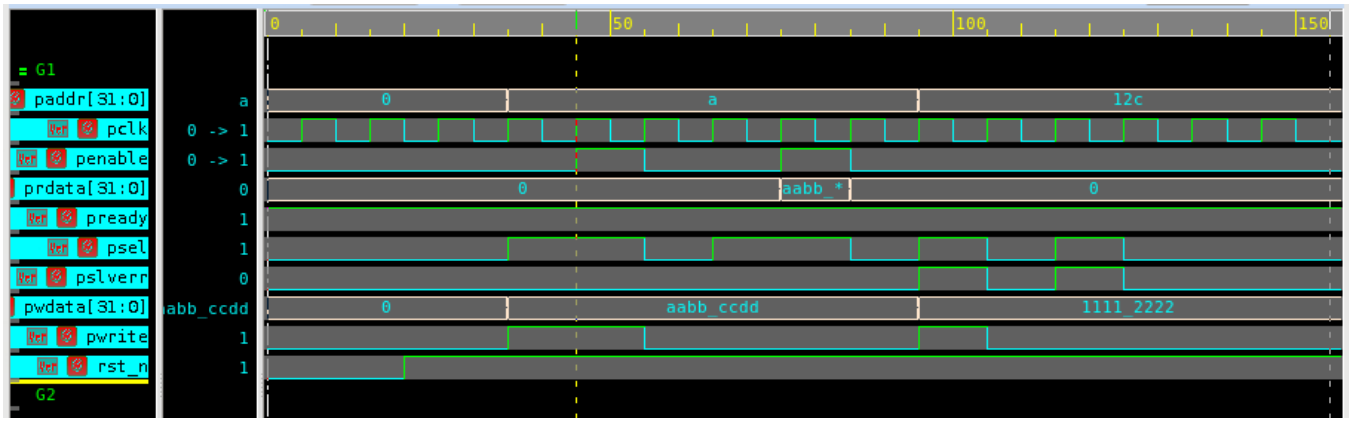
Simulation Results:

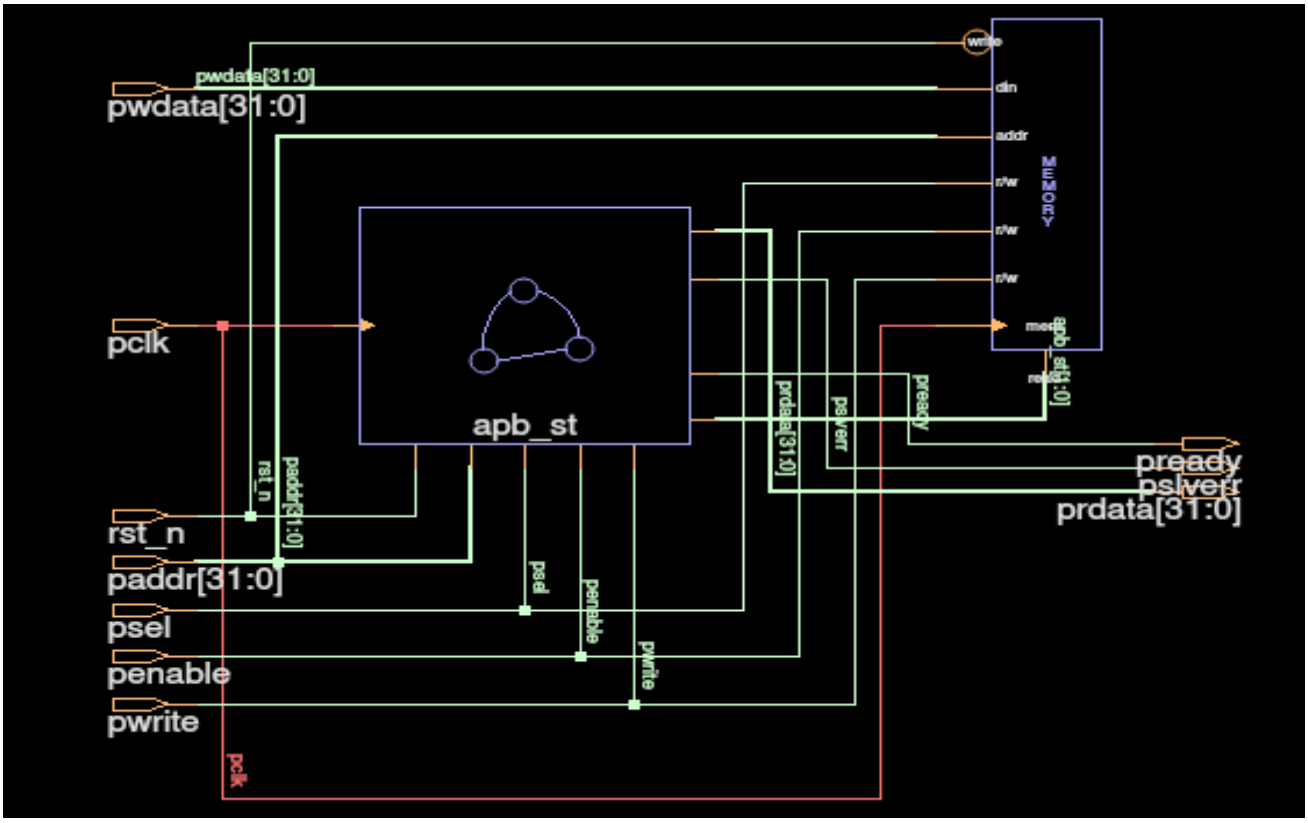
```
[55] WRITE: Addr=10 Data=aabbccdd PSLVERR=0
[85] READ: Addr=10 Data=aabbccdd PSLVERR=0
[105] WRITE ERROR: Addr=300 Data=11112222 PSLVERR=1
[125] READ ERROR: Addr=300 Data=00000000 PSLVERR=1
$finish called from file "apb_tb.sv", line 82.
$finish at simulation time                155
          V C S   S i m u l a t i o n   R e p o r t
```

Time: 155

CPU Time: 0.090 seconds; Data structure size: 0.0Mb

Tue Aug 18 10:04:28 2026





APB Protocol Verification using verilog

//

module apb_slave #(parameter integer WAIT_CYCLES = 2)(

input logic pclk,

input logic rst_n,

```
    input logic [31:0] paddr,
    input logic        psel,
    input logic        penable,
    input logic        pwrite,
    input logic [31:0] pwdata,
      output logic [31:0] prdata,
    output logic       pready,
    output logic       pslverr
);

logic [31:0] mem [0:255];
typedef enum logic [1:0] { SETUP = 2'b00, W_ENABLE = 2'b01, R_ENABLE = 2'b10 } state_t;
state_t apb_st;
integer wait_count;

always_ff @(posedge pclk or negedge rst_n) begin
        if (!rst_n) begin
                apb_st     <= SETUP;
                prdata     <= 32'd0;
            pready     <= 1'b1;
            pslverr    <= 1'b0;
            wait_count <= 0;

                for (int i = 0; i < 256; i++)
                        mem[i] <= i;
        end
```

```
else begin
        case (apb_st)
                SETUP:
                begin
                        pready  <= 1'b0;
                        pslverr <= 1'b0;
                        prdata  <= 32'd0;

                        if (psel && !penable) begin
                                if (paddr > 32'd255) begin
                                        pslverr <= 1'b1;
                                        pready  <= 1'b1;
                                        apb_st <= SETUP;
                                end
                                else begin
                                        wait_count <= WAIT_CYCLES;
                                        if (pwrite)
                                                apb_st <= W_ENABLE;
                                        else
                                                apb_st <= R_ENABLE;
                                end
                        end
                end
                W_ENABLE:
                begin
                        if (psel && penable && pwrite) begin
                                if (wait_count > 0) begin
```

```
                pready     <= 1'b0;
                wait_count <= wait_count - 1;
            end
            else begin
                pready <= 1'b1;
                if (paddr <= 32'd255)
                    mem[paddr] <= pwdata;
                apb_st <= SETUP;
            end
        end
    end
    R_ENABLE:
    begin
        if (psel && penable && !pwrite) begin
            if (wait_count > 0) begin
                pready     <= 1'b0;
                wait_count <= wait_count - 1;
            end
            else begin
                pready <= 1'b1;
                if (paddr <= 32'd255)
                    prdata <= mem[paddr];
                apb_st <= SETUP;
            end
        end
    end
    default:
```

```
                begin
                        apb_st     <= SETUP;
                        pready     <= 1'b1;
                        pslverr    <= 1'b0;
                        prdata     <= 32'd0;
                        wait_count <= 0;
                end
            endcase
        end
end
endmodule
```

Testbench

```
//
module tb_apb;
logic        pclk;
logic        rst_n;
logic [31:0] paddr;
logic        psel;
logic        penable;
logic        pwrite;
logic [31:0] pwdata;

logic [31:0] prdata;
logic        pready;
logic        pslverr;
```

```
apb_slave #(.WAIT_CYCLES(0))
dut(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pwrite),.p
wdata(pwdata),.prdata(prdata),.pready(pready),.pslverr(pslverr));


initial begin
        pclk = 1'b0;
        forever #5 pclk = ~pclk;
end


initial begin
        rst_n = 1'b0;
        #20;
        rst_n = 1'b1;
end


task automatic apb_write(input logic [31:0] addr, input logic [31:0] data);
begin
        @(posedge pclk);
        #1;
      paddr   = addr;
      pwdata  = data;
      pwrite  = 1'b1;
      psel    = 1'b1;
      penable = 1'b0;
        $display("[%0t] WRITE SETUP : ADDR=%0d DATA=%h", $time, addr, data);


      @(posedge pclk);
      #1;
```

```
        penable = 1'b1;
          $display("[%0t] WRITE ACCESS: PREADY=%b", $time, pready);

        while (!pready) begin
                @(posedge pclk);
                #1;
                $display("[%0t] WRITE WAIT  : PREADY=%b", $time, pready);
          end
          $display("[%0t] WRITE DONE  : ADDR=%0d DATA=%h PSLVERR=%b",
$time, addr, data, pslverr);

          @(posedge pclk);
        #1;
        psel    = 1'b0;
        penable = 1'b0;
        pwrite  = 1'b0;
end
endtask

task automatic apb_read(input logic [31:0] addr, output logic [31:0] data);
begin
          @(posedge pclk);
        #1;
        paddr   = addr;
        pwrite  = 1'b0;
        psel    = 1'b1;
        penable = 1'b0;
          $display("[%0t] READ SETUP  : ADDR=%0d", $time, addr);
```

```
    @(posedge pclk);
  #1;
  penable = 1'b1;
  $display("[%0t] READ ACCESS : PREADY=%b", $time, pready);

  while (!pready) begin
      @(posedge pclk);
      #1;
      $display("[%0t] READ WAIT : PREADY=%b", $time, pready);
  end

  data = prdata;
  $display("[%0t] READ DONE : ADDR=%0d DATA=%h PSLVERR=%b", $time,
addr, data, pslverr);

  @(posedge pclk);
  #1;
  psel    = 1'b0;
  penable = 1'b0;
end
endtask

logic [31:0] rdata;

initial begin
    psel    = 1'b0;
  penable = 1'b0;
```

```
        pwrite  = 1'b0;
        paddr   = 32'd0;
        pwdata  = 32'd0;
        rdata   = 32'd0;

        wait (rst_n == 1'b1);
        #10;

        $display("\n========================================");
        $display("TEST 1: RESET");
        $display("========================================");

        @(posedge pclk);
        #1;
          $display("[%0t] RESET CHECK: PREADY=%b PSLVERR=%b",$time, pready,
pslverr);

        $display("\n========================================");
        $display("TEST 2: NORMAL WRITE");
        $display("========================================");

        apb_write(32'd10, 32'hAABBCCDD);
        $display("\n========================================");
        $display("TEST 3: NORMAL READ");
        $display("========================================");

        apb_read(32'd10, rdata);
        if (rdata == 32'hAABBCCDD)
```

```
    $display("READBACK PASS");
else
    $display("READBACK FAIL: DATA=%h", rdata);

$display("\n========================================");
$display("TEST 4: WRITE WITH WAIT STATES");
$display("========================================");

apb_write(32'd20, 32'h12345678);
$display("\n========================================");
$display("TEST 5: READ WITH WAIT STATES");
$display("========================================");

apb_read(32'd20, rdata);
if (rdata == 32'h12345678)
    $display("READBACK PASS");
else
    $display("READBACK FAIL: DATA=%h", rdata);

$display("\n========================================");
$display("TEST 6: INVALID WRITE");
$display("========================================");

apb_write(32'd300, 32'hDEADBEEF);
$display("\n========================================");
$display("TEST 7: INVALID READ");
$display("========================================");
```

```
        apb_read(32'd300, rdata);
        #20;
        $display("\n=======================================");
        $display("ALL TESTS COMPLETED");
        $display("=======================================");

        $finish;
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb_apb);
end
endmodule
```

Simulation waveform

```
=======================================
TEST 1: RESET
=======================================
[36] RESET CHECK: PREADY=0 PSLVERR=0

=======================================
TEST 2: NORMAL WRITE
=======================================
[46] WRITE SETUP : ADDR=10 DATA=aabbccdd
[56] WRITE ACCESS: PREADY=0
[66] WRITE WAIT  : PREADY=1
[66] WRITE DONE  : ADDR=10 DATA=aabbccdd PSLVERR=0
```

```
========================================
TEST 3: NORMAL READ
========================================
[86] READ SETUP  : ADDR=10
[96] READ ACCESS : PREADY=0
[106] READ WAIT : PREADY=1
[106] READ DONE : ADDR=10 DATA=aabbccdd PSLVERR=0
READBACK PASS

========================================
TEST 4: WRITE WITH WAIT STATES
========================================
[126] WRITE SETUP : ADDR=20 DATA=12345678
[136] WRITE ACCESS: PREADY=0
[146] WRITE WAIT  : PREADY=1
[146] WRITE DONE  : ADDR=20 DATA=12345678 PSLVERR=0

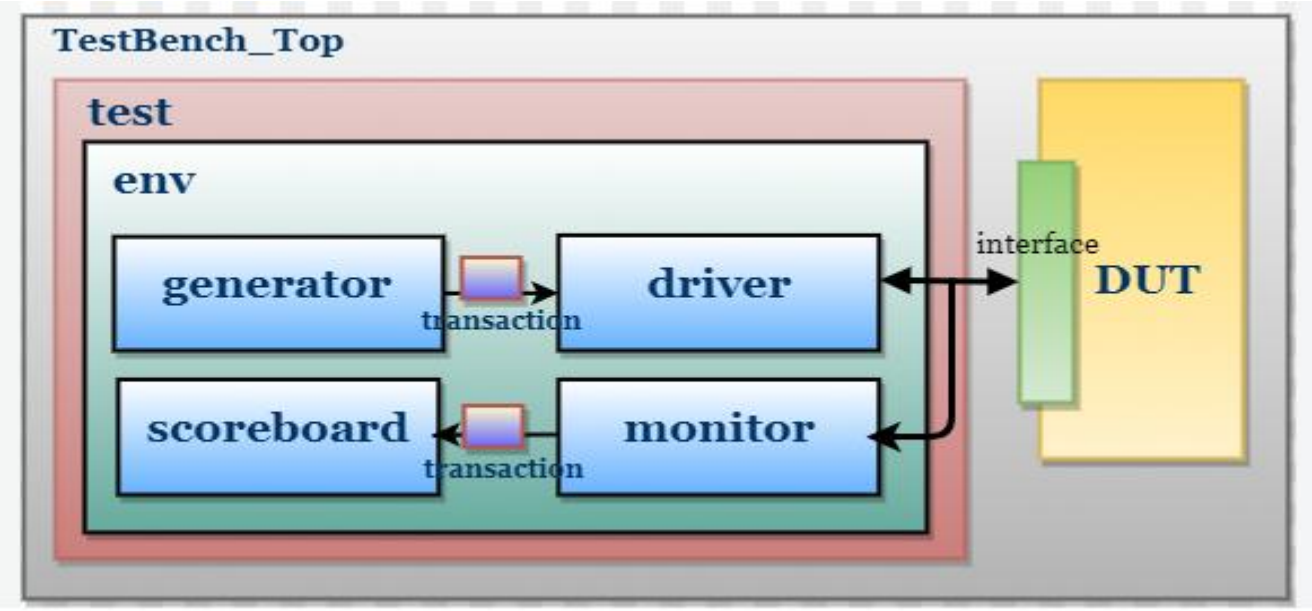
========================================
TEST 5: READ WITH WAIT STATES
========================================
[166] READ SETUP  : ADDR=20
[176] READ ACCESS : PREADY=0
[186] READ WAIT : PREADY=1
[186] READ DONE : ADDR=20 DATA=12345678 PSLVERR=0
READBACK PASS
```

```
========================================
TEST 6: INVALID WRITE
========================================
[206] WRITE SETUP : ADDR=300 DATA=deadbeef
[216] WRITE ACCESS: PREADY=1
[216] WRITE DONE  : ADDR=300 DATA=deadbeef PSLVERR=1

========================================
TEST 7: INVALID READ
========================================
[236] READ SETUP  : ADDR=300
[246] READ ACCESS : PREADY=1
[246] READ DONE : ADDR=300 DATA=00000000 PSLVERR=1

========================================
ALL TESTS COMPLETED
========================================
$finish called from file "apb_tb.sv", line 156.
$finish at simulation time                276
           V C S   S i m u l a t i o n   R e p o r t
Time: 276
CPU Time:      0.090 seconds;       Data structure size:   0.0Mb
Wed Aug 19 13:51:42 2026
```

System Verilog Architecture Diagram:-



This diagram represents a **non-UVM SystemVerilog verification architecture**. It shows how stimulus is generated, applied to the DUT, monitored, and checked.

1. Main blocks

TestBench_Top

This is the **top-level module**.

It contains:

- test
- DUT
- Interface

It connects the testbench to the actual DUT.

Test

The test controls the overall verification.

Its main job is to:

- Create the environment (env)
- Start the test
- Control when the simulation starts/ends

In your diagram:

TestBench_Top
|
test
|
env

Environment (env)

The environment is a container that contains the main verification components:

env
├── generator
├── driver
├── monitor
└── scoreboard

It connects these components together.

Generator

The generator creates **test transactions**.

For example, for a half adder:

a = 0
b = 1

The generator sends this transaction to the driver through a **mailbox**.

Generator
|
| transaction
↓
Driver

Driver

The driver converts the transaction into **DUT input signals**.

For example:

Transaction:
a = 0
b = 1

Driver drives:

interface.a = 0
interface.b = 1

So the driver is the component that actually **applies stimulus to the DUT**.

Interface

The interface is the connection between the **testbench and DUT**.

It carries signals such as:

a
b
sum
carry

The driver uses the interface to send inputs to the DUT.

The monitor uses the same interface to observe DUT outputs.

DUT

DUT means **Design Under Test**.

For your example, it could be:

Half Adder

The DUT receives:

a
b

and produces:

sum
carry

Monitor

The monitor observes the signals coming from the DUT through the interface.

For example:

a = 0
b = 1
sum = 1
carry = 0

It converts these observed signals back into a **transaction** and sends it to the scoreboard.

Monitor
  |
  | transaction
  ↓
Scoreboard

The monitor **does not drive the DUT**. It only observes.

Scoreboard

The scoreboard checks whether the DUT output is correct.

For a half adder:

Expected:
sum = a ^ b
carry = a & b

If:

a = 0
b = 1

then:

Expected sum = 1
Expected carry = 0

The scoreboard compares this expected result with the actual DUT output.

Expected ≠ Actual → FAIL

Expected = Actual → PASS

Complete data flow

The most important thing to remember for an interview is this flow:

Generator
  |
  | Transaction
  ↓
Driver
  |
  | Signals
  ↓
Interface
  |
  ↓
DUT
  |
  | Output Signals
  ↓
Interface
  |
  ↓
Monitor
  |
  | Transaction
  ↓
Scoreboard
  |
  ↓
PASS / FAIL

In one sentence: **Generator generates the transaction, driver drives it to the DUT through the interface, monitor observes the DUT response, and scoreboard compares the actual response with the expected response.**

How to explain this diagram in an interview

If interviewer shows you this diagram and asks:

"Explain your SystemVerilog verification architecture."

You can say:

"This is a SystemVerilog-based verification architecture. At the top level, we have the testbench top, which contains the test, environment, interface, and DUT.

The test creates and controls the environment. The environment contains the generator, driver, monitor, and scoreboard.

The generator generates stimulus in the form of transactions and sends them to the driver using a mailbox. The driver converts these transactions into DUT input signals and drives them through the interface.

The DUT processes the inputs and generates the outputs. The monitor observes the DUT inputs and outputs through the interface and converts them into transactions. These transactions are sent to the scoreboard.

Finally, the scoreboard calculates the expected result and compares it with the actual DUT output. If they match, the test passes; otherwise, it reports an error."

Short version to memorize

If the interviewer wants a short explanation, say:

"The generator creates transactions, the driver drives them to the DUT through the interface, the monitor observes the DUT response, and the scoreboard compares the actual output with the expected output. The test and environment control and organize all these components."

That is a **good interview-ready explanation** for this architecture.

1.0 System Verilog Architecture Code for half adder

Design Code

```
// DUT of half adder
module half_adder (
        input  logic a,
        input  logic b,
        output logic sum,
```

```
        output logic carry
);

assign sum   = a ^ b;
assign carry = a & b;
endmodule
```

Testbench Cmponent

// interface

```
interface half_adder_if(input logic clk);
        logic a;
        logic b;
        logic sum;
        logic carry;

        clocking drv_cb @(posedge clk);
                default input #1step output #1ns;
                output a,b;
        endclocking

        clocking mon_cb @(posedge clk);
                default input #1step;
                input a,b,sum,carry;
        endclocking

endinterface
```

Packet

```
// packet of half adder
class packet;
	rand bit a;
	rand bit b;
	bit sum;
	bit carry;

	function void display(string name = "PACKET");
		$display("[%s] a=%0b b=%0b sum=%0b carry=%0b",name,a,b,sum,carry);
	endfunction

endclass
```

Generator

```
//
class generator;
	packet pkt;
	mailbox #(packet) gen2drv;

	function new(mailbox #(packet) gen2drv);
		this.gen2drv = gen2drv;
	endfunction

	task run();
		$display("[GEN] :Test Cases HAS BEEN START Generated");
		repeat(10) begin
```

```
                pkt = new();
                assert(pkt.randomize());
                gen2drv.put(pkt);
                $display("[GENERATOR] a=%0b b=%0b",pkt.a, pkt.b);
        end
        $display("[GEN] :All Test Cases HAS BEEN Completed");
    endtask

endclass
```

Driver

```
//
class driver;
    mailbox #(packet) gen2drv;
   virtual half_adder_if vif;
    packet pkt;

    function new(mailbox #(packet) gen2drv, virtual half_adder_if vif);
        this.gen2drv = gen2drv;
      this.vif     = vif;
    endfunction

    task run();
        forever begin
            gen2drv.get(pkt);
            @(vif.drv_cb);
            vif.drv_cb.a <= pkt.a;
```

```
                vif.drv_cb.b <= pkt.b;
                $display("[DRIVER] a=%0b b=%0b",pkt.a, pkt.b);
            end
        endtask

endclass
```

Monitor

```
//
class monitor;
        mailbox #(packet) mon2sb;
        mailbox #(packet) mon2cov;
        virtual half_adder_if vif;
        packet pkt;

        function new(mailbox #(packet) mon2sb, mailbox #(packet) mon2cov, virtual half_adder_if vif);
            this.mon2sb  = mon2sb;
            this.mon2cov = mon2cov;
            this.vif     = vif;
        endfunction

        task run();
            forever begin
                @(vif.mon_cb);
                pkt = new();
                pkt.a     = vif.mon_cb.a;
```

```
                pkt.b     = vif.mon_cb.b;
                pkt.sum   = vif.mon_cb.sum;
                pkt.carry = vif.mon_cb.carry;
                $display("[MONITOR] a=%0b b=%0b sum=%0b carry=%0b",pkt.a, pkt.b, pkt.sum, pkt.carry);
                mon2sb.put(pkt);
                mon2cov.put(pkt);
        end
    endtask

endclass
```

Scoreboard

```
//
class scoreboard;
    mailbox #(packet) mon2sb;
    packet pkt;

    function new(mailbox #(packet) mon2sb);
        this.mon2sb = mon2sb;
    endfunction

    task run();
        bit expected_sum;
        bit expected_carry;

        forever begin
```

```
                mon2sb.get(pkt);


                expected_sum   = pkt.a ^ pkt.b;
            expected_carry = pkt.a & pkt.b;


                if ((pkt.sum == expected_sum) && (pkt.carry == expected_carry))
begin
                        $display("[SCOREBOARD] PASS: a=%0b b=%0b sum=%0b
carry=%0b", pkt.a, pkt.b, pkt.sum, pkt.carry);
                end
                else begin
                        $error("[SCOREBOARD] FAIL: a=%0b b=%0b
expected_sum=%0b actual_sum=%0b expected_carry=%0b actual_carry=%0b", pkt.a,
pkt.b, expected_sum, pkt.sum,expected_carry, pkt.carry);
                end
            end
        endtask


endclass
```

Environment

```
//
class environment;
        mailbox #(packet) gen2drv;
       mailbox #(packet) mon2sb;
        mailbox #(packet) mon2cov;
        virtual half_adder_if vif;
```

```
generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;
coverage cov;

function new(virtual half_adder_if vif);
        this.vif = vif;

        gen2drv = new();
     mon2sb  = new();
        mon2cov = new();

        gen = new(gen2drv);
        drv = new(gen2drv, vif);
        mon = new(mon2sb, mon2cov, vif);
        sb  = new(mon2sb);
        cov = new();
endfunction

task run();
        fork
                gen.run();
                drv.run();
                mon.run();
                sb.run();
                cov.run(mon2cov);
```

```
            join_none
        endtask

endclass
```

Test

```
//
class test;
        environment env;

        function new(virtual half_adder_if vif);
            env = new(vif);
        endfunction

        task run();
            env.run();
            #120;
        endtask

endclass
```

Testbench_top

```
//
`include "half_adder.sv"
`include "half_adder_if.sv"
`include "packet.sv"
`include "generator.sv"
```

```
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
`include "coverage.sv"
`include "environment.sv"
`include "test.sv"

module tb_top;

logic clk;

initial begin
        clk = 0;
        forever #5 clk = ~clk;
end

half_adder_if vif(clk);
half_adder DUT (.a(vif.a),.b(vif.b),.sum(vif.sum),.carry(vif.carry));

test t;
initial begin
        vif.a = 0;
      vif.b = 0;

        t = new(vif);
      t.run();
      $display(" TEST has been COMPLETED ");
```

```
        $finish;


    end
initial begin
          $fsdbDumpfile("half_adder.fsdb");
          $fsdbDumpvars(0,tb_top);
end

endmodule
```

Coverage

```
//
class coverage;
          packet pkt;

          covergroup half_adder_cg;

        cp_a: coverpoint pkt.a {
            bins a_0 = {0};
            bins a_1 = {1};
        }



        cp_b: coverpoint pkt.b {
            bins b_0 = {0};
            bins b_1 = {1};
        }
```

```
        cp_sum: coverpoint pkt.sum {
            bins sum_0 = {0};
            bins sum_1 = {1};
        }

        cp_carry: coverpoint pkt.carry {
            bins carry_0 = {0};
            bins carry_1 = {1};
        }

        cross_ab: cross cp_a, cp_b;
        cross_a_b_sum: cross cp_a, cp_b, cp_sum;
        cross_all: cross cp_a, cp_b, cp_sum, cp_carry;
    endgroup

    function new();
            half_adder_cg = new();
    endfunction

    task run(mailbox #(packet) mon2cov);
            forever begin
                    mon2cov.get(pkt);
                    half_adder_cg.sample();
            end
    endtask
endclass
```

Simulation Run Command:

[b23n_sarif_727@prac3 half_adder]$ ls

coverage.sv environment.sv half_adder_if.sv novas_dump.log scoreboard.sv simv.vdb ucli.key

csrc generator.sv half_adder.sv novas.rc simv tb_top.sv vdCov.conf

driver.sv half_adder.fsdb monitor.sv packet.sv simv.daidir test.sv vdCovLog

[b23n_sarif_727@prac3 half_adder]$ vcs -sverilog tb_top.sv -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 half_adder]$ ./simv

Simulation Output

[GEN] :Test Cases HAS BEEN START Generated

[GENERATOR] a=0 b=1

[GENERATOR] a=1 b=1

[GENERATOR] a=0 b=1

[GENERATOR] a=1 b=0

[GENERATOR] a=0 b=1

[GENERATOR] a=1 b=1

[GENERATOR] a=1 b=0

[GENERATOR] a=0 b=1

[GENERATOR] a=0 b=0

[GENERATOR] a=0 b=1

[GEN] :All Test Cases HAS BEEN Completed

[DRIVER] a=0 b=1

[MONITOR] a=0 b=0 sum=0 carry=0

[SCOREBOARD] PASS: a=0 b=0 sum=0 carry=0

[DRIVER] a=1 b=1

[MONITOR] a=0 b=1 sum=1 carry=0

[SCOREBOARD] PASS: a=0 b=1 sum=1 carry=0

[DRIVER] a=0 b=1

[MONITOR] a=1 b=1 sum=0 carry=1

[SCOREBOARD] PASS: a=1 b=1 sum=0 carry=1

[DRIVER] a=1 b=0

[MONITOR] a=0 b=1 sum=1 carry=0

[SCOREBOARD] PASS: a=0 b=1 sum=1 carry=0

[DRIVER] a=0 b=1

[MONITOR] a=1 b=0 sum=1 carry=0

[SCOREBOARD] PASS: a=1 b=0 sum=1 carry=0

[DRIVER] a=1 b=1

[MONITOR] a=0 b=1 sum=1 carry=0

[SCOREBOARD] PASS: a=0 b=1 sum=1 carry=0

[DRIVER] a=1 b=0

[MONITOR] a=1 b=1 sum=0 carry=1

[SCOREBOARD] PASS: a=1 b=1 sum=0 carry=1

[DRIVER] a=0 b=1

[MONITOR] a=1 b=0 sum=1 carry=0

[SCOREBOARD] PASS: a=1 b=0 sum=1 carry=0

[DRIVER] a=0 b=0

[MONITOR] a=0 b=1 sum=1 carry=0

[SCOREBOARD] PASS: a=0 b=1 sum=1 carry=0

[DRIVER] a=0 b=1

[MONITOR] a=0 b=0 sum=0 carry=0

[SCOREBOARD] PASS: a=0 b=0 sum=0 carry=0

[MONITOR] a=0 b=1 sum=1 carry=0

[SCOREBOARD] PASS: a=0 b=1 sum=1 carry=0

[MONITOR] a=0 b=1 sum=1 carry=0

[SCOREBOARD] PASS: a=0 b=1 sum=1 carry=0

TEST has been COMPLETED

$finish called from file "tb_top.sv", line 33.

$finish at simulation time 120

V C S S i m u l a t i o n R e p o r t

Time: 120

CPU Time: 0.210 seconds; Data structure size: 0.0Mb

Fri Aug 28 19:32:48 2026

Technology node 32nm means

A **32-nanometer (32nm) technology node** refers to a specific generation of semiconductor manufacturing where microchips are built with features scaled to the 32nm level.

Historical definition: It originally represented the physical gate length or minimum feature size of a transistor.